

SELF-HOSTED ACADEMY

Nom du Cours : Introduction à l'API Vulkan

COURS ACADÉMIQUE COMPLET

Ce cours académique introduit les concepts fondamentaux de Vulkan, une API graphique et de calcul de bas niveau et haute performance. Les étudiants apprendront à initialiser l'API, configurer la pipeline graphique et afficher un premier triangle.

Généré par SelfLearned • Technologie AI-Driven

MODULE 1

Fondations et Initialisation de Vulkan

Module 1.1 : Architecture de Vulkan, Instance et Périphériques

1. Introduction Conceptuelle

1.1 Qu'est-ce qu'une API Graphique ?

Pour comprendre Vulkan, il faut d'abord comprendre le rôle d'une API (Application Programming Interface) graphique. Votre ordinateur possède un processeur central (CPU) et un processeur graphique (GPU). Le CPU est excellent pour les tâches séquentielles complexes (logique du jeu, physique, IA), tandis que le GPU est un monstre de calcul parallèle, conçu pour traiter des millions de pixels et de polygones simultanément.

Une API graphique est le pont de communication entre ces deux processeurs. Sans elle, vous devriez écrire du code machine spécifique pour chaque carte graphique existante (Nvidia, AMD, Intel, Apple). L'API standardise cette communication.

```
+-----+          API Graphique          +-----+
|  Votre Code  | -----> |   Pilote   |
| (C++ / CPU)  | (Vulkan / OpenGL) | (Nvidia/AMD/GPU) |
+-----+                                     +-----+
```

1.2 L'évolution : D'OpenGL à Vulkan

Pendant des décennies, OpenGL a été la norme multiplateforme. OpenGL a été conçu à une époque (1992) où les cartes graphiques étaient très différentes et où les processeurs n'avaient qu'un seul cœur.

- **Le problème d'OpenGL** : Il fonctionne comme une "boîte noire" ou une machine à états globale. Le pilote (driver) de la carte graphique fait énormément de travail en coulisses : il devine ce que le programmeur veut faire, compile les shaders au dernier moment,

gère la mémoire de manière opaque et force toutes les commandes à passer par un seul thread CPU. Cela crée un goulot d'étranglement majeur.

- **La solution Vulkan** : Introduit en 2016 par le groupe Khronos, Vulkan est une API dite de **bas niveau** (low-overhead). Vulkan supprime la quasi-totalité des abstractions du pilote. C'est vous, le développeur, qui prenez le contrôle total du GPU. Vous devez allouer explicitement la mémoire, gérer la synchronisation entre les tâches et enregistrer les commandes graphiques en parallèle sur plusieurs cœurs CPU.

Caractéristique	OpenGL	Vulkan
Niveau d'abstraction	Élevé (Simple mais rigide)	Bas (Complexe mais ultra-performant)
Gestion de la mémoire	Automatique (gérée par le pilote)	Explicite (gérée par le développeur)
Multi-threading CPU	Très difficile / Inefficace	Conçu nativement pour cela
Contrôle des erreurs	Permanent (ralentit l'exécution)	Absent par défaut (via couches de validation)

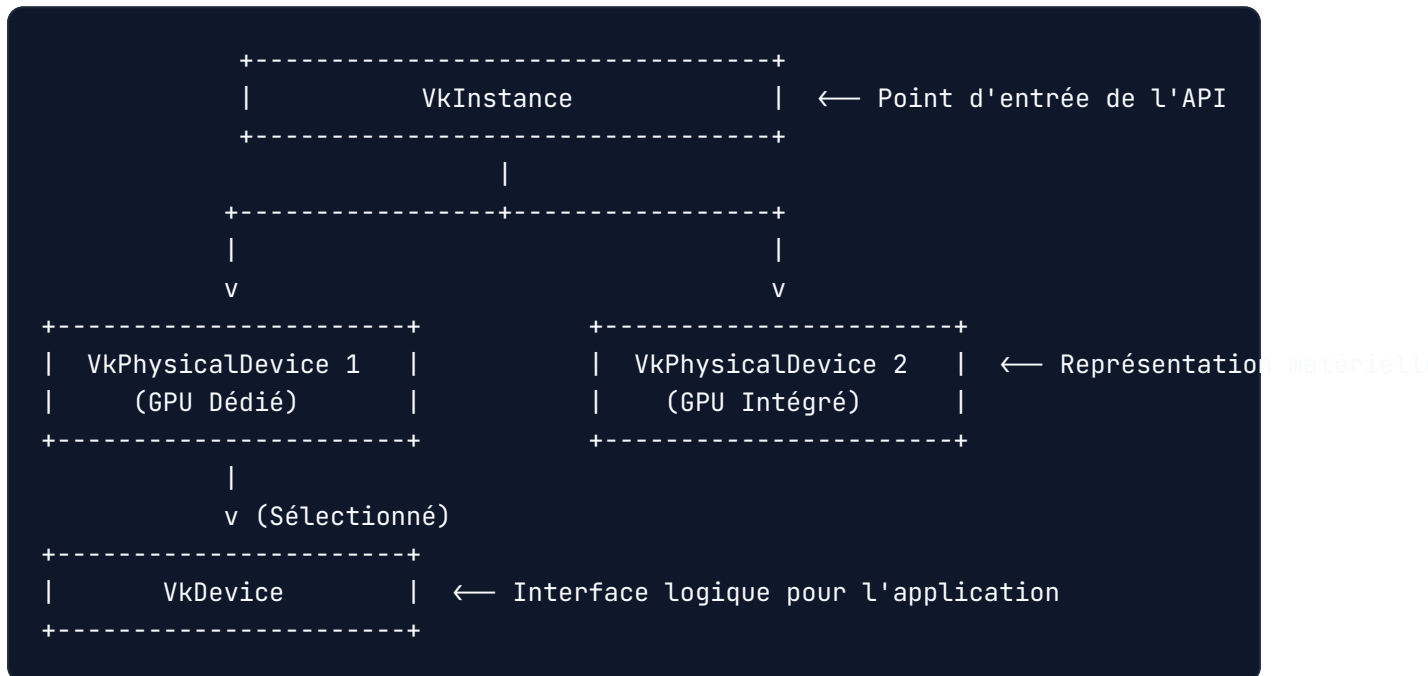
1.3 Pourquoi Vulkan est-il difficile mais indispensable ?

Vulkan respecte le principe : "Vous payez uniquement pour ce que vous utilisez". Le pilote ne fait aucune vérification d'erreur en mode production pour maximiser les performances. Si vous faites une erreur (par exemple, utiliser une ressource mémoire mal allouée), Vulkan ne lèvera pas d'exception propre : votre application plantera ou affichera des artefacts visuels aléatoires.

Ce cours va vous guider pas à pas. Même si l'initialisation de Vulkan demande des centaines de lignes de code là où OpenGL n'en demandait que dix, chaque ligne a un but précis que nous allons décortiquer.

2. Fondations Théoriques

Avant d'écrire notre premier code C++, nous devons comprendre les quatre piliers de l'initialisation de Vulkan.



2.1 L'Instance Vulkan (`VkInstance`)

La **`VkInstance`** est le point de départ de toute application Vulkan. Elle initialise la bibliothèque Vulkan et permet d'établir le lien entre votre application et le runtime Vulkan installé sur le système d'exploitation.

Lors de sa création, vous devez spécifier : * Les métadonnées de votre application (nom, version). * Les extensions globales que vous souhaitez utiliser (par exemple, l'extension nécessaire pour afficher une fenêtre à l'écran). * Les couches de validation (Validation Layers) que vous souhaitez activer.

2.2 Les Couches de Validation (Validation Layers)

Comme mentionné, Vulkan par défaut ne fait aucune vérification d'erreur pour des raisons de performance. Pour éviter de coder à l'aveugle, Vulkan utilise un système modulaire appelé Validation Layers.

Ce sont des bibliothèques externes qui s'insèrent entre votre application et le pilote Vulkan. Elles interceptent chaque appel API pour :

- * Vérifier que les paramètres passés sont valides.
- * Détecter les fuites de mémoire GPU.
- * S'assurer que vous respectez les règles de synchronisation.
- * Générer des rapports d'erreur détaillés dans la console.

Règle d'or : Activez toujours les couches de validation en mode *Debug* (développement) et désactivez-les impérativement en mode *Release* (production) pour retrouver les performances maximales.

2.3 Le Périphérique Physique (`VkPhysicalDevice`)

Une fois l'instance créée, vous devez interroger le système pour savoir quelles cartes graphiques sont disponibles. Vulkan représente chaque carte graphique physique par un objet `VkPhysicalDevice`.

Il peut y en avoir plusieurs sur un même ordinateur (par exemple, un processeur Intel avec un GPU intégré et une carte graphique dédiée Nvidia ou AMD). Votre code devra lister ces périphériques, interroger leurs caractéristiques (limites de mémoire, fonctionnalités supportées) et choisir le plus adapté.

2.4 Le Périphérique Logique (`VkDevice`)

Après avoir sélectionné un périphérique physique (le matériel), vous devez créer un périphérique logique (`VkDevice`).

Le périphérique logique est l'interface logicielle que vous utiliserez pour créer toutes les ressources futures (les textures, les buffers de géométrie, les pipelines de rendu). C'est à ce moment que vous spécifiez précisément quelles fonctionnalités matérielles vous allez activer (comme le *Ray Tracing* ou le support des textures géantes).

3. Implémentation Pratique Pas-à-Pas

Nous allons maintenant écrire le code C++ pour initialiser ces composants.

Rappels de C++ pour Vulkan

1. **Gestion de la mémoire** : Vulkan utilise massivement des structures C. Pour éviter les comportements indéterminés, nous devons toujours initialiser nos structures à zéro. En C++, nous utiliserons l'initialisation par accolades `{}` .
2. **Le pattern de création Vulkan** : Presque tous les objets Vulkan sont créés via le même schéma :
 - Création d'une structure de configuration : `Vk[NomDeLObjet]CreateInfo` .
 - Appel de la fonction de création : `vkCreate[NomDeLObjet](parent, &createInfo, allocateurs, &objet)` .
3. **Le pattern du double appel (Two-Call Pattern)** : Pour récupérer des listes d'éléments de taille inconnue (comme la liste des GPU disponibles), Vulkan utilise un double appel de fonction :
 - *Premier appel* : On passe un pointeur nul pour récupérer le nombre d'éléments.
 - *Allocation* : On alloue un tableau (`std::vector`) de cette taille.
 - *Second appel* : On passe le pointeur vers notre tableau pour le remplir.

Code Source Complet : `VulkanApp.hpp` et `VulkanApp.cpp`

Voici l'implémentation complète et fonctionnelle d'une classe C++ encapsulant l'initialisation de base de Vulkan.

Fichier : VulkanApp.hpp


```

#pragma once

#include <vulkan/vulkan.h>
#include <vector>
#include <iostream>
#include <stdexcept>
#include <cstring>

class VulkanApp {
public:
    void run() {
        initVulkan();
        mainLoop();
        cleanup();
    }

private:
    // Variables membres pour stocker nos objets Vulkan
    VkInstance instance = VK_NULL_HANDLE;
    VkDebugUtilsMessengerEXT debugMessenger = VK_NULL_HANDLE;
    VkPhysicalDevice physicalDevice = VK_NULL_HANDLE;
    VkDevice device = VK_NULL_HANDLE;

    // Configuration des Couches de Validation
    const bool enableValidationLayers = true;
    const std::vector<const char*> validationLayers = {
        "VK_LAYER_KHRONOS_validation"
    };

    void initVulkan() {
        createInstance();
        setupDebugMessenger();
        pickPhysicalDevice();
        createLogicalDevice();
    }

    void mainLoop() {
        std::cout << "Boucle principale active. Appuyez sur Entree pour quitter..." << std::endl;
        std::cin.get();
    }

    void cleanup() {
        // L'ordre de destruction est CRUCIAL. C'est l'inverse de la création.
        if (device != VK_NULL_HANDLE) {
            vkDestroyDevice(device, nullptr);
        }

        if (enableValidationLayers && debugMessenger != VK_NULL_HANDLE) {

```

```

        destroyDebugUtilsMessengerEXT(instance, debugMessenger, nullptr);
    }

    if (instance != VK_NULL_HANDLE) {
        vkDestroyInstance(instance, nullptr);
    }
}

// Fonctions d'initialisation détaillées ci-dessous
void createInstance();
bool checkValidationLayerSupport();
std::vector<const char*> getRequiredExtensions();
void setupDebugMessenger();
void populateDebugMessengerCreateInfo(VkDebugUtilsMessengerCreateInfoEXT& createInfo);
void pickPhysicalDevice();
bool isDeviceSuitable(VkPhysicalDevice device);
void createLogicalDevice();

// Fonctions utilitaires pour charger dynamiquement les extensions de debug
VkResult createDebugUtilsMessengerEXT(VkInstance instance,
                                       const VkDebugUtilsMessengerCreateInfoEXT* pCreateInfo,
                                       const VkAllocationCallbacks* pAllocator,
                                       VkDebugUtilsMessengerEXT* pDebugMessenger);
void destroyDebugUtilsMessengerEXT(VkInstance instance,
                                   VkDebugUtilsMessengerEXT debugMessenger,
                                   const VkAllocationCallbacks* pAllocator);
};

```

Fichier : VulkanApp.cpp

```
#include "VulkanApp.hpp"
```

```
// Callback de debug : Cette fonction sera appelée par Vulkan lorsqu'un avertissement ou un
static VKAPI_ATTR VkBool32 VKAPI_CALL debugCallback(
    VkDebugUtilsMessageSeverityFlagBitsEXT messageSeverity,
    VkDebugUtilsMessageTypeFlagsEXT messageType,
    const VkDebugUtilsMessengerCallbackDataEXT* pCallbackData,
    void* pUserData) {

    std::cerr << "[Validation Layer] : " << pCallbackData->pMessage << std::endl;
    return VK_FALSE; // Doit toujours retourner VK_FALSE
}
```

```
// 1. Création de l'Instance
```

```
void VulkanApp::createInstance() {
    if (enableValidationLayers && !checkValidationLayerSupport()) {
        throw std::runtime_error("Couches de validation demandées mais non disponibles !");
    }
}
```

```
// Informations sur l'application (Optionnel mais recommandé pour les pilotes)
VkApplicationInfo appInfo{};
appInfo.sType = VK_STRUCTURE_TYPE_APPLICATION_INFO; // Obligatoire pour identifier la s
appInfo.pNext = nullptr;
appInfo.pApplicationName = "Mon Application Vulkan";
appInfo.applicationVersion = VK_MAKE_VERSION(1, 0, 0);
appInfo.pEngineName = "Aucun Moteur";
appInfo.engineVersion = VK_MAKE_VERSION(1, 0, 0);
appInfo.apiVersion = VK_API_VERSION_1_0; // Nous ciblons Vulkan 1.0 pour une compatibil
```

```
// Configuration de la création de l'instance
VkInstanceCreateInfo createInfo{};
createInfo.sType = VK_STRUCTURE_TYPE_INSTANCE_CREATE_INFO;
createInfo.pApplicationInfo = &appInfo;
```

```
// Récupération des extensions requises
auto extensions = getRequiredExtensions();
createInfo.enabledExtensionCount = static_cast<uint32_t>(extensions.size());
createInfo.ppEnabledExtensionNames = extensions.data();
```

```
// Intégration des couches de validation
VkDebugUtilsMessengerCreateInfoEXT debugCreateInfo{};
if (enableValidationLayers) {
    createInfo.enabledLayerCount = static_cast<uint32_t>(validationLayers.size());
    createInfo.ppEnabledLayerNames = validationLayers.data();

    // Permet de capturer les messages de debug durant la création et destruction de l'
    populateDebugMessengerCreateInfo(debugCreateInfo);
    createInfo.pNext = (VkDebugUtilsMessengerCreateInfoEXT*) &debugCreateInfo;
```

```

    } else {
        createInfo.enabledLayerCount = 0;
        createInfo.pNext = nullptr;
    }

    // Appel système pour créer l'instance
    VkResult result = vkCreateInstance(&createInfo, nullptr, &instance);
    if (result != VK_SUCCESS) {
        throw std::runtime_error("Echec de la creation de l'instance Vulkan ! Code erreur : " + std::to_string(result));
    }
}

// Vérification de la présence des couches de validation sur le système
bool VulkanApp::checkValidationLayerSupport() {
    uint32_t layerCount;
    vkEnumerateInstanceLayerProperties(&layerCount, nullptr);

    std::vector<VkLayerProperties> availableLayers(layerCount);
    vkEnumerateInstanceLayerProperties(&layerCount, availableLayers.data());

    for (const char* layerName : validationLayers) {
        bool layerFound = false;

        for (const auto& layerProperties : availableLayers) {
            if (strcmp(layerName, layerProperties.layerName) == 0) {
                layerFound = true;
                break;
            }
        }

        if (!layerFound) {
            return false;
        }
    }

    return true;
}

// Récupération des extensions nécessaires
std::vector<const char*> VulkanApp::getRequiredExtensions() {
    std::vector<const char*> extensions;

    if (enableValidationLayers) {
        // Cette extension permet de recevoir les messages de debug détaillés
        extensions.push_back(VK_EXT_DEBUG_UTILS_EXTENSION_NAME);
    }

    return extensions;
}

```

```

// Configuration du messenger de debug
void VulkanApp::populateDebugMessengerCreateInfo(VkDebugUtilsMessengerCreateInfoEXT& createInfo) {
    createInfo = {};
    createInfo.sType = VK_STRUCTURE_TYPE_DEBUG_UTILS_MESSENGER_CREATE_INFO_EXT;
    // Filtrage des messages à recevoir (Erreurs et Avertissements importants)
    createInfo.messageSeverity = VK_DEBUG_UTILS_MESSAGE_SEVERITY_WARNING_BIT_EXT |
                                VK_DEBUG_UTILS_MESSAGE_SEVERITY_ERROR_BIT_EXT;
    // Filtrage par type de message (Général, Validation de spécification, Performance)
    createInfo.messageType = VK_DEBUG_UTILS_MESSAGE_TYPE_GENERAL_BIT_EXT |
                             VK_DEBUG_UTILS_MESSAGE_TYPE_VALIDATION_BIT_EXT |
                             VK_DEBUG_UTILS_MESSAGE_TYPE_PERFORMANCE_BIT_EXT;
    createInfo.pfnUserCallback = debugCallback; // Notre fonction de callback définie précédemment
    createInfo.pUserData = nullptr; // Optionnel
}

void VulkanApp::setupDebugMessenger() {
    if (!enableValidationLayers) return;

    VkDebugUtilsMessengerCreateInfoEXT createInfo;
    populateDebugMessengerCreateInfo(createInfo);

    if (createDebugUtilsMessengerEXT(instance, &createInfo, nullptr, &debugMessenger) != VK_SUCCESS) {
        throw std::runtime_error("Echec de la configuration du messenger de debug !");
    }
}

// 2. Sélection du Périphérique Physique (GPU)
void VulkanApp::pickPhysicalDevice() {
    uint32_t deviceCount = 0;
    vkEnumeratePhysicalDevices(instance, &deviceCount, nullptr);

    if (deviceCount == 0) {
        throw std::runtime_error("Aucun GPU supportant Vulkan n'a été trouvé !");
    }

    std::vector<VkPhysicalDevice> devices(deviceCount);
    vkEnumeratePhysicalDevices(instance, &deviceCount, devices.data());

    for (const auto& device : devices) {
        if (isDeviceSuitable(device)) {
            physicalDevice = device;
            break;
        }
    }

    if (physicalDevice == VK_NULL_HANDLE) {
        throw std::runtime_error("Aucun GPU ne remplit les conditions requises !");
    }
}

```

```

}

// Évaluation de la compatibilité du GPU
bool VulkanApp::isDeviceSuitable(VkPhysicalDevice device) {
    VkPhysicalDeviceProperties deviceProperties;
    VkPhysicalDeviceFeatures deviceFeatures;
    vkGetPhysicalDeviceProperties(device, &deviceProperties);
    vkGetPhysicalDeviceFeatures(device, &deviceFeatures);

    // Pour l'instant, nous acceptons n'importe quel GPU dédié (Discrete GPU)
    // Dans le Module 1.2, nous ajouterons la vérification des familles de files d'attente
    std::cout << "GPU detecte : " << deviceProperties.deviceName << std::endl;
    return deviceProperties.deviceType == VK_PHYSICAL_DEVICE_TYPE_DISCRETE_GPU;
}

// 3. Création du Périphérique Logique
void VulkanApp::createLogicalDevice() {
    // Note pédagogique : Pour créer un périphérique logique, nous devons normalement spéci
    // les files d'attente (Queues). Ce concept sera approfondi dans le Module 1.2.
    // Pour que ce code compile et fonctionne, nous configurons une file d'attente minimale

    float queuePriority = 1.0f;
    VkDeviceQueueCreateInfo queueCreateInfo{};
    queueCreateInfo.sType = VK_STRUCTURE_TYPE_DEVICE_QUEUE_CREATE_INFO;
    queueCreateInfo.queueFamilyIndex = 0; // Hypothèse temporaire (Sera validé dans le Modu
    queueCreateInfo.queueCount = 1;
    queueCreateInfo.pQueuePriorities = &queuePriority;

    VkPhysicalDeviceFeatures deviceFeatures{}; // Pas de fonctionnalités spéciales requises

    VkDeviceCreateInfo createInfo{};
    createInfo.sType = VK_STRUCTURE_TYPE_DEVICE_CREATE_INFO;
    createInfo.pQueueCreateInfos = &queueCreateInfo;
    createInfo.queueCreateInfoCount = 1;
    createInfo.pEnabledFeatures = &deviceFeatures;

    // Configuration des extensions spécifiques au périphérique (ex: Swapchain, voir Module
    createInfo.enabledExtensionCount = 0;

    // Compatibilité avec les anciennes implémentations de Vulkan
    if (enableValidationLayers) {
        createInfo.enabledLayerCount = static_cast<uint32_t>(validationLayers.size());
        createInfo.ppEnabledLayerNames = validationLayers.data();
    } else {
        createInfo.enabledLayerCount = 0;
    }

    if (vkCreateDevice(physicalDevice, &createInfo, nullptr, &device) != VK_SUCCESS) {
        throw std::runtime_error("Echec de la creation du peripherique logique !");
    }
}

```

```

    }
}

// --- Chargement dynamique des fonctions d'extension ---
// Les fonctions liées aux extensions (comme le Debug Messenger) ne sont pas chargées automatiquement
// Nous devons demander à l'instance de récupérer leurs adresses mémoire.

VkResult VulkanApp::createDebugUtilsMessengerEXT(VkInstance instance,
                                                  const VkDebugUtilsMessengerCreateInfoEXT* pCreateInfo,
                                                  const VkAllocationCallbacks* pAllocator,
                                                  VkDebugUtilsMessengerEXT* pDebugMessenger) {
    auto func = (PFN_vkCreateDebugUtilsMessengerEXT) vkGetInstanceProcAddr(instance, "vkCreateDebugUtilsMessengerEXT");
    if (func != nullptr) {
        return func(instance, pCreateInfo, pAllocator, pDebugMessenger);
    } else {
        return VK_ERROR_EXTENSION_NOT_PRESENT;
    }
}

void VulkanApp::destroyDebugUtilsMessengerEXT(VkInstance instance,
                                              VkDebugUtilsMessengerEXT debugMessenger,
                                              const VkAllocationCallbacks* pAllocator) {
    auto func = (PFN_vkDestroyDebugUtilsMessengerEXT) vkGetInstanceProcAddr(instance, "vkDestroyDebugUtilsMessengerEXT");
    if (func != nullptr) {
        func(instance, debugMessenger, pAllocator);
    }
}

```

4. Pièges Fréquents et Bonnes Pratiques

4.1 L'Oubli d'Initialisation des Structures

C'est l'erreur numéro un des débutants en Vulkan. En C++, si vous écrivez :

```
VkInstanceCreateInfo createInfo; // Sans accolades !
```

Les variables internes de la structure contiendront des valeurs résiduelles de la mémoire (garbage data). Vulkan lira ces données, tentera d'accéder à des pointeurs invalides et provoquera un crash immédiat. * Bonne pratique : Toujours utiliser l'initialisation par défaut

```
{}
```



```
VkInstanceCreateInfo createInfo{}; // Initialise tous les membres à 0 ou nullptr
```

4.2 L'Ordre de Destruction des Ressources

Vulkan est extrêmement strict sur l'ordre de libération de la mémoire. Un objet parent ne peut pas être détruit avant ses enfants. * Règle absolue : L'ordre de destruction dans votre fonction `cleanup()` doit être le strict inverse de l'ordre de création dans `initVulkan()`. * Création : Instance -> Debug Messenger -> Device. * Destruction : Device -> Debug Messenger -> Instance.

4.3 Négliger les Couches de Validation

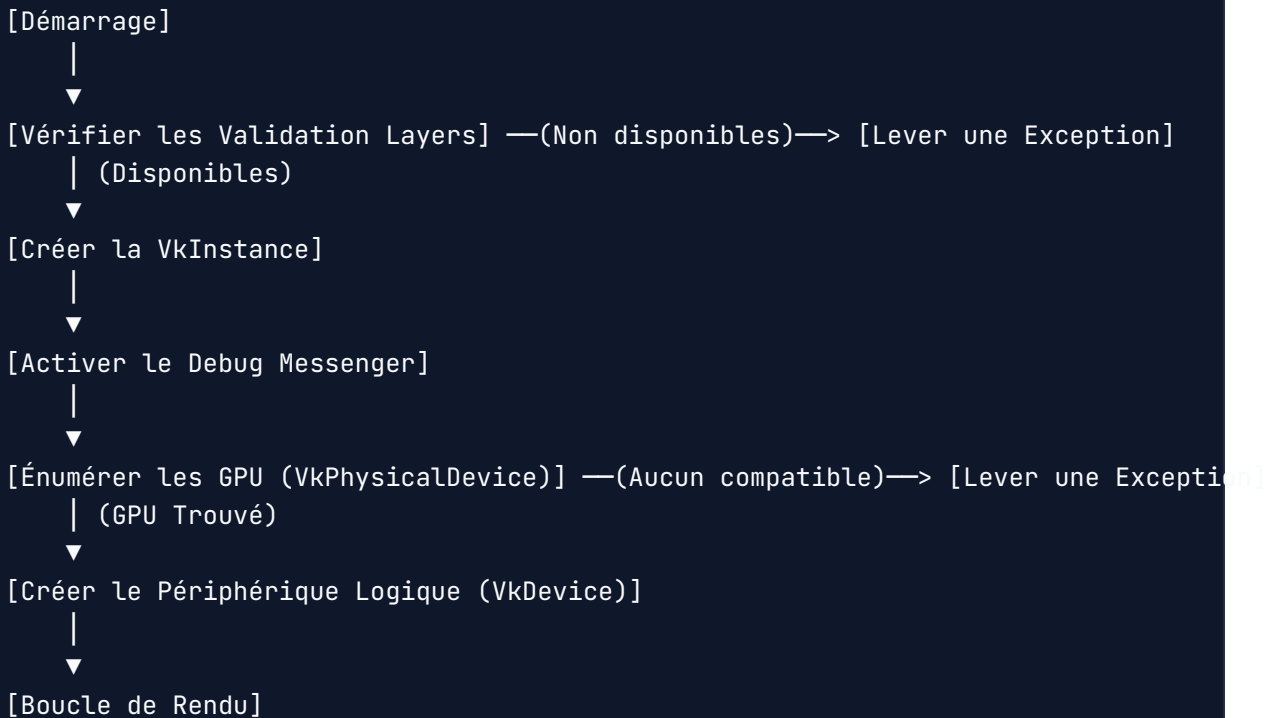
Tenter de coder en Vulkan sans activer les couches de validation équivaut à piloter un avion de nuit sans tableau de bord. Si votre programme plante sans message d'erreur, vérifiez immédiatement que `VK_LAYER_KHRONOS_validation` est bien installé via le SDK Vulkan sur votre machine.

5. Synthèse Pédagogique

5.1 Résumé des Objets Clés

Objet Vulkan	Rôle Principal	Analogie
<code>VkInstance</code>	Point d'entrée de l'API, initialise la bibliothèque.	Le contrat d'embauche avec le système graphique.
<code>VkPhysicalDevice</code>	Représente le GPU physique (matériel).	La carte graphique physique dans votre boîtier PC.
<code>VkDevice</code>	Interface logique pour créer les ressources.	Le chef de chantier à qui vous donnez vos ordres de rendu.
<code>VkDebugUtilsMessengerEXT</code>	Capture et affiche les erreurs de développement.	L'inspecteur des travaux finis qui crie en cas d'erreur.

5.2 Flux d'Exécution de l'Initialisation



6. Exercice Pratique d'Application

Énoncé : "Diagnostic du Matériel (GPU Profiler)"

L'objectif de cet exercice est de modifier et d'étendre notre code pour créer un utilitaire de diagnostic. Votre programme devra interroger le système, lister tous les GPU disponibles, et pour chacun d'eux, afficher dans la console : 1. Son nom (ex: NVIDIA GeForce RTX 4070). 2. Son type (Dédié, Intégré, CPU). 3. La version de l'API Vulkan supportée par le pilote. 4. La quantité maximale de mémoire allouable pour une seule texture (limite de dimension 2D).

Indices

- Pour récupérer les propriétés d'un GPU, utilisez la fonction :
`vkGetPhysicalDeviceProperties(device, &properties)`.
- La structure `VkPhysicalDeviceProperties` contient un membre `limits` de type `VkPhysicalDeviceLimits`. C'est là que se trouve la taille maximale des textures 2D (`maxImageDimension2D`).

- La version de l'API est stockée sous forme d'un entier encodé. Utilisez les macros suivantes pour décoder la version : `VK_VERSION_MAJOR(version)` , `VK_VERSION_MINOR(version)` , `VK_VERSION_PATCH(version)` .

Correction Complète

Voici le code à intégrer dans votre classe pour réaliser le diagnostic. Vous pouvez remplacer la fonction `pickPhysicalDevice()` par celle-ci :

```

void VulkanApp::pickPhysicalDevice() {
    uint32_t deviceCount = 0;
    vkEnumeratePhysicalDevices(instance, &deviceCount, nullptr);

    if (deviceCount == 0) {
        throw std::runtime_error("Aucun GPU avec support Vulkan detecte !");
    }

    std::vector<VkPhysicalDevice> devices(deviceCount);
    vkEnumeratePhysicalDevices(instance, &deviceCount, devices.data());

    std::cout << "\n=== DIAGNOSTIC DU MATERIEL (GPU PROFILER) ===" << std::endl;
    std::cout << "Nombre de GPU compatibles Vulkan trouves : " << deviceCount << "\n" << std::endl;

    for (size_t i = 0; i < devices.size(); i++) {
        VkPhysicalDeviceProperties properties;
        vkGetPhysicalDeviceProperties(devices[i], &properties);

        std::cout << "--- GPU #" << i << " ---" << std::endl;
        std::cout << "Nom du peripherique : " << properties.deviceName << std::endl;

        // Détermination du type de GPU
        std::cout << "Type de GPU          : ";
        switch (properties.deviceType) {
            case VK_PHYSICAL_DEVICE_TYPE_OTHER:        std::cout << "Autre (Inconnu)"; break;
            case VK_PHYSICAL_DEVICE_TYPE_INTEGRATED_GPU: std::cout << "GPU Integre (iGPU)";
            case VK_PHYSICAL_DEVICE_TYPE_DISCRETE_GPU:  std::cout << "GPU Dedie (dGPU)"; break;
            case VK_PHYSICAL_DEVICE_TYPE_VIRTUAL_GPU:   std::cout << "GPU Virtuel"; break;
            case VK_PHYSICAL_DEVICE_TYPE_CPU:           std::cout << "Processeur (CPU)"; break;
        }
        std::cout << std::endl;

        // Décodage de la version de l'API Vulkan
        uint32_t apiVer = properties.apiVersion;
        std::cout << "Version API Vulkan : "
            << VK_VERSION_MAJOR(apiVer) << "."
            << VK_VERSION_MINOR(apiVer) << "."
            << VK_VERSION_PATCH(apiVer) << std::endl;

        // Récupération des limites matérielles
        std::cout << "Taille max Texture : "
            << properties.limits.maxImageDimension2D << " x "
            << properties.limits.maxImageDimension2D << " pixels" << std::endl;
        std::cout << "-----\n" << std::endl;

        // Sélection automatique du premier GPU dédié rencontré
        if (physicalDevice == VK_NULL_HANDLE && properties.deviceType == VK_PHYSICAL_DEVICE_TYPE_DISCRETE_GPU)
            physicalDevice = devices[i];
    }
}

```

```

        std::cout << ">> Selection de " << properties.deviceName << " pour le rendu." << endl;
    }
}

// Si aucun GPU dédié n'est trouvé, on prend le premier disponible par défaut
if (physicalDevice == VK_NULL_HANDLE && !devices.empty()) {
    physicalDevice = devices[0];
    VkPhysicalDeviceProperties properties;
    vkGetPhysicalDeviceProperties(physicalDevice, &properties);
    std::cout << ">> Aucun GPU dédié trouvé. Selection par défaut de : " << properties.deviceName << endl;
}
}

```

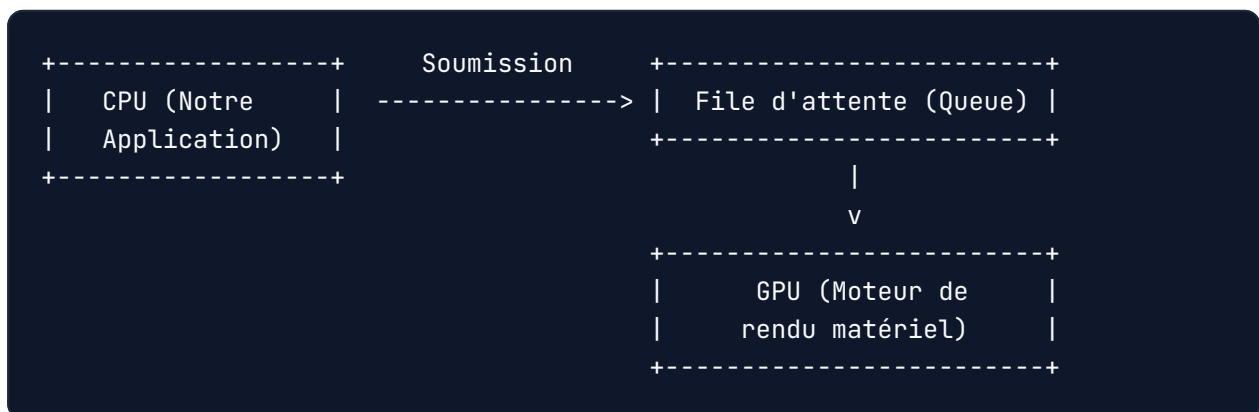
Module 1.2 : Familles de files d'attente (Queue Families) et Fenêtrage (WSI)

1. Introduction Conceptuelle

Le "Pourquoi" : La philosophie asynchrone de Vulkan

Dans les anciennes API graphiques (comme OpenGL), le processeur (CPU) envoyait des commandes au processeur graphique (GPU) de manière synchrone et implicite. Le GPU était vu comme une boîte noire qui exécutait les instructions immédiatement ou presque.

Vulkan rompt totalement avec ce modèle pour se rapprocher de la réalité matérielle des cartes graphiques modernes. Un GPU moderne est un coprocesseur hautement parallèle et asynchrone. Pour lui donner du travail, nous devons déposer des paquets d'instructions (appelés *Command Buffers*) dans des files d'attente (ou *Queues*). Le GPU pioche ensuite dans ces files pour exécuter les tâches à son propre rythme.



Qu'est-ce qu'une "Famille de files d'attente" (Queue Family) ?

Sur une carte graphique, tous les circuits intégrés ne sont pas identiques. Certains circuits sont spécialisés dans le calcul mathématique lourd, d'autres dans le transfert ultra-rapide de mémoire, et d'autres dans le rendu de triangles (la rasterisation).

Vulkan regroupe ces files d'attente par Familles (Queue Families). Chaque famille possède des capacités spécifiques : * **Graphique (Graphics)** : Capable de dessiner des triangles, d'exécuter des shaders de fragments, etc. * **Calcul (Compute)** : Capable de lancer des calculs massivement parallèles (GPGPU), par exemple pour de la physique ou de l'intelligence artificielle. * **Transfert (Transfer)** : Spécialisée dans la copie ultra-rapide de données entre la mémoire système (RAM) et la mémoire vidéo (VRAM). * **Présentation (Present)** : Capable d'envoyer l'image finale à l'écran pour l'afficher à l'utilisateur.

Le problème du fenêtrage : L'indépendance de Vulkan

Par conception, Vulkan est une API multiplateforme et agnostique. Cela signifie qu'elle ne sait pas ce qu'est une fenêtre Windows (Win32), une fenêtre Linux (X11 ou Wayland) ou une application macOS. Vulkan sait uniquement calculer des pixels et les écrire dans de la mémoire.

Pour afficher ces pixels dans une fenêtre système, nous avons besoin d'une extension appelée WSI (Window System Integration). Le WSI introduit deux concepts clés : 1. La Surface (`VkSurfaceKHR`) : Une abstraction Vulkan représentant la fenêtre de notre système d'exploitation. 2. La file de présentation (Present Queue) : Une file d'attente capable de prendre une image calculée par le GPU et de l'envoyer à la Surface.

2. Fondations Théoriques

1. Les Familles de files d'attente sous la loupe

Chaque GPU physique expose un certain nombre de familles de files d'attente. Pour chaque famille, nous pouvons interroger ses propriétés via la structure `VkQueueFamilyProperties`. Cette structure contient notamment : * `queueFlags` : Un masque de bits indiquant les capacités (ex: `VK_QUEUE_GRAPHICS_BIT`, `VK_QUEUE_COMPUTE_BIT`, `VK_QUEUE_TRANSFER_BIT`). * `queueCount` : Le nombre de files d'attente physiques disponibles dans cette famille précise.

2. Le mécanisme du WSI (Window System Integration)

Comme le WSI fait partie des extensions optionnelles de Vulkan (car un serveur de calcul sans écran n'en a pas besoin), son nom contient le suffixe `KHR` (Khronos Group).

Pour faire du fenêtrage, nous devons activer : 1. Au niveau de l'Instance : Des extensions pour créer la surface (ex: `VK_KHR_surface` combinée avec `VK_KHR_win32_surface` sur Windows). 2. Au niveau du Logical Device : L'extension de la Swapchain (`VK_KHR_swapchain`), qui gère la double/triple mise en mémoire tampon des images (ce point sera détaillé dans le *Module 1.3*).

3. La distinction cruciale : Graphique vs Présentation

Il est fondamental de comprendre qu'une file d'attente capable de faire des calculs graphiques (dessiner) n'est pas obligatoirement capable d'afficher le résultat sur votre écran.

- **File Graphique** : Écrit dans une texture en mémoire.
- **File de Présentation** : Communique avec le système d'exploitation pour afficher cette texture.

Sur la majorité des ordinateurs de bureau modernes, la même famille de files d'attente supporte à la fois le graphisme et la présentation. Cependant, sur des architectures mobiles ou des configurations multi-GPU (ordinateurs portables avec GPU intégré Intel/AMD et GPU dédié NVIDIA/AMD), ces deux tâches peuvent être gérées par des familles différentes. Notre code doit impérativement être capable de gérer ces deux scénarios.

3. Implémentation Pratique Pas-à-Pas

Nous allons maintenant écrire le code C++ permettant de : 1. Initialiser une fenêtre avec la bibliothèque GLFW. 2. Créer la surface de rendu Vulkan (`VkSurfaceKHR`). 3. Rechercher les indices des familles de files d'attente (Graphique et Présentation). 4. Créer le périphérique logique (`VkDevice`) en activant ces files d'attente. 5. Récupérer les poignées (`VkQueue`) pour pouvoir les utiliser plus tard.

Rappel C++ pour les débutants

- `std::optional<uint32_t>` : Un conteneur introduit en C++17 qui peut ou non contenir une valeur. Très utile pour représenter un index de file d'attente qui n'a pas encore été trouvé.

- `std::set` : Une collection d'éléments uniques. Nous l'utiliserons pour éviter de créer deux fois la même file d'attente si l'index graphique et l'index de présentation sont identiques.

Code Source Complet et Structuré

```

#define GLFW_INCLUDE_VULKAN
#include <GLFW/glfw3.h>

#include <iostream>
#include <vector>
#include <optional>
#include <set>
#include <stdexcept>

// Structure regroupant les indices des familles de files d'attente dont nous avons besoin
struct QueueFamilyIndices {
    std::optional<uint32_t> graphicsFamily;
    std::optional<uint32_t> presentFamily;

    // Vérifie si toutes les files d'attente requises ont été trouvées
    bool isComplete() const {
        return graphicsFamily.has_value() && presentFamily.has_value();
    }
};

class VulkanApplication {
public:
    void run() {
        initWindow();
        initVulkan();
        mainLoop();
        cleanup();
    }

private:
    // Variables membres
    GLFWwindow* window = nullptr;
    VkInstance instance = VK_NULL_HANDLE;
    VkSurfaceKHR surface = VK_NULL_HANDLE;
    VkPhysicalDevice physicalDevice = VK_NULL_HANDLE;
    VkDevice device = VK_NULL_HANDLE;

    // Poignées vers nos files d'attente (récupérées après la création du Device)
    VkQueue graphicsQueue = VK_NULL_HANDLE;
    VkQueue presentQueue = VK_NULL_HANDLE;

    const uint32_t WIDTH = 800;
    const uint32_t HEIGHT = 600;

    // 1. Initialisation de la fenêtre avec GLFW
    void initWindow() {
        if (!glfwInit()) {
            throw std::runtime_error("Échec de l'initialisation de GLFW");
        }
    }

```

```

    }

    // Indiquer à GLFW de ne pas charger l'API OpenGL par défaut
    glfwWindowHint(GLFW_CLIENT_API, GLFW_NO_API);
    // Désactiver la possibilité de redimensionner la fenêtre pour simplifier ce module
    glfwWindowHint(GLFW_RESIZABLE, GLFW_FALSE);

    window = glfwCreateWindow(WIDTH, HEIGHT, "Vulkan - Files d'attente & WSI", nullptr, nullptr);
    if (!window) {
        throw std::runtime_error("Échec de la création de la fenêtre GLFW");
    }
}

// 2. Initialisation de Vulkan
void initVulkan() {
    createInstance();
    createSurface(); // La surface doit être créée juste après l'instance !
    pickPhysicalDevice();
    createLogicalDevice();
}

// Création de l'Instance Vulkan (Concept vu dans le Module 1.1)
void createInstance() {
    VkApplicationInfo appInfo{};
    appInfo.sType = VK_STRUCTURE_TYPE_APPLICATION_INFO;
    appInfo.pApplicationName = "Vulkan App";
    appInfo.applicationVersion = VK_MAKE_VERSION(1, 0, 0);
    appInfo.pEngineName = "No Engine";
    appInfo.engineVersion = VK_MAKE_VERSION(1, 0, 0);
    appInfo.apiVersion = VK_API_VERSION_1_0;

    VkInstanceCreateInfo createInfo{};
    createInfo.sType = VK_STRUCTURE_TYPE_INSTANCE_CREATE_INFO;
    createInfo.pApplicationInfo = &appInfo;

    // Récupération des extensions requises par GLFW pour le fenêtrage
    uint32_t glfwExtensionCount = 0;
    const char** glfwExtensions = glfwGetRequiredInstanceExtensions(&glfwExtensionCount);

    createInfo.enabledExtensionCount = glfwExtensionCount;
    createInfo.ppEnabledExtensionNames = glfwExtensions;
    createInfo.enabledLayerCount = 0;

    if (vkCreateInstance(&createInfo, nullptr, &instance) != VK_SUCCESS) {
        throw std::runtime_error("Échec de la création de l'instance Vulkan");
    }
}

// 3. Création de la Surface (WSI)

```

```

void createSurface() {
    // GLFW encapsule les appels système complexes (Win32, X11, Cocoa)
    // et crée la VkSurfaceKHR de manière portable.
    if (glfwCreateWindowSurface(instance, window, nullptr, &surface) != VK_SUCCESS) {
        throw std::runtime_error("Échec de la création de la surface de fenêtre");
    }
}

// Sélection du GPU Physique (Concept initié dans le Module 1.1, complété ici)
void pickPhysicalDevice() {
    uint32_t deviceCount = 0;
    vkEnumeratePhysicalDevices(instance, &deviceCount, nullptr);

    if (deviceCount == 0) {
        throw std::runtime_error("Aucun GPU supportant Vulkan n'a été trouvé");
    }

    std::vector<VkPhysicalDevice> devices(deviceCount);
    vkEnumeratePhysicalDevices(instance, &deviceCount, devices.data());

    for (const auto& dev : devices) {
        if (isDeviceSuitable(dev)) {
            physicalDevice = dev;
            break;
        }
    }

    if (physicalDevice == VK_NULL_HANDLE) {
        throw std::runtime_error("Aucun GPU ne convient aux exigences de l'application");
    }
}

// Vérification de la compatibilité du GPU
bool isDeviceSuitable(VkPhysicalDevice dev) {
    QueueFamilyIndices indices = findQueueFamilies(dev);
    return indices.isComplete();
}

// 4. Recherche des familles de files d'attente requises
QueueFamilyIndices findQueueFamilies(VkPhysicalDevice dev) {
    QueueFamilyIndices indices;

    uint32_t queueFamilyCount = 0;
    vkGetPhysicalDeviceQueueFamilyProperties(dev, &queueFamilyCount, nullptr);

    std::vector<VkQueueFamilyProperties> queueFamilies(queueFamilyCount);
    vkGetPhysicalDeviceQueueFamilyProperties(dev, &queueFamilyCount, queueFamilies.data());

    int i = 0;

```

```

    for (const auto& queueFamily : queueFamilies) {
        // A. Vérification du support graphique
        if (queueFamily.queueFlags & VK_QUEUE_GRAPHICS_BIT) {
            indices.graphicsFamily = i;
        }

        // B. Vérification du support de la présentation
        VkBool32 presentSupport = false;
        vkGetPhysicalDeviceSurfaceSupportKHR(dev, i, surface, &presentSupport);

        if (presentSupport) {
            indices.presentFamily = i;
        }

        // Si on a trouvé toutes nos files, on arrête la recherche
        if (indices.isComplete()) {
            break;
        }

        i++;
    }

    return indices;
}

// 5. Création du Périphérique Logique (Logical Device) et récupération des files d'attente
void createLogicalDevice() {
    QueueFamilyIndices indices = findQueueFamilies(physicalDevice);

    // Nous utilisons un std::set pour stocker les indices uniques.
    // Si l'index graphique et l'index de présentation sont identiques, le set n'aura qu'un seul élément.
    std::vector<VkDeviceQueueCreateInfo> queueCreateInfos;
    std::set<uint32_t> uniqueQueueFamilies = {
        indices.graphicsFamily.value(),
        indices.presentFamily.value()
    };

    float queuePriority = 1.0f; // Priorité de la file d'attente (entre 0.0 et 1.0)
    for (uint32_t queueFamily : uniqueQueueFamilies) {
        VkDeviceQueueCreateInfo queueCreateInfo{};
        queueCreateInfo.sType = VK_STRUCTURE_TYPE_DEVICE_QUEUE_CREATE_INFO;
        queueCreateInfo.queueFamilyIndex = queueFamily;
        queueCreateInfo.queueCount = 1;
        queueCreateInfo.pQueuePriorities = &queuePriority;
        queueCreateInfos.push_back(queueCreateInfo);
    }

    // Spécification des fonctionnalités du GPU requises (aucune pour l'instant)
    VkPhysicalDeviceFeatures deviceFeatures{};

```

```

VkDeviceCreateInfo createInfo{};
createInfo.sType = VK_STRUCTURE_TYPE_DEVICE_CREATE_INFO;
createInfo.queueCreateInfoCount = static_cast<uint32_t>(queueCreateInfos.size());
createInfo.pQueueCreateInfos = queueCreateInfos.data();
createInfo.pEnabledFeatures = &deviceFeatures;

// Activation de l'extension de la Swapchain (nécessaire pour le rendu à l'écran)
const std::vector<const char*> deviceExtensions = {
    VK_KHR_SWAPCHAIN_EXTENSION_NAME
};
createInfo.enabledExtensionCount = static_cast<uint32_t>(deviceExtensions.size());
createInfo.ppEnabledExtensionNames = deviceExtensions.data();
createInfo.enabledLayerCount = 0;

// Création finale du périphérique logique
if (vkCreateDevice(physicalDevice, &createInfo, nullptr, &device) != VK_SUCCESS) {
    throw std::runtime_error("Échec de la création du périphérique logique");
}

// Récupération des poignées de files d'attente (Queues)
// Le troisième paramètre (0) correspond à l'index de la file au sein de la famille
vkGetDeviceQueue(device, indices.graphicsFamily.value(), 0, &graphicsQueue);
vkGetDeviceQueue(device, indices.presentFamily.value(), 0, &presentQueue);
}

// Boucle de rendu principale (vide pour le moment)
void mainLoop() {
    while (!glfwWindowShouldClose(window)) {
        glfwPollEvents();
    }
}

// Nettoyage rigoureux des ressources
void cleanup() {
    // 1. Détruire le périphérique logique
    if (device != VK_NULL_HANDLE) {
        vkDestroyDevice(device, nullptr);
    }

    // 2. Détruire la surface (doit être fait AVANT l'instance)
    if (surface != VK_NULL_HANDLE) {
        vkDestroySurfaceKHR(instance, surface, nullptr);
    }

    // 3. Détruire l'instance Vulkan
    if (instance != VK_NULL_HANDLE) {
        vkDestroyInstance(instance, nullptr);
    }
}

```



```

        // 4. Fermer la fenêtre GLFW et terminer la bibliothèque
        if (window) {
            glfwDestroyWindow(window);
        }
        glfwTerminate();
    }
};

int main() {
    VulkanApplication app;
    try {
        app.run();
    } catch (const std::exception& e) {
        std::cerr << "Erreur fatale : " << e.what() << std::endl;
        return EXIT_FAILURE;
    }
    return EXIT_SUCCESS;
}

```

4. Pièges Fréquents et Bonnes Pratiques

1. L'ordre de destruction des objets (Le "Crash de fermeture")

C'est l'erreur numéro un des débutants en Vulkan. Vulkan est extrêmement strict sur l'ordre de destruction. * **Règle d'or** : Les objets enfants doivent être détruits avant leurs parents. * La surface (`VkSurfaceKHR`) est créée à partir de l'instance (`VkInstance`). Elle doit donc être détruite avant l'instance. * Le périphérique logique (`VkDevice`) utilise la surface indirectement. Il doit être détruit avant la surface. * **Conséquence** : Si vous détruisez l'instance avant la surface, votre programme plantera instantanément à la fermeture avec une violation d'accès mémoire.

2. Doubler la création de files d'attente identiques

Si la famille de files graphiques et la famille de présentation ont le même index (ce qui est très fréquent), et que vous tentez de créer deux configurations de files d'attente distinctes avec le même index dans `VkDeviceCreateInfo`, Vulkan générera une erreur ou un comportement indéfini. * **Solution** : Utilisez toujours un conteneur d'éléments uniques

comme `std::set` pour filtrer les doublons d'indices avant de remplir la structure de création du périphérique logique.

3. Oublier d'activer les extensions de fenêtrage de GLFW

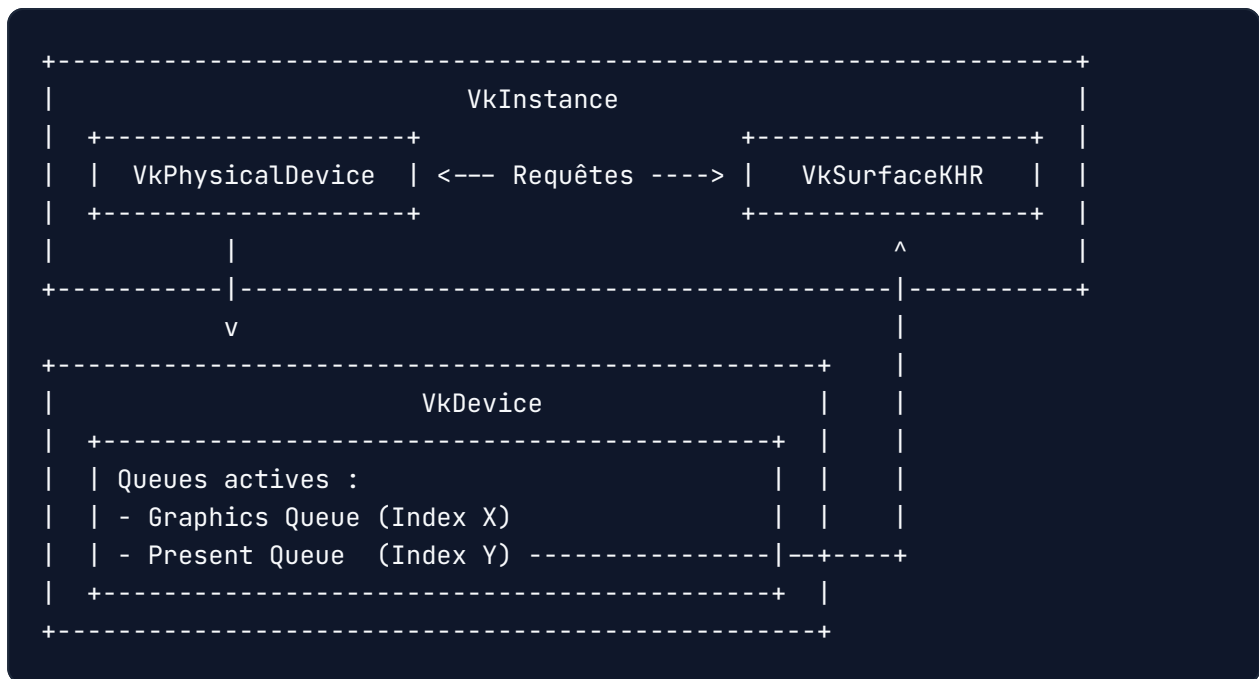
Si vous créez une instance Vulkan standard sans passer les extensions requises par GLFW (`glfwGetRequiredInstanceExtensions`), l'appel à `glfwCreateWindowSurface` échouera lamentablement avec l'erreur `VK_ERROR_EXTENSION_NOT_PRESENT`.

5. Synthèse Pédagogique

Résumé des concepts clés

Concept	Rôle principal	Durée de vie / Parent
<code>VkSurfaceKHR</code>	Représentation abstraite de la fenêtre OS pour Vulkan.	Parent : <code>VkInstance</code> . Doit être détruite avant l'Instance.
Queue Family	Catégorie de processeurs sur le GPU (Graphisme, Calcul, Transfert).	Liée au <code>VkPhysicalDevice</code> . Non détruite explicitement.
<code>VkQueue</code>	Poignée de contrôle pour envoyer des commandes au GPU.	Parent : <code>VkDevice</code> . Pas de destruction manuelle (gérée par le Device).
WSI	Couche d'intégration permettant de lier Vulkan à l'affichage système.	Extension globale de l'API.

Schéma d'architecture des objets créés



6. Exercice Pratique d'Application

Énoncé : Intégration d'une file de transfert dédiée

Dans les applications professionnelles, on évite de surcharger la file d'attente graphique principale avec des transferts de textures ou de modèles 3D depuis la RAM vers la VRAM. On préfère utiliser une file de transfert dédiée (Transfer Queue) si le GPU en propose une.

Votre objectif : Modifier la structure `QueueFamilyIndices` et la fonction `findQueueFamilies` pour :

1. Chercher une famille de files d'attente supportant spécifiquement le transfert (`VK_QUEUE_TRANSFER_BIT`).
2. Bonus de rigueur : Choisir de préférence une famille qui supporte le transfert mais qui n'est pas graphique (pour avoir une vraie file de transfert asynchrone dédiée). Si ce n'est pas possible, se rabattre sur la file graphique (qui sait aussi faire du transfert).

Indices

1. Le flag pour le transfert est `VK_QUEUE_TRANSFER_BIT`.

2. Pour vérifier qu'une famille est *uniquement* dédiée au transfert et pas au graphisme, vous pouvez utiliser des opérations binaires : `((properties.queueFlags & VK_QUEUE_TRANSFER_BIT) && !(properties.queueFlags & VK_QUEUE_GRAPHICS_BIT))`

Correction complète de l'exercice

Voici comment modifier le code pour intégrer cette logique :

```

// 1. Mise à jour de la structure pour inclure la file de transfert
struct QueueFamilyIndices {
    std::optional<uint32_t> graphicsFamily;
    std::optional<uint32_t> presentFamily;
    std::optional<uint32_t> transferFamily; // Nouvelle file

    bool isComplete() const {
        return graphicsFamily.has_value() &&
            presentFamily.has_value() &&
            transferFamily.has_value();
    }
};

// 2. Mise à jour de la fonction de recherche
QueueFamilyIndices findQueueFamilies(VkPhysicalDevice dev, VkSurfaceKHR surf) {
    QueueFamilyIndices indices;

    uint32_t queueFamilyCount = 0;
    vkGetPhysicalDeviceQueueFamilyProperties(dev, &queueFamilyCount, nullptr);

    std::vector<VkQueueFamilyProperties> queueFamilies(queueFamilyCount);
    vkGetPhysicalDeviceQueueFamilyProperties(dev, &queueFamilyCount, queueFamilies.data());

    int i = 0;
    for (const auto& queueFamily : queueFamilies) {
        // A. Recherche du support Graphique
        if (queueFamily.queueFlags & VK_QUEUE_GRAPHICS_BIT) {
            indices.graphicsFamily = i;
        }

        // B. Recherche du support de Présentation
        VkBool32 presentSupport = false;
        vkGetPhysicalDeviceSurfaceSupportKHR(dev, i, surf, &presentSupport);
        if (presentSupport) {
            indices.presentFamily = i;
        }

        // C. Recherche d'une file de transfert dédiée (sans graphisme)
        if ((queueFamily.queueFlags & VK_QUEUE_TRANSFER_BIT) &&
            !(queueFamily.queueFlags & VK_QUEUE_GRAPHICS_BIT)) {
            indices.transferFamily = i;
        }

        if (indices.isComplete()) {
            break;
        }
        i++;
    }
}

```

```

// Solution de repli (Fallback) : Si aucune file de transfert dédiée n'a été trouvée,
// on réutilise la file graphique (qui supporte implicitement le transfert).
if (!indices.transferFamily.has_value()) {
    indices.transferFamily = indices.graphicsFamily;
}

return indices;
}

```

Cette architecture robuste garantit que votre moteur graphique tirera le meilleur parti des capacités matérielles de n'importe quel GPU, qu'il s'agisse d'une puce intégrée légère ou d'une carte graphique dédiée haut de gamme.

Module 1.3 : Création et configuration de la Swapchain

1. Introduction Conceptuelle

1.1 Le Problème du Rendu Direct : Le Déchirement d'Image (Tearing)

Dans les balbutiements de l'informatique graphique, les cartes graphiques écrivaient directement les pixels dans la mémoire d'affichage de l'écran (le *Frame Buffer*). Cette approche naïve pose un problème physique majeur : l'écran possède son propre rythme de rafraîchissement (par exemple, 60 Hz, soit une mise à jour toutes les 16,67 millisecondes), tandis que le processeur graphique (GPU) calcule les images à un rythme variable dépendant de la complexité de la scène.

Si le GPU écrit dans la mémoire d'affichage pendant que l'écran est en train de la lire pour l'afficher, l'utilisateur verra une partie de l'ancienne image et une partie de la nouvelle sur le même écran. Ce phénomène visuel désagréable est appelé déchirement d'image ou *tearing*.

```
Écran (Lecture) :  [--- Image N-1 ---] [--- Image N-1 ---] [--- Image N ---]
GPU (Écriture) :   [===== Écriture Image N =====]
Résultat Visuel :  | Haut : Image N-1 | Bas : Image N | ← TEARING !
```

1.2 La Solution : Le Double et Triple Buffering

Pour éviter le tearing, nous introduisons un mécanisme de synchronisation appelé **Double Buffering** (Double Tampon). * **Front Buffer** (Tampon Avant) : L'image actuellement lue par l'écran. Cette mémoire est en lecture seule pour le système d'affichage. * **Back Buffer** (Tampon Arrière) : L'image en cours de dessin par le GPU.

Une fois que le GPU a terminé de dessiner dans le *Back Buffer*, on procède à un échange (*Swap*). Le *Back Buffer* devient le *Front Buffer*, et inversement. Cet échange est synchronisé avec le signal de synchronisation verticale de l'écran (*V-Sync*).

Le *Triple Buffering* ajoute un troisième tampon (un deuxième *Back Buffer*). Cela permet au GPU de continuer à dessiner la frame suivante même si le *Back Buffer* précédent attend d'être affiché, éliminant ainsi le blocage du GPU tout en évitant le *tearing*.

1.3 Qu'est-ce que la Swapchain dans Vulkan ?

Vulkan est une API totalement agnostique du système d'exploitation et du matériel d'affichage. Par défaut, un périphérique Vulkan (`VkDevice`) ne sait pas ce qu'est un "écran" ou une "fenêtre".

La *Swapchain* (chaîne d'échange) est l'infrastructure qui fait le pont entre Vulkan et le système de fenêtrage de l'OS (Windows, macOS, Linux). Elle est gérée par l'extension de surface de présentation (`VK_KHR_swapchain`). La Swapchain possède et gère une file d'attente d'images de rendu. Notre application devra acquérir une image de la Swapchain, dessiner dedans, puis la restituer à la Swapchain pour qu'elle soit présentée à l'écran.

2. Fondations Théoriques

Avant de pouvoir créer une Swapchain, nous devons interroger le GPU pour connaître ses capacités d'affichage sur notre surface de fenêtre (`VkSurfaceKHR` , créée au Module 1.2). Trois types de paramètres doivent être analysés :

Propriétés de la Surface		
[1. Capabilities]	[2. Formats]	[3. Present Modes]
- Min/Max d'images	- Format de pixel	- Immediate
- Résolution (Extent)	- Espace colorimétrique	- FIFO (V-Sync)
- Transformations admises		- Mailbox

2.1 Les Capacités de la Surface (`VkSurfaceCapabilitiesKHR`)

Cette structure définit les limites physiques de la surface de présentation : * Nombre d'images minimal et maximal que la Swapchain peut contenir (généralement 2 pour le double buffering, 3 pour le triple buffering). * Dimensions actuelles, minimales et maximales (en pixels) de la surface (appelées *Extent*). * Transformations supportées (comme une rotation de 90 degrés pour les écrans de smartphones). * Modes de transparence (Composite Alpha) supportés pour l'intégration avec les autres fenêtres du système d'exploitation.

2.2 Les Formats de Surface (`VkSurfaceFormatKHR`)

Chaque format supporté est un couple composé de : 1. Format de couleur (`VkFormat`) : Décrit la disposition des canaux de couleur en mémoire. Par exemple, `VK_FORMAT_B8G8R8A8_SRGB` signifie que chaque pixel est codé sur 32 bits : 8 bits pour le Bleu, 8 bits pour le Vert, 8 bits pour le Rouge, et 8 bits pour l'Alpha (transparence), le tout interprété dans l'espace colorimétrique non-linéaire sRGB. 2. Espace colorimétrique (`VkColorSpaceKHR`) : Définit comment les couleurs doivent être interprétées par l'écran (ex: `VK_COLOR_SPACE_SRGB_NONLINEAR_KHR`).

2.3 Les Modes de Présentation (`VkPresentModeKHR`)

Le mode de présentation détermine la façon dont les images sont envoyées à l'écran et comment la synchronisation est gérée. Vulkan propose quatre modes principaux :

Mode	Description	V-Sync	Tearing	Latence	Support
IMMEDIATE	Les images sont envoyées à l'écran immédiatement.	Non	Oui	Très faible	Optionnel
FIFO	File d'attente classique (First-In, First-Out). Le GPU attend si la file est pleine.	Oui	Non	Moyenne	Obligatoire
FIFO_RELAXED	Similaire à FIFO, mais si le GPU est en retard, l'image est affichée immédiatement (tearing).	Oui	Oui	Faible	Optionnel
MAILBOX	Triple buffering optimal. Si la file est pleine, l'image la plus ancienne non affichée est remplacée.	Oui	Non	Minimale	Optionnel

2.4 Le Concept d'Image View (**VkImageView**)

Une image Vulkan (**VkImage**) représente une zone de mémoire brute contenant des pixels. Cependant, on ne peut pas l'utiliser directement comme cible de rendu.

Pour des raisons de sécurité et de flexibilité, Vulkan exige la création d'une **Vue d'Image** (**VkImageView**). Une **VkImageView** enveloppe une **VkImage** et lui donne un contexte d'interprétation précis : * Est-ce une texture 2D, 3D, ou une texture cube ? * Comment interpréter les canaux (ex: intervertir le Rouge et le Bleu) ? * À quelle plage de Mipmaps ou de couches de texture avons-nous accès ?

3. Implémentation Pratique Pas-à-Pas

Nous allons maintenant écrire le code C++ permettant de configurer et de créer la Swapchain ainsi que ses vues d'images associées.

3.1 Structure de Support de la Swapchain

Pour structurer notre code, nous définissons une structure C++ regroupant les trois propriétés de compatibilité :

```
#include <vulkan/vulkan.h>
#include <vector>

struct SwapChainSupportDetails {
    VkSurfaceCapabilitiesKHR capabilities;
    std::vector<VkSurfaceFormatKHR> formats;
    std::vector<VkPresentModeKHR> presentModes;
};
```

3.2 Requête des Détails de Support

Nous devons interroger le périphérique physique (`VkPhysicalDevice`) pour remplir cette structure. Nous utilisons ici le motif classique de Vulkan en deux appels pour récupérer les tableaux dynamiques.

```

SwapChainSupportDetails querySwapChainSupport(VkPhysicalDevice device, VkSurfaceKHR surface,
    SwapChainSupportDetails details;

    // 1. Récupération des capacités de base
    vkGetPhysicalDeviceSurfaceCapabilitiesKHR(device, surface, &details.capabilities);

    // 2. Récupération des formats de pixels supportés
    uint32_t formatCount;
    vkGetPhysicalDeviceSurfaceFormatsKHR(device, surface, &formatCount, nullptr);

    if (formatCount != 0) {
        details.formats.resize(formatCount);
        vkGetPhysicalDeviceSurfaceFormatsKHR(device, surface, &formatCount, details.formats);
    }

    // 3. Récupération des modes de présentation supportés
    uint32_t presentModeCount;
    vkGetPhysicalDeviceSurfacePresentModesKHR(device, surface, &presentModeCount, nullptr);

    if (presentModeCount != 0) {
        details.presentModes.resize(presentModeCount);
        vkGetPhysicalDeviceSurfacePresentModesKHR(device, surface, &presentModeCount, details.presentModes);
    }

    return details;
}

```

3.3 Sélection des Meilleurs Paramètres

Choix du Format de Surface

Nous recherchons idéalement un format de couleur 8 bits non-linéaire sRGB (`VK_FORMAT_B8G8R8A8_SRGB`) pour une fidélité de couleur optimale sur les écrans modernes.

```

VkSurfaceFormatKHR chooseSwapSurfaceFormat(const std::vector<VkSurfaceFormatKHR>& availableFormats) {
    for (const auto& availableFormat : availableFormats) {
        if (availableFormat.format == VK_FORMAT_B8G8R8A8_SRGB &&
            availableFormat.colorSpace == VK_COLOR_SPACE_SRGB_NONLINEAR_KHR) {
            return availableFormat;
        }
    }
    // Si notre format idéal n'est pas disponible, on retourne le premier de la liste
    return availableFormats[0];
}

```

Choix du Mode de Présentation

Nous privilégions le mode **MAILBOX** (Triple Buffering à faible latence). S'il n'est pas disponible, nous nous rabattons sur le mode **FIFO** qui est garanti par la spécification Vulkan.

```

VkPresentModeKHR chooseSwapPresentMode(const std::vector<VkPresentModeKHR>& availablePresentModes) {
    for (const auto& availablePresentMode : availablePresentModes) {
        if (availablePresentMode == VK_PRESENT_MODE_MAILBOX_KHR) {
            return availablePresentMode;
        }
    }
    // Garanti d'être présent
    return VK_PRESENT_MODE_FIFO_KHR;
}

```

Choix de la Dimension des Images (Extent)

L'étendue de la Swapchain correspond à la résolution des images. Généralement, elle correspond exactement à la taille de la fenêtre. Cependant, certains systèmes (comme les écrans Retina d'Apple) utilisent des coordonnées logiques différentes des pixels physiques. Nous devons donc utiliser **glfwGetFramebufferSize** pour obtenir la taille réelle en pixels.

```

#include <GLFW/glfw3.h>
#include <algorithm> // Pour std::clamp

VkExtent2D chooseSwapExtent(const VkSurfaceCapabilitiesKHR& capabilities, GLFWwindow* window)
{
    // Si la largeur actuelle n'est pas égale à la valeur maximale d'un entier non signé,
    // cela signifie que le système nous impose la taille de la fenêtre.
    if (capabilities.currentExtent.width != std::numeric_limits<uint32_t>::max()) {
        return capabilities.currentExtent;
    } else {
        // Sinon, nous devons interroger la taille réelle du framebuffer de la fenêtre
        int width, height;
        glfwGetFramebufferSize(window, &width, &height);

        VkExtent2D actualExtent = {
            static_cast<uint32_t>(width),
            static_cast<uint32_t>(height)
        };

        // On s'assure que la taille choisie respecte les limites matérielles du GPU
        actualExtent.width = std::clamp(actualExtent.width,
                                       capabilities.minImageExtent.width,
                                       capabilities.maxImageExtent.width);
        actualExtent.height = std::clamp(actualExtent.height,
                                       capabilities.minImageExtent.height,
                                       capabilities.maxImageExtent.height);

        return actualExtent;
    }
}

```

3.4 Création de la Swapchain

Nous assemblons maintenant ces briques logiques pour initialiser l'objet `VkSwapchainKHR`.

```

// Variables globales ou membres de classe requis pour la création
VkDevice device; // Créé au Module 1.1
VkSurfaceKHR surface; // Créée au Module 1.2
GLFWwindow* window;

VkSwapchainKHR swapChain;
std::vector<VkImage> swapChainImages;
VkFormat swapChainImageFormat;
VkExtent2D swapChainExtent;

void createSwapChain() {
    SwapChainSupportDetails swapChainSupport = querySwapChainSupport(device, surface);

    VkSurfaceFormatKHR surfaceFormat = chooseSwapSurfaceFormat(swapChainSupport.formats);
    VkPresentModeKHR presentMode = chooseSwapPresentMode(swapChainSupport.presentModes);
    VkExtent2D extent = chooseSwapExtent(swapChainSupport.capabilities, window);

    // Déterminer le nombre d'images dans notre swapchain.
    // Il est recommandé de demander au moins une image de plus que le minimum requis
    // pour éviter d'attendre que le pilote termine ses opérations internes.
    uint32_t imageCount = swapChainSupport.capabilities.minImageCount + 1;

    // On s'assure de ne pas dépasser le nombre maximum d'images supporté (0 signifie pas de limite)
    if (swapChainSupport.capabilities.maxImageCount > 0 &&
        imageCount > swapChainSupport.capabilities.maxImageCount) {
        imageCount = swapChainSupport.capabilities.maxImageCount;
    }

    VkSwapchainCreateInfoKHR createInfo{};
    createInfo.sType = VK_STRUCTURE_TYPE_SWAPCHAIN_CREATE_INFO_KHR;
    createInfo.surface = surface;
    createInfo.minImageCount = imageCount;
    createInfo.imageFormat = surfaceFormat.format;
    createInfo.imageColorSpace = surfaceFormat.colorSpace;
    createInfo.imageExtent = extent;
    createInfo.imageArrayLayers = 1; // Toujours 1 sauf pour les applications 3D stéréoscopiques
    createInfo.imageUsage = VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT; // Rendu direct sur l'image

    // Gestion du partage des images entre les files d'attente (Queue Families)
    // (Voir Module 1.2 pour la sélection des indices de familles de files d'attente)
    uint32_t queueFamilyIndices[] = { graphicsFamilyIndex, presentFamilyIndex };

    if (graphicsFamilyIndex != presentFamilyIndex) {
        // Si les files de rendu et de présentation sont différentes, on partage les images
        createInfo.imageSharingMode = VK_SHARING_MODE_CONCURRENT;
        createInfo.queueFamilyIndexCount = 2;
        createInfo.pQueueFamilyIndices = queueFamilyIndices;
    } else {

```

```

        // Mode exclusif pour de meilleures performances si c'est la même file d'attente
        createInfo.imageSharingMode = VK_SHARING_MODE_EXCLUSIVE;
        createInfo.queueFamilyIndexCount = 0; // Optionnel
        createInfo.pQueueFamilyIndices = nullptr; // Optionnel
    }

    createInfo.preTransform = swapChainSupport.capabilities.currentTransform;
    createInfo.compositeAlpha = VK_COMPOSITE_ALPHA_OPAQUE_BIT_KHR; // Ignorer la transparence
    createInfo.presentMode = presentMode;
    createInfo.clipped = VK_TRUE; // Permet d'ignorer les pixels masqués par une autre fenêtre
    createInfo.oldSwapchain = VK_NULL_HANDLE; // Utilisé lors de la re-création (voir section 3.6)

    if (vkCreateSwapchainKHR(device, &createInfo, nullptr, &swapChain) != VK_SUCCESS) {
        throw std::runtime_error("Échec de la création de la Swapchain !");
    }

    // Récupération des handles des images créées automatiquement par la Swapchain
    vkGetSwapchainImagesKHR(device, swapChain, &imageCount, nullptr);
    swapChainImages.resize(imageCount);
    vkGetSwapchainImagesKHR(device, swapChain, &imageCount, swapChainImages.data());

    // Sauvegarde du format et de l'étendue pour utilisation future
    swapChainImageFormat = surfaceFormat.format;
    swapChainExtent = extent;
}

```

3.5 Création des Vues d'Images (Image Views)

Pour chaque image de la Swapchain, nous devons créer sa `VkImageView` correspondante.


```

std::vector<VkImageView> swapChainImageViews;

void createImageViews() {
    swapChainImageViews.resize(swapChainImages.size());

    for (size_t i = 0; i < swapChainImages.size(); i++) {
        VkImageViewCreateInfo createInfo{};
        createInfo.sType = VK_STRUCTURE_TYPE_IMAGE_VIEW_CREATE_INFO;
        createInfo.image = swapChainImages[i];
        createInfo.viewType = VK_IMAGE_VIEW_TYPE_2D; // Traitée comme texture 2D
        createInfo.format = swapChainImageFormat;

        // Configuration des canaux de couleur (Swizzle)
        // Ici, on garde la correspondance par défaut (identité)
        createInfo.components.r = VK_COMPONENT_SWIZZLE_IDENTITY;
        createInfo.components.g = VK_COMPONENT_SWIZZLE_IDENTITY;
        createInfo.components.b = VK_COMPONENT_SWIZZLE_IDENTITY;
        createInfo.components.a = VK_COMPONENT_SWIZZLE_IDENTITY;

        // Description des sous-ressources de l'image
        createInfo.subresourceRange.aspectMask = VK_IMAGE_ASPECT_COLOR_BIT; // Canal de couleur
        createInfo.subresourceRange.baseMipLevel = 0; // Pas de mipmapping pour la swapchain
        createInfo.subresourceRange.levelCount = 1;
        createInfo.subresourceRange.baseArrayLayer = 0;
        createInfo.subresourceRange.layerCount = 1;

        if (vkCreateImageView(device, &createInfo, nullptr, &swapChainImageViews[i]) != VK_SUCCESS)
            throw std::runtime_error("Échec de la création d'une vue d'image !")
    }
}

```

3.6 Nettoyage de la Mémoire

La Swapchain et ses vues d'images doivent être explicitement détruites avant la destruction du périphérique logique (`VkDevice`).

```

void cleanupSwapChain() {
    // 1. D  truire d'abord les vues d'images
    for (auto imageView : swapChainImageViews) {
        vkDestroyImageView(device, imageView, nullptr);
    }
    swapChainImageViews.clear();

    // 2. D  truire la Swapchain
    // Note : Les VkImage de la Swapchain sont automatiquement d  truites avec elles.
    // Il ne faut JAMAIS appeler vkDestroyImage sur les images de la Swapchain !
    if (swapChain != VK_NULL_HANDLE) {
        vkDestroySwapchainKHR(device, swapChain, nullptr);
    }
}

```

4. Pi  ges Fr  quents et Bonnes Pratiques

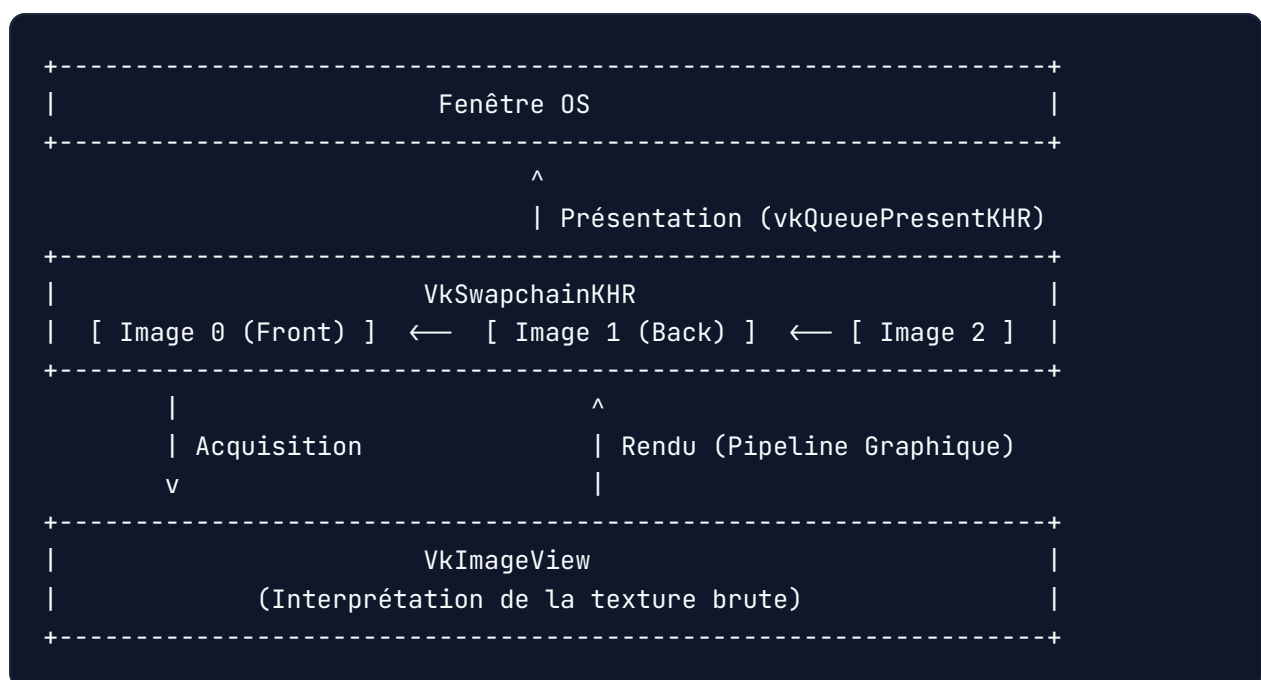
1. Le Pi  ge des   crans Haute Densit   (Retina/High-DPI) :
2. *Erreur* : Utiliser la taille de la fen  tre retourn  e par `glfwGetWindowSize` pour d  finir l'  tendue de la Swapchain.
3. *Cons  quence* : Une image floue ou un crash lors de la cr  ation de la Swapchain car la taille demand  e ne correspond pas    la taille physique du framebuffer.
4. *Solution* : Toujours utiliser `glfwGetFramebufferSize` .
5. La Destruction Incorrecte des Images :
6. *Erreur* : Tenter d'appeler `vkDestroyImage` sur les   l  ments du tableau `swapChainImages` .
7. *Cons  quence* : Crash critique de l'application (Violation d'acc  s m  moire).
8. *Explication* : Ces images appartiennent    l'impl  mentation de la Swapchain de l'OS. Elles sont automatiquement lib  r  es lors de l'appel    `vkDestroySwapchainKHR` . Seules les `VkImageView` doivent   tre d  truites manuellement par votre code.
9. Le Blocage du GPU lors de l'Acquisition :

10. *Erreur* : Demander exactement le nombre d'images minimal (`minImageCount`) sans ajouter de marge.
 11. *Conséquence* : Le GPU peut se retrouver bloqué à attendre qu'une image soit libérée par le serveur d'affichage de l'OS avant de pouvoir commencer à dessiner la frame suivante.
 12. *Solution* : Toujours demander `minImageCount + 1` tout en vérifiant que cela ne dépasse pas `maxImageCount` .
-

5. Synthèse Pédagogique

5.1 Flux de Données de Présentation

Le schéma ci-dessous résume le cycle de vie d'une image dans notre architecture graphique :



5.2 Résumé des Étapes de Configuration

1. Interroger le GPU pour obtenir les capacités, les formats et les modes de présentation supportés pour notre surface.
2. Sélectionner le meilleur format de couleur (idéalement sRGB).
3. Sélectionner le mode de présentation (idéalement Mailbox pour le triple buffering).

4. Calculer la résolution réelle en pixels physiques (Extent).
 5. Créer la `VkSwapchainKHR`.
 6. Récupérer les handles des `VkImage` générées par la Swapchain.
 7. Envelopper chaque image dans une `VkImageView` pour la rendre exploitable par la suite de notre pipeline graphique (qui sera étudiée au Module 2).
-

6. Exercice Pratique d'Application

Énoncé

Lorsqu'un utilisateur redimensionne la fenêtre de l'application, la Swapchain existante devient invalide (sa taille ne correspond plus à la surface d'affichage). Vulkan ne gère pas ce redimensionnement automatiquement. L'application doit détecter ce changement, détruire l'ancienne Swapchain et en recréer une nouvelle avec les dimensions mises à jour.

Votre objectif est d'implémenter la fonction `recreateSwapChain()` qui gère proprement cette transition sans fuite de mémoire et en gérant le cas particulier où la fenêtre est minimisée (taille de 0×0 pixels).

Indices

1. Si la fenêtre est minimisée, la largeur ou la hauteur du framebuffer sera égale à `0`. Dans ce cas, l'application doit se mettre en pause et attendre (`glfwWaitEvents()`) jusqu'à ce que la fenêtre soit restaurée.
2. Vous devez appeler `vkDeviceWaitIdle(device)` avant de commencer la destruction pour vous assurer que le GPU a fini d'exécuter toutes ses commandes en cours.
3. Pour une transition fluide sans scintillement visuel, vous pouvez passer le handle de l'ancienne Swapchain au champ `oldSwapchain` de la structure `VkSwapchainCreateInfoKHR` lors de la création de la nouvelle.

Correction Complète

Voici l'implémentation C++ complète et robuste de la gestion du redimensionnement :

```

#include <iostream>

// Variable globale pour suivre si un redimensionnement a eu lieu
bool framebufferResized = false;

// Callback GLFW appelé automatiquement lors d'un redimensionnement
void framebufferResizeCallback(GLFWwindow* window, int width, int height) {
    framebufferResized = true;
}

// Fonction de nettoyage spécifique à la Swapchain
void cleanupSwapChain() {
    // 1. Destruction des vues d'images
    for (size_t i = 0; i < swapChainImageViews.size(); i++) {
        vkDestroyImageView(device, swapChainImageViews[i], nullptr);
    }
    swapChainImageViews.clear();

    // 2. Destruction de la Swapchain elle-même
    if (swapChain != VK_NULL_HANDLE) {
        vkDestroySwapchainKHR(device, swapChain, nullptr);
        swapChain = VK_NULL_HANDLE;
    }
}

// Fonction de re-cr  ation compl  te de la Swapchain
void recreateSwapChain() {
    // Gestion de la minimisation de la fen  tre
    int width = 0, height = 0;
    glfwGetFramebufferSize(window, &width, &height);
    while (width == 0 || height == 0) {
        // On attend un nouvel   v  nement (ex: restauration de la fen  tre)
        glfwGetFramebufferSize(window, &width, &height);
        glfwWaitEvents();
    }

    // Attendre que le GPU ait fini de travailler avant de d  truire les ressources actives
    vkDeviceWaitIdle(device);

    // Nettoyage de l'ancienne Swapchain
    cleanupSwapChain();

    // Recr  ation des ressources avec les nouvelles dimensions de la fen  tre
    createSwapChain();
    createImageViews();

    std::cout << "Swapchain re-creee avec succes ! Nouvelle taille : "
                << swapChainExtent.width << "x" << swapChainExtent.height << std::endl;
}

```

```

}

// Exemple d'intégration dans la boucle de rendu principale
void drawFrame() {
    // [Étape de rendu - Sera détaillée au Module 3.3]

    // Si la présentation de l'image échoue ou si GLFW signale un redimensionnement :
    // (Simulé ici par notre variable booléenne)
    if (framebufferResized) {
        framebufferResized = false;
        recreateSwapChain();
    }
}
}

```

Pipeline Graphique et Gestion des Ressources

Module 2.1 : Shaders et compilation en bytecode SPIR-V

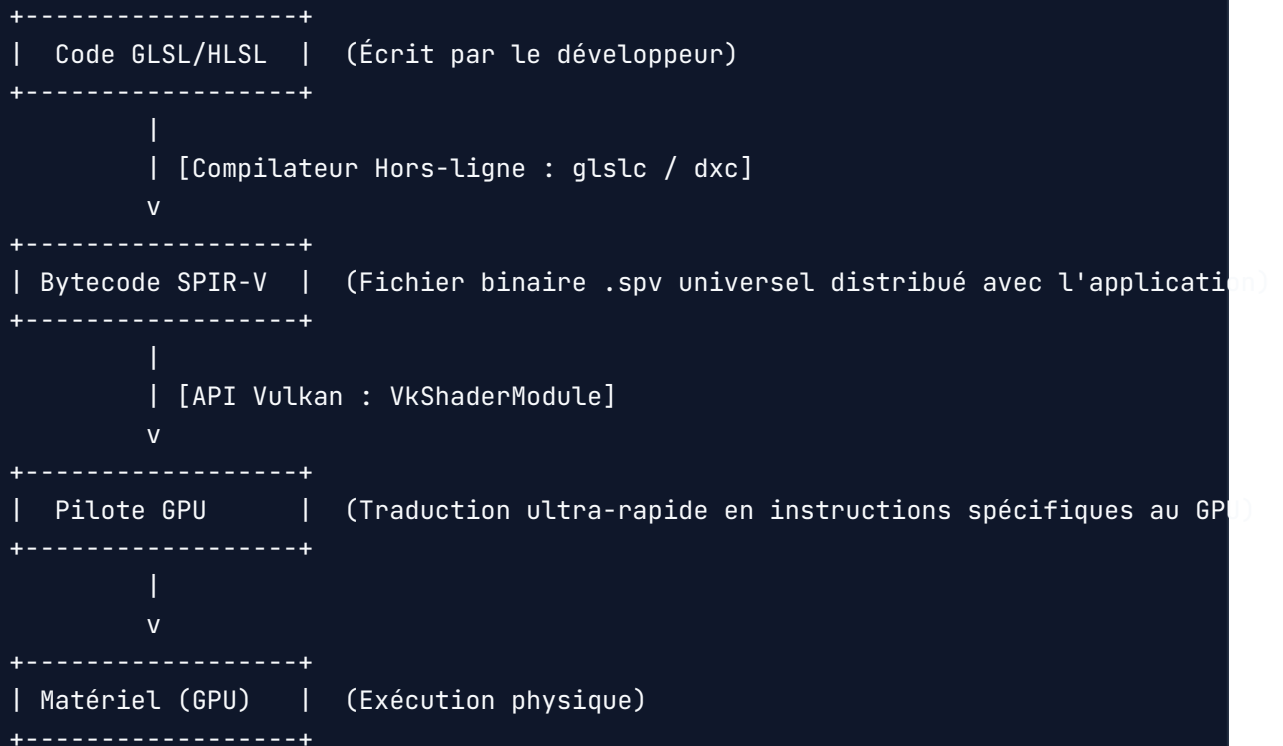
1. Introduction Conceptuelle

Dans les API graphiques historiques comme OpenGL, la programmation des processeurs de la carte graphique (les GPUs) s'effectuait en soumettant directement le code source des shaders (généralement écrit en GLSL) sous forme de chaînes de caractères textuelles. Le pilote de la carte graphique embarquait alors son propre compilateur pour traduire, à la volée et lors de l'exécution de l'application, ce texte en instructions machines spécifiques à l'architecture du GPU hôte.

Bien que flexible, cette approche présentait des inconvénients majeurs : 1. **Temps de chargement imprévisibles** : La compilation de shaders complexes au lancement d'un jeu ou d'une application CAO provoquait des micro-saccades (*stuttering*) ou des temps de chargement excessivement longs. 2. **Incohérence des pilotes** : Chaque constructeur de GPU (NVIDIA, AMD, Intel) implémentait son propre compilateur GLSL. Un shader fonctionnant parfaitement sur une machine pouvait générer des erreurs de syntaxe ou des comportements indéfinis sur une autre à cause de divergences d'interprétation de la norme. 3. **Exposition de la propriété intellectuelle** : Distribuer du code source de shader en clair dans les fichiers de l'application facilitait le vol de technologies et le rétro-ingénierie.

La solution Vulkan : SPIR-V

Vulkan résout radicalement ces problèmes en introduisant un standard intermédiaire nommé SPIR-V (*Standard Portable Intermediate Representation - V*).



SPIR-V est un format binaire intermédiaire, indépendant de la plateforme et du constructeur. Le développeur compile désormais ses shaders **hors-ligne** (pendant la phase de build de l'application) à l'aide d'un compilateur externe. Vulkan ne reçoit donc jamais de code GLSL textuel, mais uniquement ce bytecode binaire pré-analysé et optimisé.

Pourquoi cette approche est-elle supérieure ?

- **Performance** : Le pilote GPU n'a plus besoin d'analyser la syntaxe du code. Il se contente de traduire un bytecode structuré en instructions machines, ce qui est extrêmement rapide.
- **Fiabilité** : Les erreurs de syntaxe sont détectées lors de la compilation hors-ligne sur la machine du développeur, et non chez l'utilisateur final.
- **Portabilité** : Le compilateur de référence étant unique (géré par le groupe Khronos), le comportement du bytecode est garanti identique sur tous les pilotes conformes à la spécification Vulkan.
- **Flexibilité linguistique** : Bien que le GLSL reste le langage de choix, n'importe quel langage capable de compiler vers SPIR-V (comme le HLSL de Microsoft ou même des langages expérimentaux comme Rust via *rust-gpu*) peut être utilisé pour programmer des shaders Vulkan.

2. Fondations Théoriques

Les étapes de la Pipeline Graphique (Shader Stages)

La pipeline graphique de Vulkan est composée de plusieurs étapes, certaines étant configurables (fonctions fixes, étudiées dans le *Module 2.2*), d'autres étant entièrement programmables via des shaders. Pour afficher notre premier triangle, nous devons obligatoirement implémenter deux types de shaders :

1. Le Vertex Shader (Shader de Sommets) :
2. Rôle : Traiter individuellement chaque sommet (*vertex*) géométrique.
3. Entrées : Les attributs du sommet (coordonnées 3D, couleurs, coordonnées de texture).
4. Sorties : La position finale du sommet dans l'espace de projection homogène (écrite dans la variable système globale `gl_Position`) et des variables personnalisées à transmettre à l'étape suivante.
5. Fréquence d'exécution : Une fois par sommet soumis au GPU.
6. Le Fragment Shader (Shader de Pixels) :
7. Rôle : Déterminer la couleur finale de chaque pixel potentiel (appelé *fragment*) couvert par la géométrie projetée à l'écran.
8. Entrées : Les variables interpolées en provenance du Vertex Shader (par exemple, si un sommet est rouge et l'autre bleu, le fragment shader recevra une couleur intermédiaire violette selon sa position sur le triangle).
9. Sorties : La couleur finale du fragment (généralement écrite dans une variable de sortie liée à un framebuffer).
10. Fréquence d'exécution : Potentiellement des millions de fois par seconde, pour chaque pixel couvert par un triangle après l'étape de rasterisation.

Structure du format SPIR-V

Un fichier SPIR-V (`.spv`) est une suite de mots de 32 bits (4 octets). Il commence par un en-tête fixe contenant : * Un nombre magique (`0x07230203`) permettant d'identifier le

format. * La version de la spécification SPIR-V utilisée. * L'identifiant du générateur (par exemple, le compilateur Khronos). * Les limites d'ID internes utilisées pour l'allocation des registres virtuels.

Le reste du fichier contient des instructions sous forme de *bytecodes* structurés (déclaration des capacités requises, des points d'entrée, des types de données, des variables globales, puis le code d'exécution proprement dit).

L'objet Vulkan : `VkShaderModule`

Pour exploiter ce bytecode SPIR-V, Vulkan fournit un objet opaque appelé `VkShaderModule`. Cet objet encapsule le code binaire en mémoire CPU et le prépare à être lié à la pipeline graphique.

Il est crucial de comprendre que `VkShaderModule` n'est qu'un simple conteneur de code. Il ne définit pas encore si le shader est utilisé comme un vertex shader ou un fragment shader, ni comment les variables d'entrée/sortie sont connectées. Cette configuration finale s'effectue lors de la création de la pipeline globale via la structure `VkPipelineShaderStageCreateInfo`.

3. Implémentation Pratique Pas-à-Pas

Étape 1 : Écriture des Shaders en GLSL (Vulkan-compliant)

Pour éviter d'introduire la complexité de la gestion des tampons mémoire (buffers) dès maintenant (ce qui sera traité dans un module ultérieur), nous allons concevoir des shaders dits "autonomes". Les coordonnées et les couleurs des sommets du triangle seront directement codées en dur à l'intérieur du Vertex Shader.

Créez un fichier nommé `shader.vert` (extension conventionnelle pour les Vertex Shaders) :

```

#version 450 // Spécifie l'utilisation de GLSL version 4.5

// Variable de sortie interpolée vers le Fragment Shader
// L'attribut 'layout(location = 0)' est obligatoire sous Vulkan pour lier les variables et
layout(location = 0) out vec3 fragColor;

// Tableau de positions en coordonnées normalisées de périphérique (NDC)
// Sous Vulkan, l'axe Y pointe vers le BAS, contrairement à OpenGL
vec2 positions[3] = vec2[](
    vec2(0.0, -0.5), // Sommet supérieur (milieu)
    vec2(0.5, 0.5),  // Sommet inférieur droit
    vec2(-0.5, 0.5)  // Sommet inférieur gauche
);

// Tableau de couleurs associées à chaque sommet (Rouge, Vert, Bleu)
vec3 colors[3] = vec3[](
    vec3(1.0, 0.0, 0.0), // Rouge
    vec3(0.0, 1.0, 0.0), // Vert
    vec3(0.0, 0.0, 1.0)  // Bleu
);

void main() {
    // gl_VertexIndex est une variable intégrée contenant l'index du sommet en cours de traitement
    gl_Position = vec4(positions[gl_VertexIndex], 0.0, 1.0);

    // Transmission de la couleur correspondante au Fragment Shader
    fragColor = colors[gl_VertexIndex];
}

```

Créez ensuite un fichier nommé `shader.frag` (extension conventionnelle pour les Fragment Shaders):

```
#version 450

// Variable d'entrée interpolée en provenance du Vertex Shader
// L'index de 'location' doit correspondre exactement à celui du Vertex Shader (0)
layout(location = 0) in vec3 fragColor;

// Variable de sortie : la couleur finale du pixel (RGBA)
// 'location = 0' indique que cette couleur s'adresse au premier framebuffer de rendu
layout(location = 0) out vec4 outColor;

void main() {
    // Affectation de la couleur interpolée avec une opacité de 1.0 (totalement opaque)
    outColor = vec4(fragColor, 1.0);
}
```

Étape 2 : Compilation des Shaders en SPIR-V via la ligne de commande

Pour compiler ces fichiers sources en bytecode SPIR-V, nous utilisons l'outil `glslc`, qui est le compilateur officiel fourni par Google et inclus par défaut dans le Vulkan SDK.

Ouvrez votre terminal dans le dossier contenant vos fichiers shaders et exécutez les commandes suivantes :

```
# Compilation du Vertex Shader
glslc shader.vert -o vert.spv

# Compilation du Fragment Shader
glslc shader.frag -o frag.spv
```

Si aucune erreur de syntaxe n'est présente, deux nouveaux fichiers binaires sont générés : `vert.spv` et `frag.spv`. Ce sont ces fichiers que notre programme C++ va charger.

Étape 3 : Chargement robuste du binaire en C++

Pour lire ces fichiers binaires de manière optimale et sécurisée en C++, nous allons implémenter une fonction utilitaire utilisant la bibliothèque standard `<fstream>`.

```

#include <fstream>
#include <vector>
#include <string>
#include <iostream>
#include <stdexcept>

// Fonction lisant l'intégralité d'un fichier binaire et le retournant sous forme de vecteur
std::vector<char> readFile(const std::string& filename) {
    // std::ios::ate : ouvre le fichier et place le curseur de lecture directement à la fin
    // std::ios::binary : ouvre le fichier en mode binaire (évite les traductions de caractères)
    std::ifstream file(filename, std::ios::ate | std::ios::binary);

    if (!file.is_open()) {
        throw std::runtime_error("Échec de l'ouverture du fichier shader : " + filename);
    }

    // Grâce à std::ios::ate, la position actuelle du curseur correspond à la taille du fichier
    size_t fileSize = (size_t) file.tellg();
    std::vector<char> buffer(fileSize);

    // On replace le curseur au début du fichier pour commencer la lecture effective
    file.seekg(0);
    file.read(buffer.data(), fileSize);

    // Fermeture du flux de fichier
    file.close();

    return buffer;
}

```

Étape 4 : Création du `VkShaderModule`

Le bytecode SPIR-V chargé en mémoire doit être transmis à l'API Vulkan. C'est ici qu'intervient la structure `VkShaderModuleCreateInfo`.

Un point technique crucial doit être soulevé : la spécification de Vulkan exige que le pointeur vers le code binaire (`pCode`) soit aligné sur 4 octets (type `uint32_t*`), alors que notre fonction de lecture retourne un tableau d'octets génériques (`char*` , aligné sur 1 octet). Heureusement, l'allocateur par défaut de `std::vector` garantit un alignement mémoire suffisant pour stocker n'importe quel type de données de base, ce qui rend le cast sécurisé si nous prenons des précautions.

Voici la fonction C++ permettant de créer un `VkShaderModule` à partir du tampon d'octets :

```

#include <vulkan/vulkan.h>

VkShaderModule createShaderModule(VkDevice device, const std::vector<char>& code) {
    VkShaderModuleCreateInfo createInfo{};
    createInfo.sType = VK_STRUCTURE_TYPE_SHADER_MODULE_CREATE_INFO;
    createInfo.pNext = nullptr;
    createInfo.flags = 0;

    // Taille du code en octets
    createInfo.codeSize = code.size();

    // Cast du pointeur char* vers uint32_t* pour satisfaire l'alignement requis par Vulkan
    createInfo.pCode = reinterpret_cast<const uint32_t*>(code.data());

    VkShaderModule shaderModule;
    // vkCreateShaderModule prend en paramètres :
    // 1. Le périphérique logique (VkDevice) créé lors des étapes précédentes (Module 1.1)
    // 2. Un pointeur vers la structure de configuration
    // 3. Un pointeur vers des allocateurs personnalisés (nullptr ici)
    // 4. Un pointeur vers la variable qui recevra le handle de l'objet créé
    VkResult result = vkCreateShaderModule(device, &createInfo, nullptr, &shaderModule);

    if (result != VK_SUCCESS) {
        throw std::runtime_error("Échec de la création du module de shader ! Code erreur : " +
                                  std::to_string(result));
    }

    return shaderModule;
}

```

Étape 5 : Configuration des étapes de pipeline

(`VkPipelineShaderStageCreateInfo`)

Une fois les modules créés, nous devons spécifier à Vulkan comment ils s'intègrent dans la pipeline de rendu graphique. Pour chaque shader, nous devons remplir une structure `VkPipelineShaderStageCreateInfo`.

Voici comment configurer l'étape du Vertex Shader et du Fragment Shader dans votre code principal :

```

// 1. Chargement des fichiers binaires SPIR-V
std::vector<char> vertShaderCode = readFile("shaders/vert.spv");
std::vector<char> fragShaderCode = readFile("shaders/frag.spv");

// 2. Création des modules Vulkan correspondants
// Note : 'device' est le VkDevice logique préalablement initialisé
VkShaderModule vertShaderModule = createShaderModule(device, vertShaderCode);
VkShaderModule fragShaderModule = createShaderModule(device, fragShaderCode);

// 3. Configuration de l'étape Vertex Shader
VkPipelineShaderStageCreateInfo vertShaderStageInfo{};
vertShaderStageInfo.sType = VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO;
vertShaderStageInfo.pNext = nullptr;
vertShaderStageInfo.flags = 0;
// Indique que cette structure configure l'étape Vertex
vertShaderStageInfo.stage = VK_SHADER_STAGE_VERTEX_BIT;
// Le module contenant le code
vertShaderStageInfo.module = vertShaderModule;
// Le nom de la fonction d'entrée (entry point) définie dans le code GLSL
vertShaderStageInfo.pName = "main";
// pSpecializationInfo permet de définir des constantes de spécialisation au moment de la création de la pipeline
// Nous n'en utilisons pas pour le moment, donc nous passons nullptr.
vertShaderStageInfo.pSpecializationInfo = nullptr;

// 4. Configuration de l'étape Fragment Shader
VkPipelineShaderStageCreateInfo fragShaderStageInfo{};
fragShaderStageInfo.sType = VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO;
fragShaderStageInfo.pNext = nullptr;
fragShaderStageInfo.flags = 0;
// Indique que cette structure configure l'étape Fragment
fragShaderStageInfo.stage = VK_SHADER_STAGE_FRAGMENT_BIT;
fragShaderStageInfo.module = fragShaderModule;
fragShaderStageInfo.pName = "main";
fragShaderStageInfo.pSpecializationInfo = nullptr;

// 5. Regroupement des configurations dans un tableau pour la création ultérieure de la pipeline
VkPipelineShaderStageCreateInfo shaderStages[] = { vertShaderStageInfo, fragShaderStageInfo };

```

Étape 6 : Nettoyage des ressources

Une fois que la pipeline graphique globale (`VkPipeline`) a été compilée avec succès (ce qui sera traité dans le *Module 2.2*), le bytecode SPIR-V a été entièrement traduit en instructions machines spécifiques au GPU hôte.

Les objets intermédiaires `VkShaderModule` ne sont alors plus nécessaires en mémoire. Vous devez impérativement libérer ces ressources pour éviter les fuites de mémoire.

```
// Libération des modules de shader
vkDestroyShaderModule(device, fragShaderModule, nullptr);
vkDestroyShaderModule(device, vertShaderModule, nullptr);
```

4. Pièges Fréquents et Bonnes Pratiques

1. Le piège de l'alignement mémoire de `pCode`

Comme mentionné précédemment, `VkShaderModuleCreateInfo::pCode` attend un pointeur de type `const uint32_t*`. Si vous lisez votre fichier dans un `std::vector<char>`, la mémoire sous-jacente pourrait théoriquement ne pas être alignée sur une frontière de 4 octets sur certaines architectures exotiques. * Bonne pratique : Pour une sécurité absolue, vous pouvez lire directement le fichier dans un `std::vector<uint32_t>`. La taille du vecteur doit alors être ajustée car chaque élément fait 4 octets au lieu d'un.

```
std::vector<uint32_t> readShaderFileSecure(const std::string& filename) {
    std::ifstream file(filename, std::ios::ate | std::ios::binary);
    if (!file.is_open()) throw std::runtime_error("Erreur d'ouverture");

    size_t fileSize = (size_t)file.tellg();
    // On s'assure que la taille est bien un multiple de 4 octets
    if (fileSize % 4 != 0) throw std::runtime_error("Fichier SPIR-V corrompu (taille non alignée)");

    std::vector<uint32_t> buffer(fileSize / 4);
    file.seekg(0);
    file.read(reinterpret_cast<char*>(buffer.data()), fileSize);
    file.close();
    return buffer;
}
```

2. Durée de vie des modules de shader

Un piège classique consiste à détruire les `VkShaderModule` trop tôt, par exemple juste après avoir rempli la structure `VkPipelineShaderStageCreateInfo` mais *avant* d'avoir

effectivement appelé la fonction de création de la pipeline (`vkCreateGraphicsPipelines`). *

Règle absolue : Les modules de shader doivent rester valides et ne pas être détruits tant que la pipeline graphique qui les utilise n'a pas été entièrement créée.

3. Gestion des chemins d'accès relatifs

Lors de l'exécution de votre programme C++, le répertoire de travail par défaut peut varier selon que vous lancez l'application depuis votre IDE (Visual Studio, CLion) ou directement depuis le terminal. Si l'application ne trouve pas les fichiers `.spv`, elle plantera immédiatement. * **Bonne pratique** : Utilisez des chemins absolus générés dynamiquement ou configurez le répertoire de travail de votre IDE pour qu'il pointe vers le dossier contenant vos assets/shaders compilés.

5. Synthèse Pédagogique

Tableau récapitulatif du cycle de vie d'un Shader sous Vulkan

Étape	Format de données	Outil / API	Localisation
1. Écriture	Code source textuel (<code>.vert</code> , <code>.frag</code>)	Éditeur de texte	CPU (Développement)
2. Compilation	Bytecode binaire intermédiaire (<code>.spv</code>)	<code>glslc</code> (compilateur hors-ligne)	CPU (Build-time)
3. Chargement	Tableau d'octets en mémoire vive	<code>std::ifstream</code> (C++)	RAM (Runtime)
4. Encapsulation	<code>VkShaderModule</code>	<code>vkCreateShaderModule</code>	RAM / Pilote
5. Liaison	<code>VkPipelineShaderStageCreateInfo</code>	Configuration de la Pipeline	GPU (Compilation interne)
6. Destruction	Libération de la mémoire	<code>vkDestroyShaderModule</code>	RAM (Runtime)

6. Exercice Pratique d'Application

Objectif : Inverser le triangle et appliquer un effet de grisaille (Monochrome)

Pour valider votre compréhension du flux de travail des shaders, vous allez modifier les shaders existants pour accomplir deux tâches : 1. Dans le Vertex Shader : Inverser verticalement le triangle (le sommet pointant vers le haut doit désormais pointer vers le bas). 2. Dans le Fragment Shader : Convertir la couleur interpolée (RGB) en niveaux de gris en utilisant la formule de luminance standard : $Y = 0.2126 \times R + 0.7152 \times G + 0.0722 \times B$

Indices

- Pour inverser verticalement le triangle, observez le tableau `positions` dans `shader.vert`. Les coordonnées Y des sommets inférieurs sont positives (`0.5`), et celle du sommet supérieur est négative (`-0.5`). Rappelez-vous que sous Vulkan, l'axe Y pointe vers le bas.
- Pour le calcul de la luminance dans `shader.frag`, vous pouvez utiliser le produit scalaire (`dot`) entre le vecteur de couleur RGB et un vecteur contenant les coefficients de pondération, ou effectuer une multiplication membre à membre.

Correction de l'exercice

1. Nouveau code pour `shader.vert` (Inversion verticale)

Pour inverser le triangle, il suffit de multiplier la coordonnée Y de chaque position par `-1.0`, ou de modifier directement les valeurs dans le tableau :

```

#version 450

layout(location = 0) out vec3 fragColor;

// Positions modifiées pour pointer vers le bas
vec2 positions[3] = vec2[](
    vec2(0.0, 0.5),    // Sommet désormais en bas (Y positif)
    vec2(0.5, -0.5),   // Sommet en haut à droite (Y négatif)
    vec2(-0.5, -0.5)   // Sommet en haut à gauche (Y négatif)
);

vec3 colors[3] = vec3[](
    vec3(1.0, 0.0, 0.0),
    vec3(0.0, 1.0, 0.0),
    vec3(0.0, 0.0, 1.0)
);

void main() {
    gl_Position = vec4(positions[gl_VertexIndex], 0.0, 1.0);
    fragColor = colors[gl_VertexIndex];
}

```

2. Nouveau code pour `shader.frag` (Conversion en niveaux de gris)

Nous appliquons la formule de luminance pour calculer une valeur d'intensité unique, puis nous l'assignons aux canaux Rouge, Vert et Bleu de la couleur de sortie.

```

#version 450

layout(location = 0) in vec3 fragColor;
layout(location = 0) out vec4 outColor;

void main() {
    // Coefficients de luminance standards (norme ITU-R BT.709)
    vec3 coefficients = vec3(0.2126, 0.7152, 0.0722);

    // Calcul de la luminance via un produit scalaire (dot product)
    float gray = dot(fragColor, coefficients);

    // La couleur de sortie possède la même valeur d'intensité sur R, G et B
    outColor = vec4(vec3(gray), 1.0);
}

```

3. Recompile des shaders

N'oubliez pas de recompiler vos shaders modifiés en exécutant à nouveau les commandes de compilation dans votre terminal :

```
glslc shader.vert -o vert.spv  
glslc shader.frag -o frag.spv
```

Relancez votre application C++ : le triangle s'affiche désormais la pointe vers le bas, entièrement rendu dans un dégradé de niveaux de gris !

Module 2.2 : Configuration des fonctions fixes de la Pipeline (PSO)

Dans le module précédent (Module 2.1), nous avons vu comment programmer les étapes de traitement des Shaders (Vertex et Fragment) et les compiler au format universel SPIR-V. Cependant, une carte graphique (GPU) ne se contente pas d'exécuter du code de shader. Entre le moment où vous lui envoyez des coordonnées géométriques et celui où les pixels s'affichent à l'écran, les données traversent une série d'étapes matérielles non programmables mais hautement configurables : ce sont les fonctions fixes.

Ce module détaille comment configurer ces étapes matérielles à travers l'objet central de Vulkan : le Pipeline State Object (PSO).

1. Introduction Conceptuelle

1.1 Qu'est-ce qu'un Pipeline State Object (PSO) ?

Dans les API graphiques historiques (comme OpenGL), la pipeline graphique était gérée comme une "machine à états" globale. Vous pouviez modifier n'importe quel état (activer le test de profondeur, changer la couleur de fond, modifier le mode de dessin) à tout moment, juste avant de dessiner.

Bien que flexible, cette approche pose un problème majeur sur les GPU modernes : chaque fois qu'un état change, le pilote de la carte graphique doit réévaluer la cohérence de toute la configuration et souvent recompiler ou réorganiser les instructions matérielles en arrière-plan. Cela provoque des micro-saccades (*stutters*) imprévisibles durant le jeu ou l'application.

Vulkan résout ce problème en introduisant le PSO (`VkPipeline`).

Un PSO est un objet monolithique et immuable qui contient la configuration absolue et complète de la pipeline : * Les shaders compilés (Vertex, Fragment, etc.). * Le format des

données d'entrée (Vertex Input). * La manière d'assembler les géométries (Input Assembly). * La configuration de la caméra et de la zone d'affichage (Viewport & Scissor). * Le comportement de la rasterisation (Rasterizer). * Les calculs de transparence et de mélange (Color Blending). * Les tests de visibilité (Depth & Stencil).

```
[Données de sommets] —> [Vertex Input] —> [Input Assembly] —> [Vertex Shader]
                                                                |
[Color Blending] <— [Fragment Shader] <— [Rasterizer] <— [Viewport & Scissor]
```

Une fois créé, un PSO ne peut plus être modifié. Si vous voulez dessiner un objet en fil de fer (wireframe) puis un autre en relief plein, vous devez créer deux objets `VkPipeline` distincts et basculer de l'un à l'autre. Le GPU connaît ainsi instantanément et à l'avance la configuration matérielle requise, garantissant des performances maximales et constantes.

1.2 Le compromis : Les États Dynamiques (Dynamic States)

Parce que reconstruire une pipeline entière pour un simple changement de taille de fenêtre serait extrêmement coûteux, Vulkan permet de déclarer certains états comme **dynamiques**. Ces états spécifiques sont extraits du PSO et pourront être modifiés directement dans les tampons de commandes (étudiés au Module 3.1). Les exemples les plus courants sont la taille de la zone d'affichage (Viewport) et le rectangle de découpe (Scissor).

2. Fondations Théoriques

Pour configurer correctement notre PSO, nous devons comprendre le rôle de chaque étape matérielle fixe du GPU.

2.1 L'Entrée des Sommets (Vertex Input)

Cette étape décrit la structure des données que vous allez envoyer au Vertex Shader. *

Bindings (Liaisons) : Définissent l'espacement en mémoire (*stride*) entre deux sommets consécutifs et le mode de lecture (par sommet ou par instance). *

Attributes (Attributs) : Définissent le type de données de chaque variable (ex: une position 3D sous forme de 3 *floats*, une couleur sous forme de 4 *floats*) et leur décalage (*offset*) mémoire par rapport au début du sommet.

Note : Pour notre premier triangle (Module 3.3), nous coderons les coordonnées directement dans le Vertex Shader. Cette étape sera donc temporairement configurée pour n'attendre aucune donnée d'entrée.

2.2 L'Assemblage des Primitives (Input Assembly)

Cette étape prend le flux de sommets et détermine comment ils doivent être reliés pour former des formes géométriques. * Topologies courantes : *

- * `VK_PRIMITIVE_TOPOLOGY_POINT_LIST` : Chaque sommet est un point isolé. *
- * `VK_PRIMITIVE_TOPOLOGY_LINE_LIST` : Chaque paire de sommets forme un segment indépendant. *
- * `VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST` : Chaque groupe de 3 sommets forme un triangle indépendant. *

* Primitive Restart : Permet de briser une bande continue de lignes ou de triangles (strip) en utilisant un index spécial (ex: `0xFFFF`).

2.3 Le Viewport et le Scissor

Ces deux concepts définissent la zone de rendu finale sur l'écran ou l'image cible.

```
+-----+
| Fenêtre d'affichage (Render Target) |
| +-----+ |
| | Viewport (Zone de dessin) | | | |
| | +-----+ | |
| | | Scissor | | |
| | | (Découpe) | | |
| | +-----+ | |
| +-----+ |
+-----+
```

- **Viewport (Fenêtre d'affichage)** : Définit la transformation des coordonnées normalisées du GPU (NDC, allant de -1 à 1) vers l'espace pixel de votre écran (ex: de 0 à 1920 en X, et de 0 à 1080 en Y). Il gère également la profondeur (Z, généralement de 0.0 à 1.0).
- **Scissor (Rectangle de découpe)** : Agit comme un masque de pixels. Tout pixel situé en dehors de ce rectangle défini sera purement et simplement jeté par le GPU, sans même exécuter le Fragment Shader.

2.4 La Rasterisation (Rasterizer)

La rasterisation est l'étape clé qui transforme la géométrie vectorielle (les triangles mathématiques) en fragments (les futurs pixels potentiels à l'écran). * **Polygon Mode** : Détermine si les triangles sont dessinés pleins (`VK_POLYGON_MODE_FILL`), sous forme de lignes (`VK_POLYGON_MODE_LINE` / wireframe) ou de points (`VK_POLYGON_MODE_POINT`). * **Culling** (Élimination des faces cachées) : Permet de ne pas dessiner les faces arrière des objets 3D (les faces orientées vers l'intérieur d'un cube par exemple). Cela double virtuellement les performances géométriques. * **Front Face** : Définit le sens d'enroulement des sommets pour déterminer quelle face est l'avant (sens horaire `CW` ou trigonométrique `CCW`).

2.5 Le Multisampling (Antialiasing)

Le multisampling (MSAA) permet de lisser les bords crénelés (l'effet d'escalier ou *aliasing*) des géométries en combinant les informations de plusieurs échantillons par pixel. Pour débiter, nous désactiverons cette option en la configurant sur 1 seul échantillon par pixel.

2.6 Le Mélange des Couleurs (Color Blending)

Une fois que le Fragment Shader a calculé une couleur pour un pixel, cette couleur doit être écrite dans l'image de destination (le *Framebuffer*). Le mélange définit comment combiner cette nouvelle couleur (*Source*) avec la couleur déjà présente à cet endroit (*Destination*). * **Mode Écriture Directe** : La nouvelle couleur écrase l'ancienne. * **Mode Mélange Alpha** (*Transparence*) : Combine les couleurs en fonction de la valeur de transparence (Alpha). La formule mathématique standard appliquée par le GPU est :

$$\text{Couleur}_{\text{finale}} = (\text{Couleur}_{\text{source}} \times \text{Alpha}_{\text{source}}) + (\text{Couleur}_{\text{destination}} \times (1 - \text{Alpha}_{\text{source}}))$$

2.7 Le Pipeline Layout (Agencement de la Pipeline)

Bien que nous n'utilisions pas encore de variables globales (appelées *Descriptors* ou *Uniforms* en Vulkan) pour notre premier triangle, le GPU doit savoir à l'avance quelles ressources externes la pipeline va utiliser. Cet agencement est décrit par l'objet `VkPipelineLayout` .

3. Implémentation Pratique Pas-à-Pas

Vulkan étant une API C, la configuration de chaque étape passe par le remplissage de structures de données spécifiques. Nous allons détailler chaque structure nécessaire à la création de notre pipeline fixe.

3.1 Comprendre le motif de conception Vulkan

Toutes les structures Vulkan commencent par deux champs obligatoires : 1. `sType` (Structure Type) : Indique explicitement au pilote le type de la structure (ex: `VK_STRUCTURE_TYPE_PIPELINE_VERTEX_INPUT_STATE_CREATE_INFO`). 2. `pNext` : Un pointeur vers une autre structure pour étendre les fonctionnalités de l'API (généralement initialisé à `nullptr`).

3.2 Code C++ de Configuration des Structures

Voici l'implémentation complète et rigoureuse de la configuration des fonctions fixes.

```

#include <vulkan/vulkan.h>
#include <vector>
#include <iostream>

// Fonction globale de configuration de la pipeline fixe
VkPipelineLayout pipelineLayout;

void setupFixedFunctions() {

    // =====
    // 1. CONFIGURATION DE L'ENTRÉE DES SOMMETS (VERTEX INPUT)
    // =====
    VkPipelineVertexInputStateCreateInfo vertexInputInfo{};
    vertexInputInfo.sType = VK_STRUCTURE_TYPE_PIPELINE_VERTEX_INPUT_STATE_CREATE_INFO;
    vertexInputInfo.pNext = nullptr;
    vertexInputInfo.flags = 0;

    // Pour notre premier triangle, nous n'injectons pas de données de sommets directes.
    // Les données seront codées en dur dans le Vertex Shader (Module 2.1).
    vertexInputInfo.vertexBindingDescriptionCount = 0;
    vertexInputInfo.pVertexBindingDescriptions = nullptr;
    vertexInputInfo.vertexAttributeDescriptionCount = 0;
    vertexInputInfo.pVertexAttributeDescriptions = nullptr;

    // =====
    // 2. CONFIGURATION DE L'ASSEMBLAGE DES PRIMITIVES (INPUT ASSEMBLY)
    // =====
    VkPipelineInputAssemblyStateCreateInfo inputAssembly{};
    inputAssembly.sType = VK_STRUCTURE_TYPE_PIPELINE_INPUT_ASSEMBLY_STATE_CREATE_INFO;
    inputAssembly.pNext = nullptr;
    inputAssembly.flags = 0;

    // Nous voulons dessiner des triangles à partir de nos sommets
    inputAssembly.topology = VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST;

    // Désactivé car nous n'utilisons pas de topologies indexées de type "Strip"
    inputAssembly.primitiveRestartEnable = VK_FALSE;

    // =====
    // 3. CONFIGURATION DU VIEWPORT ET DU SCISSOR
    // =====
    // Nous définissons ici une configuration de base, mais nous la rendrons
    // dynamique plus bas pour éviter de recréer la pipeline lors du redimensionnement.
    VkViewport viewport{};
    viewport.x = 0.0f;
    viewport.y = 0.0f;
    viewport.width = 800.0f; // Largeur par défaut
    viewport.height = 600.0f; // Hauteur par défaut

```

```

viewport.minDepth = 0.0f; // Profondeur minimale du tampon
viewport.maxDepth = 1.0f; // Profondeur maximale du tampon

VkRect2D scissor{};
scissor.offset = {0, 0};
scissor.extent = {800, 600}; // Découpe identique à la taille de la fenêtre

VkPipelineViewportStateCreateInfo viewportState{};
viewportState.sType = VK_STRUCTURE_TYPE_PIPELINE_VIEWPORT_STATE_CREATE_INFO;
viewportState.pNext = nullptr;
viewportState.flags = 0;
viewportState.viewportCount = 1;
viewportState.pViewports = &viewport;
viewportState.scissorCount = 1;
viewportState.pScissors = &scissor;

// =====
// 4. CONFIGURATION DE LA RASTERISATION (RASTERIZER)
// =====
VkPipelineRasterizationStateCreateInfo rasterizer{};
rasterizer.sType = VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_STATE_CREATE_INFO;
rasterizer.pNext = nullptr;
rasterizer.flags = 0;

// Si TRUE, les fragments au-delà des plans proche/lointain sont projetés sur ceux-ci
// au lieu d'être éliminés (utile pour les ombres portées). Nécessite une fonctionnalité
rasterizer.depthClampEnable = VK_FALSE;

// Si TRUE, la géométrie ne passe jamais à l'étape de rasterisation (désactive l'affichage)
rasterizer.rasterizerDiscardEnable = VK_FALSE;

// Mode de dessin : Remplissage complet des polygones
rasterizer.polygonMode = VK_POLYGON_MODE_FILL;

// Épaisseur des lignes (uniquement si polygonMode est configuré sur LINE)
rasterizer.lineWidth = 1.0f;

// Élimination des faces cachées : On élimine les faces arrière (Back Faces)
rasterizer.cullMode = VK_CULL_MODE_BACK_BIT;

// Sens de définition de la face avant : Sens horaire (Clockwise)
rasterizer.frontFace = VK_FRONT_FACE_CLOCKWISE;

// Le "Depth Bias" permet d'ajouter un décalage de profondeur (utile pour éviter le Z-fighting)
rasterizer.depthBiasEnable = VK_FALSE;
rasterizer.depthBiasConstantFactor = 0.0f;
rasterizer.depthBiasClamp = 0.0f;
rasterizer.depthBiasSlopeFactor = 0.0f;

```

```

// =====
// 5. CONFIGURATION DU MULTISAMPLING (MSAA)
// =====
VkPipelineMultisampleStateCreateInfo multisampling{};
multisampling.sType = VK_STRUCTURE_TYPE_PIPELINE_MULTISAMPLE_STATE_CREATE_INFO;
multisampling.pNext = nullptr;
multisampling.flags = 0;

// Désactivé pour le moment (1 seul échantillon par pixel)
multisampling.sampleShadingEnable = VK_FALSE;
multisampling.rasterizationSamples = VK_SAMPLE_COUNT_1_BIT;
multisampling.minSampleShading = 1.0f;
multisampling.pSampleMask = nullptr;
multisampling.alphaToCoverageEnable = VK_FALSE;
multisampling.alphaToOneEnable = VK_FALSE;

// =====
// 6. CONFIGURATION DU MÉLANGE DES COULEURS (COLOR BLENDING)
// =====
// Configuration spécifique pour l'attachement de couleur (notre image cible)
VkPipelineColorBlendAttachmentState colorBlendAttachment{};

// Masque définissant quelles composantes de couleur (R, G, B, A) seront écrites
colorBlendAttachment.colorWriteMask = VK_COLOR_COMPONENT_R_BIT |
                                       VK_COLOR_COMPONENT_G_BIT |
                                       VK_COLOR_COMPONENT_B_BIT |
                                       VK_COLOR_COMPONENT_A_BIT;

// Configuration pour le mélange Alpha (transparence classique)
colorBlendAttachment.blendEnable = VK_TRUE;
colorBlendAttachment.srcColorBlendFactor = VK_BLEND_FACTOR_SRC_ALPHA;
colorBlendAttachment.dstColorBlendFactor = VK_BLEND_FACTOR_ONE_MINUS_SRC_ALPHA;
colorBlendAttachment.colorBlendOp = VK_BLEND_OP_ADD;
colorBlendAttachment.srcAlphaBlendFactor = VK_BLEND_FACTOR_ONE;
colorBlendAttachment.dstAlphaBlendFactor = VK_BLEND_FACTOR_ZERO;
colorBlendAttachment.alphaBlendOp = VK_BLEND_OP_ADD;

// Configuration globale du mélange de couleurs
VkPipelineColorBlendStateCreateInfo colorBlending{};
colorBlending.sType = VK_STRUCTURE_TYPE_PIPELINE_COLOR_BLEND_STATE_CREATE_INFO;
colorBlending.pNext = nullptr;
colorBlending.flags = 0;

// Permet d'utiliser des opérations logiques binaires au lieu du mélange mathématique
colorBlending.logicOpEnable = VK_FALSE;
colorBlending.logicOp = VK_LOGIC_OP_COPY;

// Liaison avec notre attachement unique
colorBlending.attachmentCount = 1;

```

```

colorBlending.pAttachments = &colorBlendAttachment;

// Constantes de mélange optionnelles
colorBlending.blendConstants[0] = 0.0f;
colorBlending.blendConstants[1] = 0.0f;
colorBlending.blendConstants[2] = 0.0f;
colorBlending.blendConstants[3] = 0.0f;

// =====
// 7. CONFIGURATION DES ÉTATS DYNAMIQUES (DYNAMIC STATES)
// =====
// Nous déclarons le Viewport et le Scissor comme dynamiques.
// Cela signifie que nous devons obligatoirement les définir via des commandes
// lors de l'enregistrement de notre boucle de rendu (Module 3.1).
std::vector<VkDynamicState> dynamicStates = {
    VK_DYNAMIC_STATE_VIEWPORT,
    VK_DYNAMIC_STATE_SCISSOR
};

VkPipelineDynamicStateCreateInfo dynamicState{};
dynamicState.sType = VK_STRUCTURE_TYPE_PIPELINE_DYNAMIC_STATE_CREATE_INFO;
dynamicState.pNext = nullptr;
dynamicState.flags = 0;
dynamicState.dynamicStateCount = static_cast<uint32_t>(dynamicStates.size());
dynamicState.pDynamicStates = dynamicStates.data();

// =====
// 8. CRÉATION DE L'AGENCEMENT DE LA PIPELINE (PIPELINE LAYOUT)
// =====
VkPipelineLayoutCreateInfo pipelineLayoutInfo{};
pipelineLayoutInfo.sType = VK_STRUCTURE_TYPE_PIPELINE_LAYOUT_CREATE_INFO;
pipelineLayoutInfo.pNext = nullptr;
pipelineLayoutInfo.flags = 0;

// Pas de variables uniformes (descriptors) pour notre premier triangle
pipelineLayoutInfo.setLayoutCount = 0;
pipelineLayoutInfo.pSetLayouts = nullptr;

// Pas de constantes intégrées (push constants)
pipelineLayoutInfo.pushConstantRangeCount = 0;
pipelineLayoutInfo.pPushConstantRanges = nullptr;

// Note : Dans un code de production, 'device' est le VkDevice logique initialisé au Module 3.
VkDevice device;

// Simulation de la création de l'agencement (Layout)
/*
if (vkCreatePipelineLayout(device, &pipelineLayoutInfo, nullptr, &pipelineLayout) != VK_SUCCESS)
    throw std::runtime_error("Échec de la création du Pipeline Layout !");
*/

```

```
}  
*/  
}
```

4. Pièges Fréquents et Bonnes Pratiques

4.1 L'explosion du nombre de pipelines (Pipeline Explosion)

Puisque chaque état est immuable, si votre jeu contient 50 objets avec des configurations de rendu légèrement différentes (ex: certains transparents, d'autres en fil de fer, d'autres avec ou sans test de profondeur), vous devrez créer 50 objets `VkPipeline` distincts. *
Bonne pratique : Utilisez des Pipeline Caches (`VkPipelineCache`). Cet objet permet d'enregistrer sur le disque dur de l'utilisateur les pipelines déjà compilées lors des sessions précédentes, réduisant drastiquement le temps de chargement du jeu.

4.2 L'oubli de configuration des États Dynamiques

Si vous déclarez un état comme dynamique dans `VkPipelineDynamicStateCreateInfo` (comme le Viewport ou le Scissor), vous devez obligatoirement appeler les fonctions de commande correspondantes (`vkCmdSetViewport` et `vkCmdSetScissor`) lors de l'enregistrement de vos commandes de rendu (Module 3.1). Si vous oubliez de le faire, le GPU aura un comportement indéfini et l'application plantera immédiatement.

4.3 Incohérence avec la Render Pass

Le nombre d'attachements de couleur définis dans `VkPipelineColorBlendStateCreateInfo` (le champ `attachmentCount`) doit correspondre exactement au nombre d'attachements de couleur décrits dans la Render Pass (qui sera étudiée dans le Module 2.3). Une seule erreur à ce niveau provoquera une violation d'accès détectée par les couches de validation.

5. Synthèse Pédagogique

Étape de la Pipeline	Rôle Principal	Peut être rendu Dynamique ?	Configuration par défaut (Triangle)
Vertex Input	Décrit le format des données de sommets en mémoire.	Non	Aucun (0 binding, 0 attribute)
Input Assembly	Définit comment lier les sommets (points, lignes, triangles).	Non	<code>VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST</code>
Viewport & Scissor	Gère la zone d'affichage et le rectangle de découpe.	Oui (Recommandé)	Configuré dynamiquement
Rasterizer	Transforme les géométries en fragments (pixels potentiels).	Non	Remplissage complet, élimination face arrière
Multisampling	Gère l'antialiasing (MSAA).	Non	Désactivé (1 échantillon)
Color Blending	Combine la couleur	Non	Mélange Alpha classique activé

Étape de la Pipeline	Rôle Principal	Peut être rendu Dynamique ?	Configuration par défaut (Triangle)
	calculée avec celle déjà présente à l'écran.		

6. Exercice Pratique d'Application

Énoncé

Dans le cadre du développement d'un outil de débogage pour notre moteur de rendu Vulkan, vous devez modifier la configuration de la pipeline présentée dans ce cours pour répondre aux deux exigences suivantes : 1. Les objets doivent être dessinés en fil de fer (wireframe) au lieu d'être pleins. 2. L'élimination des faces cachées (Culling) doit être complètement désactivée afin que nous puissions voir l'intérieur et l'extérieur des géométries.

Indices

- Inspectez attentivement la structure `VkPipelineRasterizationStateCreateInfo` .
- Repérez le membre qui contrôle le mode de dessin des polygones (`polygonMode`).
- Repérez le membre qui contrôle le mode d'élimination des faces (`cullMode`).

Correction de l'Exercice

Pour réaliser cette configuration de débogage, il suffit de modifier deux lignes spécifiques dans la configuration de la structure `VkPipelineRasterizationStateCreateInfo` :

```

VkPipelineRasterizationStateCreateInfo debugRasterizer{};
debugRasterizer.sType = VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_STATE_CREATE_INFO;
debugRasterizer.pNext = nullptr;
debugRasterizer.flags = 0;
debugRasterizer.depthClampEnable = VK_FALSE;
debugRasterizer.rasterizerDiscardEnable = VK_FALSE;

// MODIFICATION 1 : Passage du mode de remplissage (FILL) au mode fil de fer (LINE)
debugRasterizer.polygonMode = VK_POLYGON_MODE_LINE;

// L'épaisseur de la ligne doit être définie. 1.0f est garanti supporté par toutes les cartes
debugRasterizer.lineWidth = 1.0f;

// MODIFICATION 2 : Désactivation complète du Culling (élimination des faces cachées)
debugRasterizer.cullMode = VK_CULL_MODE_NONE;

debugRasterizer.frontFace = VK_FRONT_FACE_CLOCKWISE;
debugRasterizer.depthBiasEnable = VK_FALSE;

```

Explications de la correction : * En changeant `polygonMode` à `VK_POLYGON_MODE_LINE`, le GPU va automatiquement ignorer le remplissage des triangles et dessiner uniquement leurs contours. * En passant `cullMode` à `VK_CULL_MODE_NONE`, le GPU ne jettera plus les triangles orientés vers l'arrière. Cela nous permet d'inspecter la structure géométrique complète de notre modèle 3D sous tous les angles.

Module 2.3 : Passes de rendu (Render Passes) et Framebuffers

Dans les modules précédents, nous avons configuré l'instance Vulkan, sélectionné notre carte graphique (Module 1.1), mis en place la Swapchain pour obtenir des images d'affichage (Module 1.3), compilé nos shaders en SPIR-V (Module 2.1) et configuré les fonctions fixes de notre pipeline graphique (Module 2.2).

Cependant, une question fondamentale demeure : où et comment le pipeline doit-il dessiner ?

Pour y répondre, Vulkan sépare strictement la définition logique des cibles de rendu et les ressources mémoire réelles qui recevront les pixels. C'est le rôle respectif des Passes de rendu (Render Passes) et des Framebuffers. Ce module détaille ces deux concepts indispensables pour franchir l'étape de l'affichage.

1. Introduction Conceptuelle

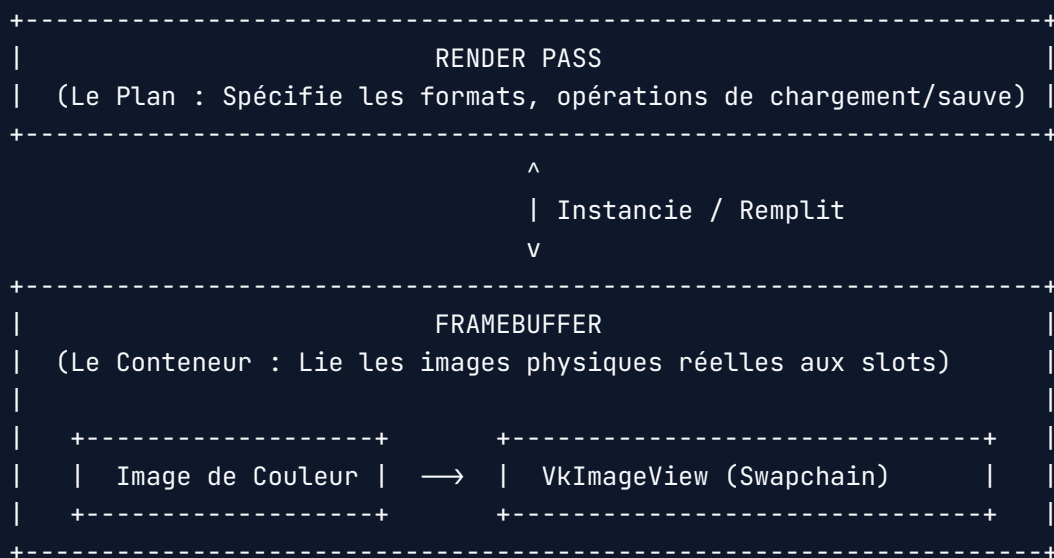
Pour comprendre pourquoi Vulkan introduit ces objets, comparons sa philosophie avec celle d'anciennes API comme OpenGL.

Le "Pourquoi" : L'optimisation matérielle

En OpenGL, vous liez un framebuffer, effacez l'écran, dessinez, puis affichez le résultat. Le pilote graphique devait deviner vos intentions à la volée. Sur les architectures GPU modernes (en particulier les puces mobiles et les GPU de bureau à architecture *Tile-Based Rendering*), la mémoire vidéo globale (VRAM) est lente et énergivore par rapport à la mémoire cache ultra-rapide située directement sur la puce du GPU (SRAM).

Pour maximiser les performances, le GPU divise l'écran en tuiles (*tiles*). Il charge les données d'une tuile dans son cache ultra-rapide, effectue tous les calculs de dessin, puis réécrit le résultat final dans la VRAM.

C'est exactement le rôle de la **Render Pass** : elle fournit au GPU un plan d'action complet et statique de l'opération de rendu.



2. Fondations Théoriques

2.1 La Passe de Rendu (`VkRenderPass`)

Une passe de rendu est composée de trois éléments principaux : 1. Les Attachements (Attachments) : Les descriptions des images de destination (couleur, profondeur, stencil, etc.). 2. Les Sous-passes (Subpasses) : Les étapes successives du rendu qui utilisent ces attachements. 3. Les Dépendances de sous-passes (Subpass Dependencies) : Les règles de synchronisation entre les sous-passes ou avec les opérations extérieures.

A. Les Attachements (`VkAttachmentDescription`)

Pour chaque image utilisée durant la passe, nous devons spécifier : * **Format** : Le format de pixel (doit correspondre exactement au format de la Swapchain, ex: `VK_FORMAT_B8G8R8A8_SRGB`). * **Samples** : Le nombre d'échantillons pour l'anti-crénelage (MSAA). Nous utiliserons `VK_SAMPLE_COUNT_1_BIT` (pas de MSAA) pour notre premier triangle. * **Load Op** : Ce que l'on fait des pixels existants au début de la passe : * `VK_ATTACHMENT_LOAD_OP_CLEAR` : Efface l'image avec une couleur unie (très performant). * `VK_ATTACHMENT_LOAD_OP_LOAD` : Conserve le contenu existant (nécessite une lecture VRAM coûteuse). * `VK_ATTACHMENT_LOAD_OP_DONT_CARE` : Le contenu existant est ignoré/écrasé (le plus rapide, idéal si on redessine sur tout l'écran). * **Store Op** : Ce que l'on fait des pixels générés à la fin de la passe : * `VK_ATTACHMENT_STORE_OP_STORE` : Sauvegarde le résultat en mémoire pour affichage ou lecture ultérieure. * `VK_ATTACHMENT_STORE_OP_DONT_CARE` : Jette le contenu (très utile pour un tampon de profondeur après que le rendu soit fini). * **Layouts d'image (Transitions de Layout)** : Les GPU stockent les pixels différemment selon l'usage (optimisé pour l'écriture, pour la lecture de texture, ou pour l'affichage). Vulkan requiert de spécifier : * `initialLayout` : Le layout dans lequel se trouve l'image avant le début de la passe (ex: `VK_IMAGE_LAYOUT_UNDEFINED` si on s'en fiche et qu'on va l'effacer). * `finalLayout` : Le layout vers lequel Vulkan doit automatiquement transitionner l'image à la fin de la passe (ex: `VK_IMAGE_LAYOUT_PRESENT_SRC_KHR` pour que l'image soit prête à être envoyée à l'écran).

B. Les Sous-passes (`VkSubpassDescription`)

Une seule passe de rendu peut enchaîner plusieurs opérations successives appelées sous-passes. Par exemple, en rendu différé (*Deferred Shading*) : * Sous-passe 1 : Écriture des

positions, normales et couleurs dans des textures intermédiaires (G-Buffer). * Sous-passe 2 : Lecture de ces textures pour calculer l'éclairage final.

Pour notre premier triangle, nous n'aurons besoin que d'une seule sous-passe de type graphique.

C. Les Dépendances de sous-passes (`VkSubpassDependency`)

Le GPU est une machine massivement parallèle qui exécute des tâches de manière asynchrone. Nous devons lui indiquer explicitement quand il a le droit de commencer à écrire sur notre image de la Swapchain.

L'image de la Swapchain est acquise depuis le système de présentation de l'OS. Nous devons nous assurer que le GPU n'essaie pas d'écrire dessus avant que l'OS ait fini de la lire. Cette synchronisation est gérée par une dépendance de sous-passe.

2.2 Le Framebuffer (`VkFramebuffer`)

Le Framebuffer est un objet extrêmement simple. Il s'agit d'un tableau de vues d'images (`VkImageView`) qui viennent s'associer un-à-un aux attachements décrits par la Render Pass.

Si votre Render Pass décrit : * Attachement 0 : Image de Couleur. * Attachement 1 : Image de Profondeur.

Votre Framebuffer devra contenir exactement deux `VkImageView` : la première pointant vers une image de couleur, la seconde vers une image de profondeur.

Règle d'or de la Swapchain : Comme la Swapchain contient plusieurs images (généralement 2 pour le Double Buffering ou 3 pour le Triple Buffering) pour éviter les déchirures d'écran, nous devons créer un Framebuffer distinct pour chaque image de la Swapchain.

3. Implémentation Pratique Pas-à-Pas

Nous allons maintenant écrire le code C++ permettant d'initialiser ces objets. Nous utiliserons l'API C standard de Vulkan (`<vulkan/vulkan.h>`).

3.1 Création de la Passe de Rendu

Voici la fonction complète pour créer notre `VkRenderPass` .

```

#include <vulkan/vulkan.h>
#include <stdexcept>
#include <vector>

VkRenderPass createRenderPass(VkDevice device, VkFormat swapChainImageFormat) {
    VkRenderPass renderPass;

    // 1. Description de l'attachement de couleur (notre écran)
    VkAttachmentDescription colorAttachment{};
    colorAttachment.format = swapChainImageFormat; // Doit correspondre à la swapChainImageFormat
    colorAttachment.samples = VK_SAMPLE_COUNT_1_BIT; // Pas de multisampling (MSAA)
    colorAttachment.loadOp = VK_ATTACHMENT_LOAD_OP_CLEAR; // Effacer l'écran avant de commencer
    colorAttachment.storeOp = VK_ATTACHMENT_STORE_OP_STORE; // Sauvegarder le dessin pour la swapChainImageFormat
    colorAttachment.stencilLoadOp = VK_ATTACHMENT_LOAD_OP_DONT_CARE; // Pas de stencil
    colorAttachment.stencilStoreOp = VK_ATTACHMENT_STORE_OP_DONT_CARE;
    colorAttachment.initialLayout = VK_IMAGE_LAYOUT_UNDEFINED; // Layout d'origine non important
    colorAttachment.finalLayout = VK_IMAGE_LAYOUT_PRESENT_SRC_KHR; // Prêt pour la présentation

    // 2. Référence de l'attachement pour la sous-passe
    // Une sous-passe fait référence à un attachement par son index dans le tableau des descriptions
    VkAttachmentReference colorAttachmentRef{};
    colorAttachmentRef.attachment = 0; // Index 0 (notre unique colorAttachment)
    colorAttachmentRef.layout = VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL; // Layout durant la sous-passe

    // 3. Description de la sous-passe unique
    VkSubpassDescription subpass{};
    subpass.pipelineBindPoint = VK_PIPELINE_BIND_POINT_GRAPHICS; // Sous-passe graphique
    subpass.colorAttachmentCount = 1;
    subpass.pColorAttachments = &colorAttachmentRef; // Pointeur vers notre référence d'attachement

    // 4. Dépendance de sous-passe pour la synchronisation
    // Nous devons attendre que l'image soit disponible avant d'écrire la couleur
    VkSubpassDependency dependency{};
    dependency.srcSubpass = VK_SUBPASS_EXTERNAL; // Opération externe (la présentation de l'image)
    dependency.dstSubpass = 0; // Notre sous-passe (index 0)

    // Étape d'attente : Attendre que l'étape de sortie des couleurs soit atteinte
    dependency.srcStageMask = VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT;
    dependency.srcAccessMask = 0;

    // Étape de blocage : Bloquer l'écriture des couleurs tant que l'image n'est pas prête
    dependency.dstStageMask = VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT;
    dependency.dstAccessMask = VK_ACCESS_COLOR_ATTACHMENT_WRITE_BIT;

    // 5. Création de la Render Pass
    VkRenderPassCreateInfo renderPassInfo{};
    renderPassInfo.sType = VK_STRUCTURE_TYPE_RENDER_PASS_CREATE_INFO;
    renderPassInfo.attachmentCount = 1;

```

```

renderPassInfo.pAttachments = &colorAttachment;
renderPassInfo.subpassCount = 1;
renderPassInfo.pSubpasses = &subpass;
renderPassInfo.dependencyCount = 1;
renderPassInfo.pDependencies = &dependency;

if (vkCreateRenderPass(device, &renderPassInfo, nullptr, &renderPass) ≠ VK_SUCCESS) {
    throw std::runtime_error("Erreur : Impossible de creer la passe de rendu !");
}

return renderPass;
}

```

3.2 Création des Framebuffers

Maintenant que la Render Pass est définie, nous devons créer un Framebuffer pour chaque image de notre Swapchain.

Supposons que nous ayons stocké nos `VkImageView` de la Swapchain dans un `std::vector<VkImageView> swapChainImageViews` et que nous connaissions les dimensions de la Swapchain via un `VkExtent2D swapChainExtent`.

```

std::vector<VkFramebuffer> createFramebuffers(
    VkDevice device,
    VkRenderPass renderPass,
    const std::vector<VkImageView>& swapChainImageViews,
    VkExtent2D swapChainExtent)
{
    std::vector<VkFramebuffer> framebuffers(swapChainImageViews.size());

    // Boucle sur chaque image view de la swapchain
    for (size_t i = 0; i < swapChainImageViews.size(); i++) {
        // Liste des attachements liés au framebuffer.
        // L'ordre doit correspondre EXACTEMENT aux attachements de la Render Pass.
        VkImageView attachments[] = {
            swapChainImageViews[i]
        };

        VkFramebufferCreateInfo framebufferInfo{};
        framebufferInfo.sType = VK_STRUCTURE_TYPE_FRAMEBUFFER_CREATE_INFO;
        framebufferInfo.renderPass = renderPass;           // La Render Pass compatible
        framebufferInfo.attachmentCount = 1;               // Nombre d'attachements
        framebufferInfo.pAttachments = attachments;        // Pointeur vers les images ph
        framebufferInfo.width = swapChainExtent.width;     // Largeur de la swapchain
        framebufferInfo.height = swapChainExtent.height;   // Hauteur de la swapchain
        framebufferInfo.layers = 1;                        // Nombre de couches (1 pour l

        if (vkCreateFramebuffer(device, &framebufferInfo, nullptr, &framebuffers[i]) != VK_SUCCESS)
            throw std::runtime_error("Erreur : Impossible de creer le framebuffer " + std::to_string(i));
    }
}

return framebuffers;
}

```

3.3 Libération des Ressources (Cleanup)

En C++, la gestion de la mémoire dans Vulkan est entièrement manuelle. Tout objet créé avec une fonction `vkCreate...` doit être détruit avec son équivalent `vkDestroy...` lorsque l'application se ferme.

L'ordre de destruction est crucial : les Framebuffers dépendent de la Render Pass et des Image Views. Ils doivent donc être détruits en premier.

```

void cleanupRenderingResources(
    VkDevice device,
    VkRenderPass renderPass,
    std::vector<VkFramebuffer>& framebuffers)
{
    // 1. D  truire d'abord tous les framebuffers
    for (auto framebuffer : framebuffers) {
        vkDestroyFramebuffer(device, framebuffer, nullptr);
    }
    framebuffers.clear(); // Vider le tableau

    // 2. D  truire ensuite la Render Pass
    vkDestroyRenderPass(device, renderPass, nullptr);
}

```

4. Pi  ges Fr  quents et Bonnes Pratiques

1. Incompatibilit   entre Render Pass et Framebuffer

- **Le Pi  ge** : Essayer d'utiliser un Framebuffer cr    avec une Render Pass "A" dans une Render Pass "B".
- **La R  gle** : Un Framebuffer n'est utilisable qu'avec la Render Pass qui a servi    sa cr  ation, ou avec une Render Pass compatible. Deux passes de rendu sont compatibles si elles poss  dent le m  me nombre d'attachements avec les m  mes formats et le m  me nombre d'  chantillons (MSAA).

2. Incoh  rence des dimensions

- **Le Pi  ge** : Sp  cifier des dimensions de Framebuffer sup  rieures    celles de l'image de la Swapchain. Cela provoque un crash imm  diat du pilote ou un comportement ind  fini.
- **La R  gle** : Assurez-vous que `framebufferInfo.width` et `framebufferInfo.height` correspondent exactement aux dimensions actuelles de vos images de Swapchain (`swapChainExtent`).

3. Mauvais ordre des attachements

- Le Piège : Déclarer la couleur en attachement 0 et la profondeur en attachement 1 dans la Render Pass, mais passer la vue de profondeur en premier dans le tableau `attachments[]` du Framebuffer.
- La Règle : L'indexation est stricte. L'ordre physique des `VkImageView` dans le tableau du Framebuffer doit correspondre au pixel près à l'ordre sémantique des `VkAttachmentDescription` de la Render Pass.

5. Synthèse Pédagogique

Concept	Rôle Principal	Durée de vie typique	Lien avec les autres objets
<code>VkRenderPass</code>	Décrit la structure logique du rendu (Quels formats ? Quelles opérations ?).	Créée une fois au démarrage de l'application.	Référencée par la Pipeline Graphique et le Framebuffer.
<code>VkFramebuffer</code>	Lie les ressources mémoire réelles (<code>VkImageView</code>) à la structure de la Render Pass.	Recréée à chaque fois que la fenêtre est redimensionnée (car la Swapchain change).	Contient les Image Views de la Swapchain.

Flux de données simplifié :

```
[ Pipeline Graphique ]
|
v (S'exécute dans)
[ Render Pass (Plan) ] <--- (Instanciée par) --- [ Framebuffer (Images réelles)
|
v
[ Écran de l'utilisateur ]
```

6. Exercice Pratique d'Application

Énoncé : Ajouter un tampon de profondeur (Depth Buffer)

Pour préparer notre moteur à afficher de la 3D, nous devons ajouter un tampon de profondeur (*Depth Buffer* ou *Z-Buffer*). Le tampon de profondeur permet d'éviter qu'un objet situé derrière un autre ne soit dessiné par-dessus.

Votre objectif est de modifier la fonction de création de la Render Pass pour intégrer un deuxième attachement : un attachement de profondeur.

Spécifications requises :

1. Format de profondeur : `VK_FORMAT_D32_SFLOAT` (Float 32-bits pour la précision).
 2. Load Op : `VK_ATTACHMENT_LOAD_OP_CLEAR` (Nous voulons réinitialiser la profondeur à chaque image).
 3. Store Op : `VK_ATTACHMENT_STORE_OP_DONT_CARE` (Une fois l'image dessinée, nous n'avons plus besoin de conserver la carte des profondeurs en mémoire).
 4. Layout de transition :
 5. `initialLayout` : `VK_IMAGE_LAYOUT_UNDEFINED` .
 6. `finalLayout` : `VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL` .
-

Indices pour vous aider

1. Vous devez déclarer un deuxième `VkAttachmentDescription` pour la profondeur.
 2. Vous devez créer un `VkAttachmentReference` pour cet attachement de profondeur. Son `layout` doit être `VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL` .
 3. Dans `VkSubpassDescription` , vous devez lier cette référence au pointeur `pDepthStencilAttachment` .
 4. N'oubliez pas de passer la taille du tableau d'attachements à `2` dans `VkRenderPassCreateInfo` .
-

Correction complète de l'exercice

Voici le code C++ corrigé permettant de créer une Render Pass gérant à la fois la couleur et la profondeur.


```

#include <vulkan/vulkan.h>
#include <stdexcept>
#include <array>

VkRenderPass createRenderPassWithDepth(VkDevice device, VkFormat swapChainImageFormat) {
    VkRenderPass renderPass;

    // 1. Attachement Couleur
    VkAttachmentDescription colorAttachment{};
    colorAttachment.format = swapChainImageFormat;
    colorAttachment.samples = VK_SAMPLE_COUNT_1_BIT;
    colorAttachment.loadOp = VK_ATTACHMENT_LOAD_OP_CLEAR;
    colorAttachment.storeOp = VK_ATTACHMENT_STORE_OP_STORE;
    colorAttachment.stencilLoadOp = VK_ATTACHMENT_LOAD_OP_DONT_CARE;
    colorAttachment.stencilStoreOp = VK_ATTACHMENT_STORE_OP_DONT_CARE;
    colorAttachment.initialLayout = VK_IMAGE_LAYOUT_UNDEFINED;
    colorAttachment.finalLayout = VK_IMAGE_LAYOUT_PRESENT_SRC_KHR;

    VkAttachmentReference colorAttachmentRef{};
    colorAttachmentRef.attachment = 0; // Index 0 dans le tableau des attachements
    colorAttachmentRef.layout = VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL;

    // 2. NOUVEAU : Attachement Profondeur (Z-Buffer)
    VkAttachmentDescription depthAttachment{};
    depthAttachment.format = VK_FORMAT_D32_SFLOAT; // Format haute précision pour la profondeur
    depthAttachment.samples = VK_SAMPLE_COUNT_1_BIT;
    depthAttachment.loadOp = VK_ATTACHMENT_LOAD_OP_CLEAR; // Effacer à chaque frame
    depthAttachment.storeOp = VK_ATTACHMENT_STORE_OP_DONT_CARE; // Pas besoin de le lire après
    depthAttachment.stencilLoadOp = VK_ATTACHMENT_LOAD_OP_DONT_CARE;
    depthAttachment.stencilStoreOp = VK_ATTACHMENT_STORE_OP_DONT_CARE;
    depthAttachment.initialLayout = VK_IMAGE_LAYOUT_UNDEFINED;
    depthAttachment.finalLayout = VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL;

    VkAttachmentReference depthAttachmentRef{};
    depthAttachmentRef.attachment = 1; // Index 1 dans le tableau des attachements
    depthAttachmentRef.layout = VK_IMAGE_LAYOUT_DEPTH_STENCIL_ATTACHMENT_OPTIMAL;

    // 3. Sous-passe associant Couleur ET Profondeur
    VkSubpassDescription subpass{};
    subpass.pipelineBindPoint = VK_PIPELINE_BIND_POINT_GRAPHICS;
    subpass.colorAttachmentCount = 1;
    subpass.pColorAttachments = &colorAttachmentRef;
    subpass.pDepthStencilAttachment = &depthAttachmentRef; // Liaison de l'attachement de p

    // 4. Synchronisation
    VkSubpassDependency dependency{};
    dependency.srcSubpass = VK_SUBPASS_EXTERNAL;
    dependency.dstSubpass = 0;

```

```

dependency.srcStageMask = VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT | VK_PIPELINE_STAGE_DEPTH_ATTACHMENT_UPDATE_BIT;
dependency.srcAccessMask = 0;
dependency.dstStageMask = VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT | VK_PIPELINE_STAGE_DEPTH_ATTACHMENT_UPDATE_BIT;
dependency.dstAccessMask = VK_ACCESS_COLOR_ATTACHMENT_WRITE_BIT | VK_ACCESS_DEPTH_ATTACHMENT_WRITE_BIT;

// 5. Regroupement des attachements dans un tableau
std::array<VkAttachmentDescription, 2> attachments = { colorAttachment, depthAttachment };

VkRenderPassCreateInfo renderPassInfo{};
renderPassInfo.sType = VK_STRUCTURE_TYPE_RENDER_PASS_CREATE_INFO;
renderPassInfo.attachmentCount = static_cast<uint32_t>(attachments.size());
renderPassInfo.pAttachments = attachments.data();
renderPassInfo.subpassCount = 1;
renderPassInfo.pSubpasses = &subpass;
renderPassInfo.dependencyCount = 1;
renderPassInfo.pDependencies = &dependency;

if (vkCreateRenderPass(device, &renderPassInfo, nullptr, &renderPass) != VK_SUCCESS) {
    throw std::runtime_error("Erreur : Impossible de creer la passe de rendu avec profondeur");
}

return renderPass;
}

```

Note d'architecture pour le futur : Si vous utilisez cette Render Pass avec profondeur, n'oubliez pas que vos Framebuffers devront eux aussi être modifiés pour recevoir un tableau de deux `VkImageView` (l'image de la Swapchain pour la couleur, et une image de texture de profondeur créée en VRAM). Nous aborderons la création des textures et de la mémoire GPU dans les modules de gestion des ressources.

Enregistrement des Commandes et Boucle de Rendu

Module 3.1 : Tampons de commandes (Command Pools & Buffers)

Dans les API graphiques de précédente génération (comme OpenGL), l'envoi d'instructions au processeur graphique (GPU) s'effectuait de manière synchrone et implicite. Chaque appel de fonction (par exemple `glDrawArrays`) traduisait immédiatement l'instruction pour le GPU et tentait de l'exécuter. Cette approche créait un goulot d'étranglement majeur sur le processeur (CPU), incapable d'alimenter le GPU assez rapidement, et interdisait pratiquement tout parallélisme CPU.

Vulkan résout ce problème en introduisant un paradigme d'enregistrement asynchrone. Les instructions de rendu ne sont pas exécutées immédiatement : elles sont enregistrées dans des structures appelées Tampons de commandes (Command Buffers), qui sont ensuite soumises en lot (batch) au GPU.

Ce module détaille le fonctionnement, la création et la gestion de ces tampons de commandes et de leurs conteneurs de mémoire, les Command Pools.

1. Introduction Conceptuelle

Le "Pourquoi" : Découplage CPU-GPU et Multithreading

Pour comprendre l'intérêt des tampons de commandes, il faut analyser le flux de travail d'une application haute performance. Le CPU prépare la scène (calculs de physique, d'intelligence artificielle, visibilité), tandis que le GPU dessine les pixels.

Si le CPU doit attendre que le GPU termine chaque dessin avant d'envoyer le suivant, l'un des deux processeurs reste constamment inactif.

```
Modèle OpenGL (Synchrone / Monothread) :
```

```
CPU : [Calculs] → [Appel Draw] -----> [Attente] → [Calculs]...
```

```
GPU :                                     \→ [Exécution]
```

```
Modèle Vulkan (Asynchrone / Multithread) :
```

```
CPU Thread 1 : [Enregistre Commandes A] \
```

```
CPU Thread 2 : [Enregistre Commandes B] → [Soumission unique] → [Calculs Frame N+1]...
```

```
GPU :                                     \→ [Exécution A + B]
```

Vulkan impose une séparation stricte : 1. **Génération (CPU)** : On écrit la liste des courses (les commandes de rendu, de transition de textures, de liaisons de shaders). Cette étape peut être répartie sur plusieurs cœurs CPU en parallèle. 2. **Soumission (CPU vers GPU)** : On envoie la liste complète à une file d'attente du GPU (Queue). 3. **Exécution (GPU)** : Le GPU traite la liste de manière autonome et ultra-rapide au niveau matériel.

L'analogie du Chantier de Construction

Imaginez un architecte (le CPU) et un maçon (le GPU). * **Approche ancienne** : L'architecte reste sur le chantier et dit au maçon : "Pose une brique. Maintenant pose du mortier. Maintenant pose une autre brique." Si l'architecte doit réfléchir ou répondre au téléphone, le maçon attend sans rien faire. * **Approche Vulkan** : L'architecte rédige un plan d'action ultra-détaillé (le *Command Buffer*) dans son bureau. Il donne ce plan au maçon. Le maçon peut travailler de manière totalement autonome à son rythme maximal, tandis que l'architecte prépare déjà les plans du lendemain.

2. Fondations Théoriques

Pour manipuler les commandes en Vulkan, nous devons comprendre deux objets fondamentaux de l'API : `VkCommandPool` et `VkCommandBuffer`.

2.1 Le Command Pool (`VkCommandPool`)

Les tampons de commandes stockent des listes d'instructions. Ces listes ont besoin de mémoire pour être stockées. Allouer de la mémoire directement depuis le système pour chaque commande serait catastrophique pour les performances.

Le Command Pool (ou pool de commandes) est un allocateur de mémoire opaque. Il réserve un grand bloc de mémoire auprès du système et gère l'allocation interne pour les tampons de commandes qui lui sont rattachés.

Règles cruciales de Thread-Safety (Sécurité Multi-thread)

En Vulkan, pour maximiser les performances, l'API ne contient pratiquement aucun mécanisme de verrouillage interne (mutex). C'est au développeur de s'assurer que deux threads ne modifient pas le même objet en même temps.

- Règle d'or : Un `VkCommandPool` et tous les `VkCommandBuffer` qui en découlent ne sont pas thread-safe.
- Conséquence pratique : Si vous voulez enregistrer des commandes en parallèle sur 4 threads CPU différents, vous devez créer un `VkCommandPool` par thread.

```
[Thread CPU 1] —> [VkCommandPool A] —> [VkCommandBuffer 1]
                                     ↳ [VkCommandBuffer 2]

[Thread CPU 2] —> [VkCommandPool B] —> [VkCommandBuffer 3]
```

Association à une famille de files d'attente (Queue Family)

Lors de la création d'un Command Pool, vous devez spécifier un index de famille de files d'attente (étudié dans le *Module 1.2*). Les tampons de commandes alloués depuis ce pool ne pourront être soumis qu'à des files d'attente (Queues) de cette même famille. Par exemple, un pool créé pour la famille "Graphique" ne peut pas générer de commandes destinées à une queue purement "Compute" (Calcul).

2.2 Le Tampon de Commandes (`VkCommandBuffer`)

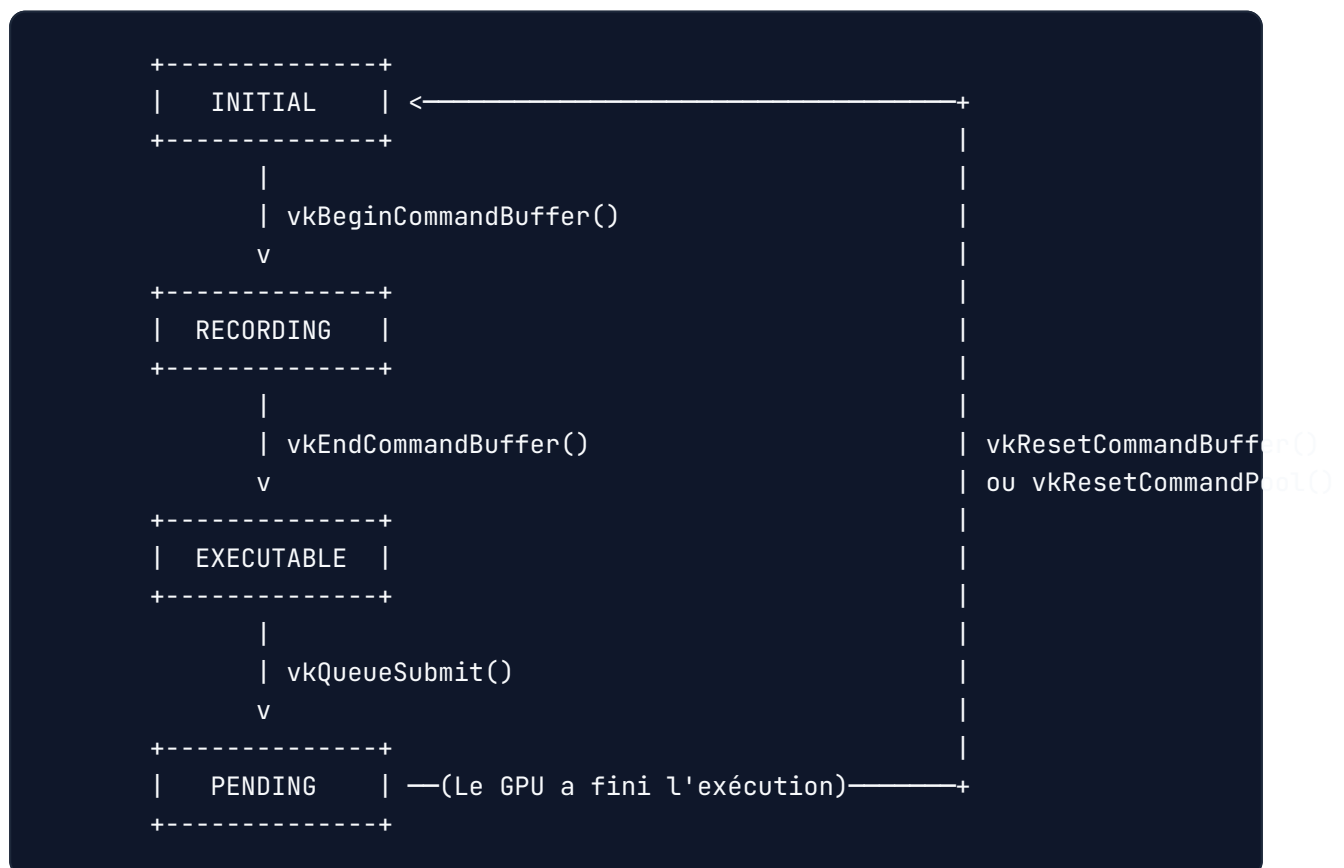
C'est l'objet dans lequel nous allons enregistrer nos commandes de rendu. Il existe deux types (niveaux) de tampons de commandes :

1. Primary Command Buffers (Tampons Primaires) :
2. Peuvent être soumis directement aux files d'attente du GPU (`vkQueueSubmit`).
3. Peuvent appeler des tampons secondaires.

4. Secondary Command Buffers (Tampons Secondaires) :
5. Ne peuvent pas être soumis directement au GPU.
6. Doivent être enregistrés ("insérés") dans un tampon primaire.
7. *Utilité* : Permettent de paralléliser finement l'enregistrement d'une même passe de rendu sur plusieurs threads CPU.

2.3 Cycle de vie d'un Tampon de Commandes

Un tampon de commandes traverse plusieurs états au cours de sa vie. Comprendre ces états est indispensable pour éviter les plantages (crashes) de l'application.



1. **Initial** : Le tampon vient d'être alloué ou réinitialisé. Il est vide.
2. **Recording (Enregistrement)** : La fonction `vkBeginCommandBuffer` a été appelée. On peut y inscrire des instructions (liaison de pipeline, dessin, etc.).
3. **Executable** : La fonction `vkEndCommandBuffer` a été appelée. L'enregistrement est clos. Le tampon est prêt à être envoyé au GPU.

4. Pending (En attente) : Le tampon a été soumis au GPU via `vkQueueSubmit` . Interdiction absolue de modifier, réenregistrer ou détruire ce tampon tant que le GPU n'a pas fini de l'exécuter !
-

3. Implémentation Pratique Pas-à-Pas

Nous allons maintenant écrire le code C++ permettant d'initialiser un pool de commandes, d'allouer un tampon de commandes, et d'y enregistrer nos premières instructions de rendu.

Étape 1 : Création du Command Pool

Pour créer un `VkCommandPool` , nous devons remplir une structure de type `VkCommandPoolCreateInfo` .


```

#include <vulkan/vulkan.h>
#include <iostream>
#include <stdexcept>

// Variables globales ou membres d'une classe (pour le contexte)
VkDevice device; // Initialisé dans le Module 1.1
uint32_t graphicsQueueFamilyIndex; // Identifié dans le Module 1.2
VkCommandPool commandPool;

void createCommandPool() {
    VkCommandPoolCreateInfo poolInfo{};
    poolInfo.sType = VK_STRUCTURE_TYPE_COMMAND_POOL_CREATE_INFO;
    poolInfo.pNext = nullptr;

    // Drapeau (Flag) de configuration du Pool
    // VK_COMMAND_POOL_CREATE_RESET_COMMAND_BUFFER_BIT permet de réinitialiser
    // individuellement les tampons de commandes alloués depuis ce pool.
    poolInfo.flags = VK_COMMAND_POOL_CREATE_RESET_COMMAND_BUFFER_BIT;

    // Liaison avec la famille de files d'attente graphique
    poolInfo.queueFamilyIndex = graphicsQueueFamilyIndex;

    if (vkCreateCommandPool(device, &poolInfo, nullptr, &commandPool) != VK_SUCCESS) {
        throw std::runtime_error("Echec de la creation du Command Pool !");
    }

    std::cout << "Command Pool cree avec succes." << std::endl;
}

```

Explication des champs importants : * `sType` : Spécifie le type de structure pour l'API Vulkan (indispensable pour le polymorphisme C de l'API). * `flags` : * `VK_COMMAND_POOL_CREATE_RESET_COMMAND_BUFFER_BIT` : Si ce flag est omis, vous ne pourrez pas réinitialiser individuellement un tampon de commandes via `vkResetCommandBuffer`. Vous devriez réinitialiser tout le pool d'un coup. * `VK_COMMAND_POOL_CREATE_TRANSIENT_BIT` : Indique à Vulkan que les tampons alloués seront détruits ou réenregistrés très souvent (optimise la stratégie d'allocation mémoire interne).

Étape 2 : Allocation du Tampon de Commandes

Contrairement à d'autres objets Vulkan, les tampons de commandes ne sont pas "créés" via une fonction `vkCreate...` mais alloués via `vkAllocateCommandBuffers` car leur mémoire provient directement du pool.

```

VkCommandBuffer commandBuffer;

void allocateCommandBuffer() {
    VkCommandBufferAllocateInfo allocInfo{};
    allocInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_ALLOCATE_INFO;
    allocInfo.pNext = nullptr;

    // Le pool qui fournira la mémoire
    allocInfo.commandPool = commandPool;

    // Niveau du tampon (Primaire ou Secondaire)
    allocInfo.level = VK_COMMAND_BUFFER_LEVEL_PRIMARY;

    // Nombre de tampons à allouer d'un coup
    allocInfo.commandBufferCount = 1;

    if (vkAllocateCommandBuffers(device, &allocInfo, &commandBuffer) != VK_SUCCESS) {
        throw std::runtime_error("Echec de l'allocation du Command Buffer !");
    }

    std::cout << "Command Buffer alloue avec succes." << std::endl;
}

```

Étape 3 : Enregistrement des Commandes de Rendu

Nous allons maintenant enregistrer les instructions pour dessiner notre premier triangle. Pour cela, nous avons besoin d'une *Render Pass* (Passe de rendu) et d'un *Framebuffer* (Tampon de trame) créés dans le *Module 2.3*, ainsi que de la *Pipeline Graphique* créée dans le *Module 2.2*.

```

// Variables supposées existantes et initialisées dans les modules précédents :
VkRenderPass renderPass;
VkFramebuffer framebuffer;
VkPipeline graphicsPipeline;
VkExtent2D swapChainExtent; // Taille de la fenêtre (ex: 800x600)

void recordCommandBuffer(VkCommandBuffer cmdBuffer, VkFramebuffer targetFramebuffer) {
    // 1. Début de l'enregistrement
    VkCommandBufferBeginInfo beginInfo{};
    beginInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO;
    beginInfo.pNext = nullptr;

    // Optionnel : VK_COMMAND_BUFFER_USAGE_SIMULTANEOUS_USE_BIT permet de soumettre
    // le tampon alors qu'il est déjà en cours d'exécution sur le GPU.
    // Nous laissons à 0 pour une utilisation standard et performante.
    beginInfo.flags = 0;
    beginInfo.pInheritanceInfo = nullptr; // Uniquement pour les tampons secondaires

    if (vkBeginCommandBuffer(cmdBuffer, &beginInfo) != VK_SUCCESS) {
        throw std::runtime_error("Echec du debut d'enregistrement du Command Buffer !");
    }

    // 2. Configuration de la Passe de Rendu (Render Pass)
    VkRenderPassBeginInfo renderPassInfo{};
    renderPassInfo.sType = VK_STRUCTURE_TYPE_RENDER_PASS_BEGIN_INFO;
    renderPassInfo.renderPass = renderPass;
    renderPassInfo.framebuffer = targetFramebuffer;

    // Zone de rendu (toute la fenêtre)
    renderPassInfo.renderArea.offset = {0, 0};
    renderPassInfo.renderArea.extent = swapChainExtent;

    // Définition de la couleur de fond (Clear Color) : Noir opaque
    VkClearValue clearColor = {{{0.0f, 0.0f, 0.0f, 1.0f}}};
    renderPassInfo.clearValueCount = 1;
    renderPassInfo.pClearValues = &clearColor;

    // 3. Commencer la passe de rendu
    // Les commandes de dessin doivent obligatoirement se situer à l'intérieur d'une passe
    // VK_SUBPASS_CONTENTS_INLINE indique que nous enregistrons les commandes directement
    // dans le tampon primaire (pas de tampons secondaires).
    vkCmdBeginRenderPass(cmdBuffer, &renderPassInfo, VK_SUBPASS_CONTENTS_INLINE)

    // 4. Lier la Pipeline Graphique (Shaders, états fixes, etc.)
    // VK_PIPELINE_BIND_POINT_GRAPHICS indique qu'il s'agit d'opérations de rendu 3D/2D.
    vkCmdBindPipeline(cmdBuffer, VK_PIPELINE_BIND_POINT_GRAPHICS, graphicsPipeline);

    // 5. Commande de Dessin (Draw)

```

```

// Paramètres :
// - cmdBuffer : Le tampon dans lequel enregistrer
// - vertexCount : 3 (pour dessiner les 3 sommets d'un unique triangle)
// - instanceCount : 1 (pas d'instanciation ici)
// - firstVertex : 0 (on commence au premier sommet)
// - firstInstance : 0 (identifiant de la première instance)
vkCmdDraw(cmdBuffer, 3, 1, 0, 0);

// 6. Terminer la passe de rendu
vkCmdEndRenderPass(cmdBuffer);

// 7. Fin de l'enregistrement
if (vkEndCommandBuffer(cmdBuffer) ≠ VK_SUCCESS) {
    throw std::runtime_error("Echec de la cloture du Command Buffer !");
}
}

```

Note de nomenclature importante : Toutes les fonctions Vulkan qui enregistrent des commandes dans un tampon commencent par le préfixe `vkCmd` (ex: `vkCmdBeginRenderPass`, `vkCmdBindPipeline`, `vkCmdDraw`). Ces fonctions ne renvoient jamais de code d'erreur (`VkResult`) car elles ne font qu'écrire dans la mémoire locale du tampon sans exécuter de logique immédiate sur le GPU.

Étape 4 : Libération des Ressources

La mémoire en Vulkan doit être gérée rigoureusement. Lorsque l'application se ferme, nous devons détruire le pool de commandes. La destruction du pool détruit automatiquement tous les tampons de commandes qui y ont été alloués.

```

void cleanup() {
    // Pas besoin de détruire individuellement le commandBuffer si on détruit le pool.
    if (commandPool ≠ VK_NULL_HANDLE) {
        vkDestroyCommandPool(device, commandPool, nullptr);
    }
}

```

4. Pièges Fréquents et Bonnes Pratiques

Piège 1 : Modifier ou détruire un tampon en cours d'utilisation par le GPU (État Pending)

C'est l'erreur numéro un des débutants en Vulkan. Le CPU envoie le tampon au GPU via `vkQueueSubmit` et passe immédiatement à la ligne de code suivante. Si vous tentez de réenregistrer ce tampon pour la frame suivante alors que le GPU est encore en train de lire ses instructions, vous provoquerez un comportement indéterminé : * Artefacts visuels clignotants. * Plantage complet du pilote graphique (TDR - Timeout Detection and Recovery). * Crash de l'application.

- **Solution** : Utiliser des barrières de synchronisation CPU-GPU appelées Fences (étudiées en détail dans le *Module 3.2*) pour forcer le CPU à attendre que le GPU ait terminé l'exécution du tampon avant de le réutiliser.

Piège 2 : Allouer et libérer des tampons de commandes à chaque frame

L'allocation de mémoire est une opération extrêmement lourde. Ne faites jamais de `vkAllocateCommandBuffers` et `vkFreeCommandBuffers` dans votre boucle de rendu principale.

- **Solution** : Allouez vos tampons une fois pour toutes lors de l'initialisation (généralement un tampon par image de la Swapchain), et réenregistrez-les simplement à chaque frame en écrasant le contenu précédent grâce au flag `VK_COMMAND_POOL_CREATE_RESET_COMMAND_BUFFER_BIT`.

Piège 3 : Utiliser un seul Command Pool pour toute l'application multi-threadée

Si vous tentez d'enregistrer des commandes depuis des threads CPU différents dans des tampons issus du même pool sans synchronisation manuelle lourde (mutex), votre application plantera de manière aléatoire.

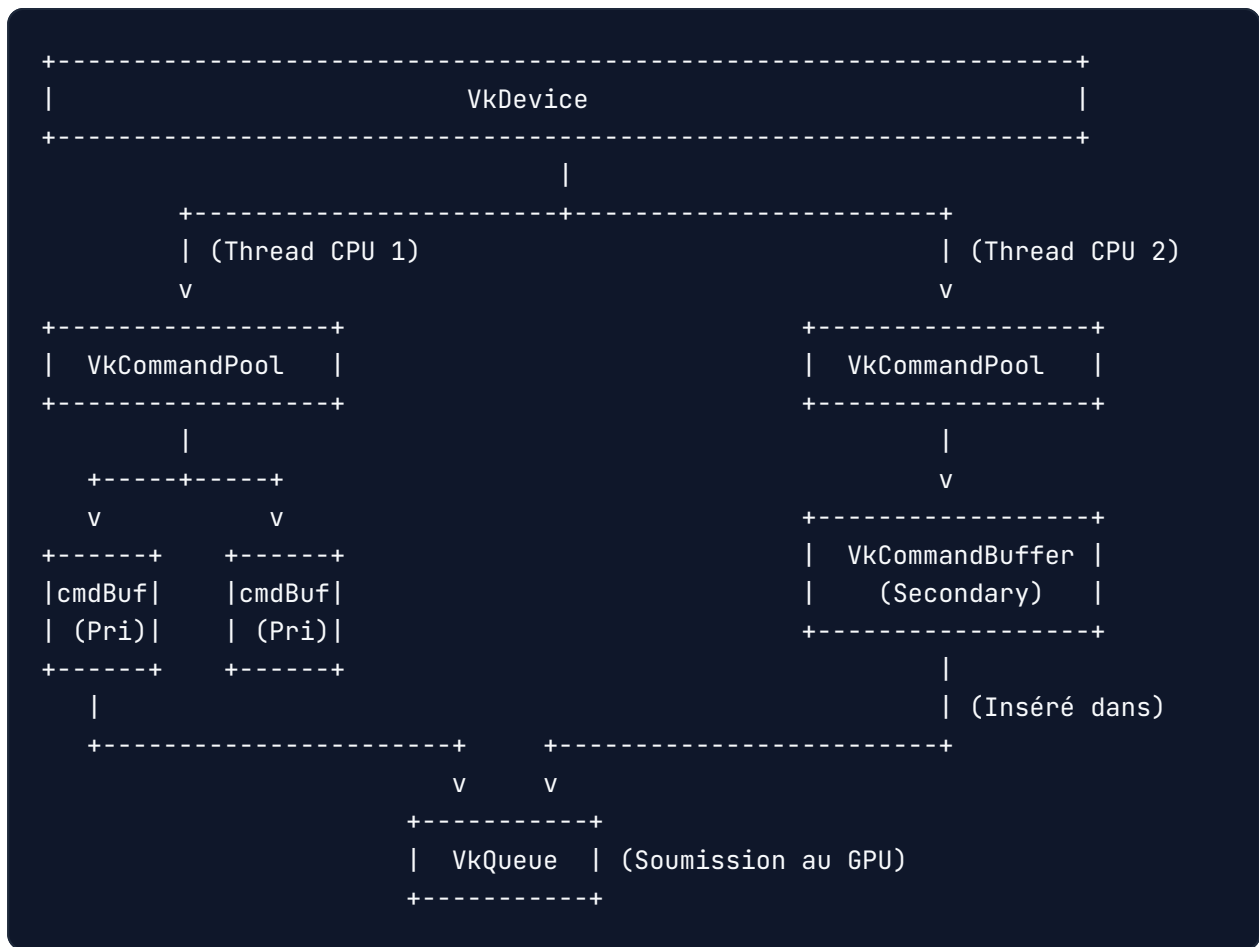
- **Solution** : Créez un `VkCommandPool` distinct pour chaque thread de votre application.
-

5. Synthèse Pédagogique

Comparaison des types de Tampons de Commandes

Caractéristique	Tampon Primaire (Primary)	Tampon Secondaire (Secondary)
Soumission directe au GPU	Oui (via <code>vkQueueSubmit</code>)	Non
Peut appeler d'autres tampons	Oui (peut exécuter des secondaires)	Non
Enregistrement en parallèle	Oui	Oui (idéal pour diviser une passe complexe)
Utilisation typique	Boucle principale, gestion globale de la frame	Rendu de sous-parties de scènes complexes (ex: ombres)

Schéma d'architecture des ressources de commandes



6. Exercice Pratique d'Application

Énoncé

Votre objectif est d'écrire une fonction C++ complète nommée `recordClearScreenCommand` qui enregistre des instructions dans un tampon de commandes existant.

Ce tampon devra :

1. Démarrer l'enregistrement.
2. Commencer une passe de rendu.
3. Configurer l'écran pour qu'il soit effacé (Cleared) avec une couleur spécifique : **Bleu Horizon** (Rouge = `0.2f` , Vert = `0.4f` , Bleu = `0.8f` , Alpha = `1.0f`).
4. Terminer la passe de rendu (sans dessiner d'objets pour l'instant).
5. Clôturer l'enregistrement du tampon.

Indices

- La couleur de fond se configure via l'union `VkClearColorValue`. Le format des couleurs est généralement de type float `RGBA`.
 - N'oubliez pas d'initialiser correctement tous les membres des structures de création (`sType`, `pNext`, etc.) pour éviter les comportements indéfinis.
 - La fonction à utiliser pour démarrer l'enregistrement est `vkBeginCommandBuffer`.
 - La fonction pour démarrer la passe de rendu est `vkCmdBeginRenderPass`.
-

Correction de l'Exercice

Voici le code C++ complet et commenté répondant à l'exercice :


```

#include <vulkan/vulkan.h>
#include <stdexcept>

/**
 * Enregistre les commandes pour effacer l'ecran avec un Bleu Horizon.
 *
 * @param cmdBuffer Le tampon de commandes dans lequel enregistrer.
 * @param renderPass La passe de rendu definissant le format de sortie.
 * @param framebuffer Le framebuffer cible lie a l'image de la swapchain.
 * @param extent Les dimensions de la zone de rendu.
 */
void recordClearScreenCommand(
    VkCommandBuffer cmdBuffer,
    VkRenderPass renderPass,
    VkFramebuffer framebuffer,
    VkExtent2D extent
) {
    // 1. Preparation des informations de debut d'enregistrement
    VkCommandBufferBeginInfo beginInfo{};
    beginInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO;
    beginInfo.pNext = nullptr;
    beginInfo.flags = VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT; // Optimisation : soumis
    beginInfo.pInheritanceInfo = nullptr;

    // Debut de l'enregistrement
    if (vkBeginCommandBuffer(cmdBuffer, &beginInfo) != VK_SUCCESS) {
        throw std::runtime_error("Erreur : Impossible de commencer l'enregistrement du tampon");
    }

    // 2. Configuration de la couleur d'effacement (Bleu Horizon)
    VkClearColorValue clearColor{};
    clearColor.color.float32[0] = 0.2f; // R
    clearColor.color.float32[1] = 0.4f; // G
    clearColor.color.float32[2] = 0.8f; // B
    clearColor.color.float32[3] = 1.0f; // A

    // 3. Configuration du debut de la passe de rendu
    VkRenderPassBeginInfo renderPassInfo{};
    renderPassInfo.sType = VK_STRUCTURE_TYPE_RENDER_PASS_BEGIN_INFO;
    renderPassInfo.pNext = nullptr;
    renderPassInfo.renderPass = renderPass;
    renderPassInfo.framebuffer = framebuffer;

    // Definition de la zone d'affichage (toute la surface de la fenetre)
    renderPassInfo.renderArea.offset = {0, 0};
    renderPassInfo.renderArea.extent = extent;

    // Liaison de notre couleur d'effacement

```

```

renderPassInfo.clearValueCount = 1;
renderPassInfo.pClearValues = &clearColor;

// 4. Enregistrement du debut de la passe de rendu
// Nous utilisons VK_SUBPASS_CONTENTS_INLINE car nous n'utilisons pas de tampons secondaires
vkCmdBeginRenderPass(cmdBuffer, &renderPassInfo, VK_SUBPASS_CONTENTS_INLINE);

// --- Note : Aucun dessin n'est effectue ici, nous voulons juste un ecran uni ---

// 5. Enregistrement de la fin de la passe de rendu
vkCmdEndRenderPass(cmdBuffer);

// 6. Fin de l'enregistrement du tampon de commandes
if (vkEndCommandBuffer(cmdBuffer) != VK_SUCCESS) {
    throw std::runtime_error("Erreur : Impossible de finaliser l'enregistrement du tampon de commandes");
}
}
}

```

Analyse de la correction :

- Nous avons utilisé le flag `VK_COMMAND_BUFFER_USAGE_ONE_TIME_SUBMIT_BIT` . C'est une excellente pratique pour les moteurs de rendu modernes : cela indique au pilote graphique qu'immédiatement après l'exécution de ce tampon, celui-ci sera réenregistré. Le pilote peut alors appliquer des optimisations de mémoire cache spécifiques.
- L'union `VkClearValue` a été explicitement initialisée canal par canal (`float32[0]` à `float32[3]`). C'est la méthode la plus sûre et la plus lisible en C++ pour éviter toute ambiguïté d'initialisation d'union.

Module 3.2 : Synchronisation GPU-GPU et CPU-GPU (Semaphores & Fences)

1. Introduction Conceptuelle

Pour comprendre la synchronisation dans Vulkan, il faut abandonner l'idée que le processeur (CPU) et la carte graphique (GPU) travaillent de concert, pas à pas. En programmation graphique moderne, le CPU et le GPU sont deux processeurs totalement indépendants qui s'échangent des données de manière asynchrone.

Le problème de l'asynchronisme

Lorsque vous écrivez un programme C++ classique, chaque ligne de code s'exécute l'une après l'autre. Si vous écrivez :

```
int x = 5;
int y = x + 2;
```

Vous êtes certain que `x` vaut `5` avant que `y` ne soit calculé.

Dans Vulkan, ce n'est plus le cas entre le CPU et le GPU. Lorsque le CPU envoie des commandes de dessin au GPU (via les *Command Buffers* étudiés dans le Module 3.1), il ne fait que les mettre dans une file d'attente (*Queue*). Le CPU continue immédiatement son exécution sans attendre que le GPU ait fini de dessiner.

```
CPU: [Envoi Commandes Frame 1] ---> [Envoi Commandes Frame 2] ---> [Calculs Physiques...]
GPU:                                     [Exécution Frame 1.....]
```

Sans synchronisation explicite, deux problèmes majeurs surviennent : 1. La collision de ressources (*Data Race*) : Le CPU tente de modifier la mémoire d'une image (par exemple pour la mettre à jour) alors que le GPU est encore en train de la lire pour l'afficher à l'écran.

Cela provoque des déchirures d'écran (*tearing*) ou des plantages. 2. Le débordement de commandes : Le CPU, beaucoup plus rapide pour soumettre des commandes que le GPU pour les exécuter, s'emballe et s'épuise à allouer de la mémoire pour des milliers de trames en avance, provoquant une explosion de la consommation mémoire.

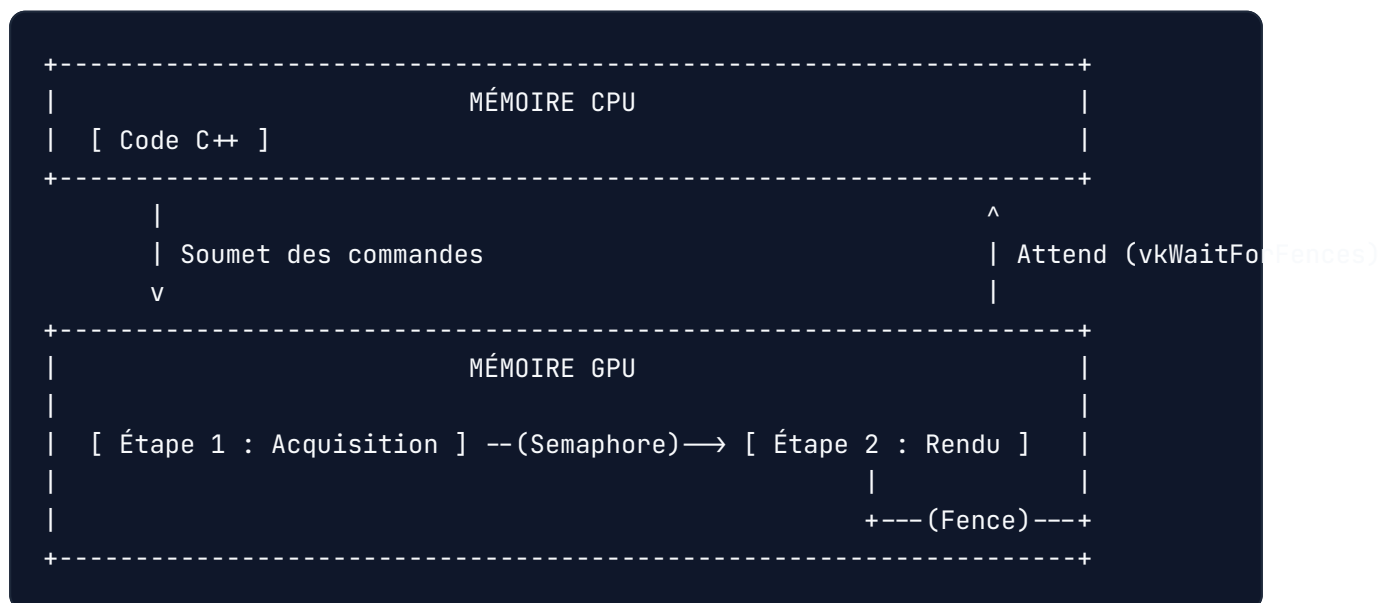
La philosophie de Vulkan : "Explicit"

Contrairement aux anciennes API comme OpenGL qui géraient cette synchronisation en arrière-plan (au prix de performances imprévisibles et de ralentissements mystérieux), Vulkan ne fait rien automatiquement. C'est à vous, le développeur, d'indiquer précisément quel processeur doit attendre quel événement.

Pour cela, Vulkan met à notre disposition deux outils principaux : * Les Fences (Barrières CPU-GPU) * Les Semaphores (Sémaphores GPU-GPU)

2. Fondations Théoriques

Pour concevoir une architecture de rendu stable, il est crucial de comprendre la différence fondamentale de cible entre les *Fences* et les *Semaphores*.



A. Les Fences (Synchronisation CPU-GPU)

Une Fence (barrière) est un objet de synchronisation utilisé par le GPU pour signaler au CPU qu'un travail est terminé.

- **Direction** : GPU $\rightarrow$ CPU.
- **État** : Une Fence est un objet binaire. Elle est soit **Signalée** (le GPU a fini), soit **Non-signalée** (le GPU travaille ou n'a pas commencé).
- **Fonctionnement** :
 - Le CPU soumet un travail au GPU et lui associe une Fence.
 - Le CPU continue ses calculs.
 - Arrivé à un point où le CPU a absolument besoin des résultats du GPU, il appelle la fonction `vkWaitForFences`. Le CPU s'arrête et "dort" tant que la Fence n'est pas passée à l'état "Signalée".
 - Une fois le signal reçu, le CPU doit **manuellement réinitialiser** la Fence à l'état "Non-signalé" via `vkResetFences` avant de pouvoir la réutiliser pour la trame suivante.

B. Les Semaphores (Synchronisation GPU-GPU)

Un Semaphore est utilisé pour coordonner des opérations au sein même du GPU, souvent entre différentes files d'attente (*Queues*) ou différentes étapes d'une même pipeline. Le CPU n'attend jamais directement sur un sémaphore.

- **Direction** : GPU $\rightarrow$ GPU.
- **État** : Binaire (dans sa forme classique). Signalé ou Non-signalé.
- **Fonctionnement** :
 - Une opération GPU $A \rightarrow B$ (ex: acquérir une image de la Swapchain) est lancée et configurée pour *signaler* un sémaphore SSS lorsqu'elle est terminée.
 - Une opération GPU $B \rightarrow C$ (ex: dessiner sur l'image) est soumise en même temps, mais on lui donne une consigne : "Attends que le sémaphore SSS soit signalé avant de commencer l'étape d'écriture des pixels".
 - Le GPU gère cela en interne, de manière extrêmement efficace, sans jamais faire intervenir le CPU.

- Contrairement aux Fences, les sémaphores sont automatiquement réinitialisés à l'état non-signalé dès qu'une opération GPU a consommé l'attente.

C. Les étapes de la Pipeline (Pipeline Stages)

Pour éviter de bloquer l'intégralité du GPU, Vulkan permet d'associer l'attente d'un sémaphore à une étape précise de la pipeline graphique (*Pipeline Stage*).

Par exemple, lors du rendu d'une trame, nous n'avons pas besoin de bloquer l'étape de lecture des sommets géométriques (*Vertex Input*) tant que l'image de la Swapchain n'est pas prête. Nous devons uniquement bloquer l'étape d'écriture des couleurs sur l'image (*Color Attachment Output*). Cela permet au GPU de commencer à traiter la géométrie 3D en parallèle, optimisant ainsi grandement les performances.

Tableau Comparatif Récapitulatif

Caractéristique	Fence (Barrière)	Semaphore (Sémaphore)
Cible de l'attente	Le CPU (votre code C++)	Le GPU (les files d'attente internes)
Signalé par	Le GPU (fin d'un lot de commandes)	Le GPU (fin d'une tâche spécifique)
Attendu par	Le CPU via <code>vkWaitForFences</code>	Le GPU via la soumission de commandes
Réinitialisation	Manuelle par le CPU (<code>vkResetFences</code>)	Automatique par l'API après consommation
Usage typique	Éviter que le CPU ne prépare la trame \$N+2\$ avant que la trame \$N\$ ne soit finie sur le GPU.	Attendre que l'image de l'écran soit disponible avant de dessiner dessus.

3. Implémentation Pratique Pas-à-Pas

Nous allons maintenant écrire le code C++ nécessaire pour créer et configurer ces objets de synchronisation. Nous allons concevoir un système de synchronisation pour gérer **plusieurs trames en vol** (*Frames in Flight*), une technique standard qui permet au CPU de préparer la trame suivante pendant que le GPU dessine la trame en cours.

Rappel C++ pour les débutants : L'initialisation dans Vulkan

Dans Vulkan, presque toutes les créations d'objets suivent le même schéma : 1. On remplit une structure de configuration de type `Vk...CreateInfo` . 2. On appelle une fonction de création de type `vkCreate...` . 3. On vérifie que la fonction renvoie `VK_SUCCESS` (un type énuméré `VkResult`).

Étape 1 : Définition des variables membres

Dans votre classe de rendu (par exemple `HelloTriangleApplication`), nous devons stocker les handles (pointeurs opaques Vulkan) de nos objets de synchronisation. Nous allons utiliser un système à double tampon (*Double Buffering*), donc nous définissons une constante :

```

#include <vulkan/vulkan.h>
#include <vector>
#include <stdexcept>

// Nous permettons d'avoir au maximum 2 trames en cours de traitement simultané
const int MAX_FRAMES_IN_FLIGHT = 2;

class HelloTriangleApplication {
private:
    VkDevice device; // Supposé initialisé dans le Module 1.1

    // Listes d'objets de synchronisation (un par trame en vol)
    std::vector<VkSemaphore> imageAvailableSemaphores;
    std::vector<VkSemaphore> renderFinishedSemaphores;
    std::vector<VkFence> inFlightFences;

    // Index pour suivre la trame logique actuelle (0 ou 1)
    size_t currentFrame = 0;

    void createSyncObjects();
    void cleanup();
};

```

Étape 2 : Création des Sémaphores

La création d'un sémaphore est extrêmement simple car la structure `VkSemaphoreCreateInfo` ne contient aucun paramètre complexe.

```

void HelloTriangleApplication::createSyncObjects() {
    // Redimensionner nos conteneurs pour accueillir les objets de chaque trame
    imageAvailableSemaphores.resize(MAX_FRAMES_IN_FLIGHT);
    renderFinishedSemaphores.resize(MAX_FRAMES_IN_FLIGHT);
    inFlightFences.resize(MAX_FRAMES_IN_FLIGHT);

    // Structure de configuration commune pour les sémaphores
    VkSemaphoreCreateInfo semaphoreInfo{};
    semaphoreInfo.sType = VK_STRUCTURE_TYPE_SEMAPHORE_CREATE_INFO;
    semaphoreInfo.pNext = nullptr; // Réservé pour des extensions futures
    semaphoreInfo.flags = 0;       // Réservé pour un usage futur
}

```


Étape 3 : Création des Fences (Le piège du démarrage)

C'est ici que réside le piège le plus classique de Vulkan. Si nous créons une Fence par défaut, elle est créée à l'état Non-signalé.

Lors de la toute première trame de notre boucle de rendu, notre code va appeler `vkWaitForFences` pour s'assurer que le GPU a fini de travailler. Mais comme aucun travail n'a encore été soumis, la Fence ne sera jamais signalée. Le programme va alors se bloquer indéfiniment (un *Deadlock* ou gel complet de l'application).

Pour résoudre cela, nous devons créer la Fence directement à l'état Signalé en utilisant le flag `VK_FENCE_CREATE_SIGNALED_BIT`.

```
VkFenceCreateInfo fenceInfo{};
fenceInfo.sType = VK_STRUCTURE_TYPE_FENCE_CREATE_INFO;
fenceInfo.pNext = nullptr;
// TRÈS IMPORTANT : Initialiser la Fence à l'état signalé
fenceInfo.flags = VK_FENCE_CREATE_SIGNALED_BIT;

// Boucle de création pour chaque trame en vol
for (size_t i = 0; i < MAX_FRAMES_IN_FLIGHT; i++) {
    // Création du sémaphore d'acquisition d'image
    if (vkCreateSemaphore(device, &semaphoreInfo, nullptr, &imageAvailableSemaphores[i]) != VK_SUCCESS) {
        throw std::runtime_error("Erreur : Impossible de créer le sémaphore d'acquisition d'image");
    }

    // Création du sémaphore de fin de rendu
    if (vkCreateSemaphore(device, &semaphoreInfo, nullptr, &renderFinishedSemaphores[i]) != VK_SUCCESS) {
        throw std::runtime_error("Erreur : Impossible de créer le sémaphore de fin de rendu");
    }

    // Création de la Fence de synchronisation CPU-GPU
    if (vkCreateFence(device, &fenceInfo, nullptr, &inFlightFences[i]) != VK_SUCCESS) {
        throw std::runtime_error("Erreur : Impossible de créer la Fence de synchronisation CPU-GPU");
    }
}
}
```

Étape 4 : Nettoyage des ressources (Destruction)

Comme pour toute ressource allouée dans Vulkan, il est impératif de détruire explicitement ces objets lorsque l'application se ferme pour éviter les fuites de mémoire. La destruction doit se faire après s'être assuré que le GPU a totalement fini de travailler.

```
void HelloTriangleApplication::cleanup() {
    // Attendre que le GPU soit complètement inactif avant de détruire les objets
    vkDeviceWaitIdle(device);

    for (size_t i = 0; i < MAX_FRAMES_IN_FLIGHT; i++) {
        vkDestroySemaphore(device, imageAvailableSemaphores[i], nullptr);
        vkDestroySemaphore(device, renderFinishedSemaphores[i], nullptr);
        vkDestroyFence(device, inFlightFences[i], nullptr);
    }

    // Le reste du nettoyage du périphérique (VkDevice) se fait après...
}
```

4. Pièges Fréquents et Bonnes Pratiques

1. Le Deadlock de la première trame

- **Le Piège** : Omettre le flag `VK_FENCE_CREATE_SIGNALED_BIT` lors de la création de la Fence.
- **Le Symptôme** : L'application s'ouvre sur une fenêtre noire et se fige instantanément sans aucun message d'erreur.
- **La Solution** : Toujours s'assurer que les Fences contrôlant le flux principal de la boucle de rendu soient créées à l'état signalé.

2. Oublier de réinitialiser la Fence (`vkResetFences`)

- **Le Piège** : Appeler `vkWaitForFences` mais oublier d'appeler `vkResetFences` avant de soumettre à nouveau le travail au GPU.
- **Le Symptôme** : Le CPU n'attend plus du tout le GPU après la première trame. Le CPU s'emballe, ce qui provoque des corruptions graphiques massives ou des crashes aléatoires du pilote graphique (souvent une erreur de type `Device Lost`).
- **La Solution** : Le flux logique doit toujours être : `vkWaitForFences` $\rightarrow$ `vkResetFences` $\rightarrow$ `vkQueueSubmit` .

3. La sur-synchronisation (Stall)

- **Le Piège** : Utiliser une seule Fence globale (`MAX_FRAMES_IN_FLIGHT = 1`).
 - **Le Symptôme** : Le jeu fonctionne correctement mais les performances (FPS) sont extrêmement basses, car le CPU attend que le GPU ait fini d'afficher la trame N avant même de commencer à calculer la trame $N+1$. Le GPU et le CPU passent la moitié de leur temps à s'attendre mutuellement.
 - **La Solution** : Utiliser le double-buffering (2 trames en vol) ou le triple-buffering (3 trames en vol). Cela permet de paralléliser le travail : pendant que le GPU dessine la trame 0, le CPU prépare déjà les commandes de la trame 1.
-

5. Synthèse Pédagogique

Pour bien fixer les idées, voici le déroulement chronologique exact de la synchronisation lors de l'affichage d'une trame (ce flux sera implémenté en détail dans le Module 3.3) :

[ÉTAPE CPU]

1. Attendre que la trame précédente soit finie : `vkWaitForFences(inFlightFences[currentFrame], 1, &fence, true, ULLONG_MAX)`
2. Verrouiller la Fence pour le nouveau travail : `vkResetFences(inFlightFences[currentFrame], 1, &fence)`

[ÉTAPE GPU - ACQUISITION]

3. Demander une image à la Swapchain.
Le GPU signalera `'imageAvailableSemaphores[currentFrame]'` quand elle sera prête.

[ÉTAPE GPU - RENDU]

4. Soumettre les commandes de dessin à la Queue.
 - Configurer pour ATTENDRE : `'imageAvailableSemaphores[currentFrame]'`
 - Configurer pour SIGNALER en fin de rendu : `'renderFinishedSemaphores[currentFrame]'`
 - Associer la Fence `'inFlightFences[currentFrame]'` pour qu'elle se signale à la toute fin.

[ÉTAPE GPU - PRÉSENTATION]

5. Envoyer l'image dessinée à l'écran.
 - Configurer pour ATTENDRE : `'renderFinishedSemaphores[currentFrame]'`
 6. Passer à la trame suivante : `currentFrame = (currentFrame + 1) % MAX_FRAMES_IN_FLIGHT`
-

6. Exercice Pratique d'Application

Énoncé

Dans cet exercice, vous devez concevoir une classe C++ autonome nommée `VulkanSyncManager`. Cette classe doit encapsuler la création, la gestion et la destruction de tous les objets de synchronisation nécessaires pour un nombre variable de trames en vol (\$N\$).

L'objectif est de fournir une interface propre pour éviter de polluer le code principal de votre moteur de rendu.

Spécifications de la classe :

1. Le constructeur doit prendre en paramètre une référence vers le périphérique logique `VkDevice` et le nombre de trames en vol souhaité.
2. Elle doit exposer une méthode `initialize()` qui crée les ressources.
3. Elle doit exposer des accesseurs (`getters`) pour récupérer le sémaphore d'acquisition, le sémaphore de rendu et la Fence pour une trame d'index donné.
4. Elle doit exposer une méthode `waitAndResetFence(size_t frameIndex)` qui gère proprement l'attente et la réinitialisation de la Fence de la trame active.
5. Elle doit gérer sa propre destruction proprement dans son destructeur.

Indices

- Pensez à stocker le `VkDevice` en tant que variable membre pour pouvoir détruire les objets dans le destructeur.
 - Utilisez `std::vector` pour stocker dynamiquement les objets de synchronisation en fonction du paramètre passé au constructeur.
 - Rappel de la signature de `vkWaitForFences` :

```
cpp VkResult vkWaitForFences( VkDevice device, uint32_t fenceCount, const  
VkFence* pFences, VkBool32 waitAll, uint64_t timeout );
```

 Pour attendre indéfiniment une seule Fence, utilisez `waitAll = VK_TRUE` et `timeout = UINT64_MAX`.
-

Correction de l'Exercice

Voici l'implémentation complète et rigoureuse de la classe `VulkanSyncManager` .

```

#include <vulkan/vulkan.h>
#include <vector>
#include <stdexcept>
#include <iostream>

class VulkanSyncManager {
private:
    VkDevice m_device;
    size_t m_maxFramesInFlight;

    std::vector<VkSemaphore> m_imageAvailableSemaphores;
    std::vector<VkSemaphore> m_renderFinishedSemaphores;
    std::vector<VkFence> m_inFlightFences;

public:
    // Constructeur
    VulkanSyncManager(VkDevice device, size_t maxFramesInFlight)
        : m_device(device), m_maxFramesInFlight(maxFramesInFlight) {}

    // Destructeur automatique
    ~VulkanSyncManager() {
        cleanup();
    }

    // Initialisation des ressources
    void initialize() {
        m_imageAvailableSemaphores.resize(m_maxFramesInFlight);
        m_renderFinishedSemaphores.resize(m_maxFramesInFlight);
        m_inFlightFences.resize(m_maxFramesInFlight);

        // Configuration des Sémaphores
        VkSemaphoreCreateInfo semaphoreInfo{};
        semaphoreInfo.sType = VK_STRUCTURE_TYPE_SEMAPHORE_CREATE_INFO;

        // Configuration des Fences (avec le flag de signalement initial !)
        VkFenceCreateInfo fenceInfo{};
        fenceInfo.sType = VK_STRUCTURE_TYPE_FENCE_CREATE_INFO;
        fenceInfo.flags = VK_FENCE_CREATE_SIGNALED_BIT;

        for (size_t i = 0; i < m_maxFramesInFlight; ++i) {
            if (vkCreateSemaphore(m_device, &semaphoreInfo, nullptr, &m_imageAvailableSemaphores[i]) != VK_SUCCESS)
                vkCreateSemaphore(m_device, &semaphoreInfo, nullptr, &m_renderFinishedSemaphores[i]) != VK_SUCCESS)
                throw std::runtime_error("Echec de la creation des semaphores pour la trame " + std::to_string(i));
        }

        if (vkCreateFence(m_device, &fenceInfo, nullptr, &m_inFlightFences[i]) != VK_SUCCESS)
            throw std::runtime_error("Echec de la creation de la Fence pour la trame " + std::to_string(i));
        }
    }

```

```

    }
    std::cout << "VulkanSyncManager : Initialisation reussie avec " << m_maxFramesInFlight;
}

// Attente et réinitialisation de la Fence pour la trame courante
void waitAndResetFence(size_t frameIndex) {
    if (frameIndex ≥ m_maxFramesInFlight) {
        throw std::out_of_range("Index de trame hors limites !");
    }

    // 1. Attendre que le GPU ait fini d'utiliser les ressources de cette trame logique
    // Nous passons 1 en nombre de fences, et l'adresse de notre fence dans le vecteur.
    vkWaitForFences(m_device, 1, &m_inFlightFences[frameIndex], VK_TRUE, UIN64_MAX);

    // 2. Réinitialiser manuellement la Fence pour bloquer le CPU au prochain tour
    vkResetFences(m_device, 1, &m_inFlightFences[frameIndex]);
}

// Getters pour l'intégration dans la boucle de rendu
VkSemaphore getImageAvailableSemaphore(size_t frameIndex) const {
    return m_imageAvailableSemaphores[frameIndex];
}

VkSemaphore getRenderFinishedSemaphore(size_t frameIndex) const {
    return m_renderFinishedSemaphores[frameIndex];
}

VkFence getFence(size_t frameIndex) const {
    return m_inFlightFences[frameIndex];
}

private:
    // Nettoyage interne
    void cleanup() {
        // S'assurer que le périphérique n'est pas en train de travailler avant de détruire
        if (m_device ≠ VK_NULL_HANDLE) {
            vkDeviceWaitIdle(m_device);

            for (size_t i = 0; i < m_maxFramesInFlight; ++i) {
                if (m_imageAvailableSemaphores[i] ≠ VK_NULL_HANDLE) {
                    vkDestroySemaphore(m_device, m_imageAvailableSemaphores[i], nullptr);
                }
                if (m_renderFinishedSemaphores[i] ≠ VK_NULL_HANDLE) {
                    vkDestroySemaphore(m_device, m_renderFinishedSemaphores[i], nullptr);
                }
                if (m_inFlightFences[i] ≠ VK_NULL_HANDLE) {
                    vkDestroyFence(m_device, m_inFlightFences[i], nullptr);
                }
            }
        }
    }
}

```

```

    }
    std::cout << "VulkanSyncManager : Ressources de synchronisation liberees" << std::endl;
}
};

```

Explication de la correction

- **Encapsulation RAII** : Le destructeur appelle automatiquement `cleanup()`, garantissant qu'aucune fuite de mémoire ne se produira, même en cas d'exception dans le reste du programme.
- **Sécurité** : La méthode `waitAndResetFence` effectue une vérification de sécurité sur l'index de la trame pour éviter des accès mémoire invalides dans le `std::vector`.
- **Propreté du code** : En utilisant cette classe dans notre boucle principale, le code de rendu sera allégé de dizaines de lignes de configuration de bas niveau, le rendant beaucoup plus lisible pour la suite du cours (Module 3.3).

Module 3.3 : Boucle de rendu et affichage du premier triangle

Bienvenue dans l'étape ultime de ce premier cycle d'apprentissage de Vulkan. Dans les modules précédents, vous avez configuré l'instance, sélectionné votre carte graphique (Module 1.1), initialisé la surface d'affichage et la Swapchain (Module 1.2 et 1.3), compilé vos shaders (Module 2.1), configuré la pipeline graphique (Module 2.2), préparé les passes de rendu et les Framebuffers (Module 2.3), enregistré vos commandes de dessin (Module 3.1) et mis en place les primitives de synchronisation (Module 3.2).

Aujourd'hui, nous allons assembler toutes ces pièces pour donner vie à notre application : concevoir la boucle de rendu (render loop) et afficher enfin notre **premier triangle** à l'écran. Nous aborderons également la gestion dynamique du redimensionnement de la fenêtre, un aspect crucial pour toute application graphique moderne.

1. Introduction Conceptuelle

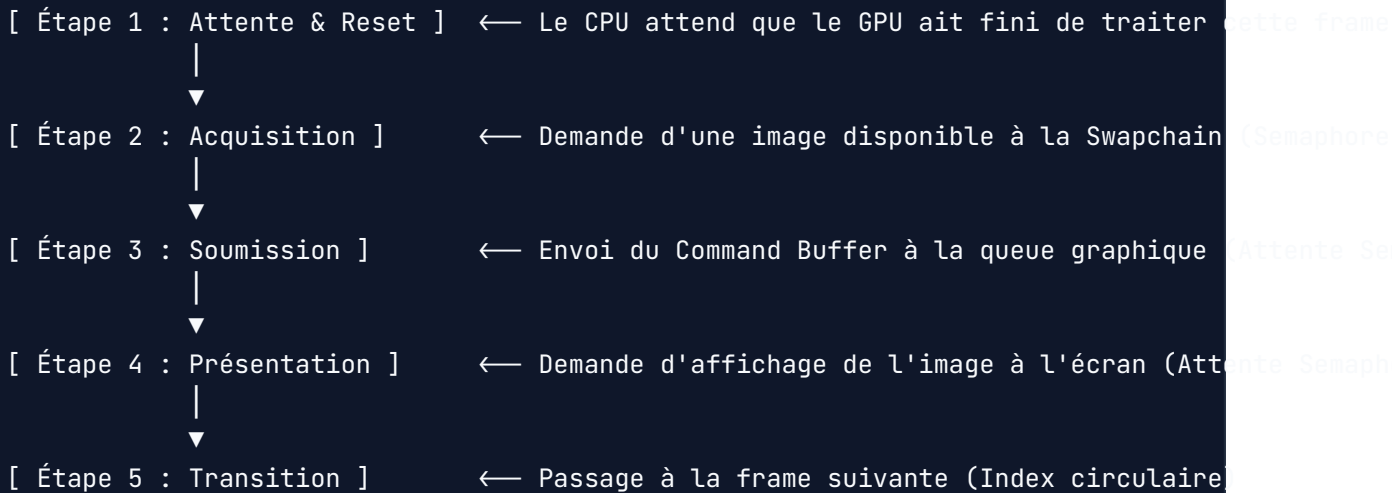
Pour comprendre la boucle de rendu de Vulkan, il faut abandonner la vision séquentielle classique de la programmation CPU. En programmation graphique, nous orchestrans un flux de travail asynchrone entre deux processeurs distincts : le CPU (le processeur central) et le GPU (le processeur graphique).

Le paradigme Producteur-Consommateur

Dans notre application : * Le CPU agit comme le **Producteur** : il prépare les instructions, gère les événements système (clavier, souris, redimensionnement) et soumet le travail au GPU. * Le GPU agit comme le **Consommateur** : il prend les paquets d'instructions (Command Buffers) soumis par le CPU, exécute les calculs géométriques et de pixellisation, puis écrit le résultat dans une image de la Swapchain pour l'afficher.

Le cycle de vie d'une image (Frame Lifecycle)

Pour afficher une seule image à l'écran, l'application doit répéter indéfiniment les cinq étapes suivantes :



Pourquoi le "Multi-buffering" (Frames in Flight) ?

Si le CPU devait attendre que le GPU ait complètement terminé d'afficher l'image N avant de commencer à préparer l'image $N+1$, le GPU resterait inactif pendant que le CPU travaille, et inversement. C'est ce qu'on appelle un goulot d'étranglement (stall).

Pour maximiser les performances, nous utilisons le concept de **Frames in Flight** (cadres en vol). Nous autorisons le CPU à préparer l'image suivante pendant que le GPU dessine la précédente. En pratique, nous définissons une constante :

```
const int MAX_FRAMES_IN_FLIGHT = 2; // Double buffering au niveau de la synchronisation
```

Cela signifie que nous allons dupliquer nos objets de synchronisation (Semaphores et Fences) pour éviter que le CPU ne prenne trop d'avance sur le GPU.

2. Fondations Théoriques

Avant d'écrire le code, analysons les fonctions clés de l'API Vulkan que nous allons utiliser.

A. L'acquisition de l'image : `vkAcquireNextImageKHR`

Cette fonction demande à la Swapchain de nous fournir l'index de la prochaine image disponible pour le dessin.

```
VkResult vkAcquireNextImageKHR(  
    VkDevice          device,  
    VkSwapchainKHR    swapchain,  
    uint64_t          timeout,  
    VkSemaphore        semaphore,  
    VkFence            fence,  
    uint32_t*         pImageIndex  
);
```

- `timeout` : Temps maximum d'attente en nanosecondes. Utiliser `UINT64_MAX` désactive le timeout (l'appel bloque jusqu'à ce qu'une image soit libre).
- `semaphore` : Le sémaphore qui sera signalé par le GPU lorsque l'image sera réellement prête à être écrite.
- `fence` : Une barrière CPU-GPU optionnelle (nous utiliserons `VK_NULL_HANDLE` ici car nous synchronisons via sémaphore).
- `pImageIndex` : Pointeur vers un entier qui recevra l'index de l'image de la Swapchain (par exemple `0`, `1` ou `2`).

B. La soumission des commandes : `vkQueueSubmit`

Une fois que nous connaissons l'index de l'image, nous soumettons le Command Buffer correspondant à la queue graphique.

```
VkResult vkQueueSubmit(  
    VkQueue          queue,  
    uint32_t         submitCount,  
    const VkSubmitInfo* pSubmits,  
    VkFence          fence  
);
```

- `queue` : La file d'attente graphique (Graphics Queue) obtenue lors de l'initialisation du périphérique logique.

- `pSubmits` : Un pointeur vers une structure `VkSubmitInfo` qui décrit ce que l'on soumet, quels sémaphores attendre, et quels sémaphores signaler à la fin.
- `fence` : Une Fence qui sera signalée par le GPU lorsque l'exécution de ce Command Buffer sera totalement terminée. Cela permet au CPU de savoir qu'il peut réutiliser les ressources de cette frame en toute sécurité.

C. La présentation de l'image : `vkQueuePresentKHR`

Enfin, nous demandons à la Swapchain d'afficher l'image calculée sur notre écran.

```
VkResult vkQueuePresentKHR(
    VkQueue          queue,
    const VkPresentInfoKHR* pPresentInfo
);
```

- `queue` : La file d'attente de présentation (Present Queue).
- `pPresentInfo` : Structure contenant les sémaphores à attendre (le rendu doit être fini) et l'index de l'image à présenter.

3. Implémentation Pratique Pas-à-Pas

Nous allons maintenant intégrer ces concepts dans notre architecture C++. Nous supposons que vous disposez d'une classe principale `HelloTriangleApplication`.

Étape 1 : Déclaration des membres de synchronisation

Nous devons stocker nos sémaphores et fences pour chaque "frame in flight". Ajoutez ces variables membres à votre classe :

```
// Nombre maximum d'images traitées simultanément par le CPU/GPU
const int MAX_FRAMES_IN_FLIGHT = 2;

// Objets de synchronisation (un jeu par frame en vol)
std::vector<VkSemaphore> imageAvailableSemaphores;
std::vector<VkSemaphore> renderFinishedSemaphores;
std::vector<VkFence> inFlightFences;

// Index de la frame logique actuelle (0 ou 1 si MAX_FRAMES_IN_FLIGHT = 2)
size_t currentFrame = 0;

// Flag pour gérer le redimensionnement de la fenêtre
bool framebufferResized = false;
```

Étape 2 : Initialisation des objets de synchronisation

Comme étudié dans le Module 3.2, nous devons instancier ces objets. Voici la fonction d'initialisation complète :

```
void createSyncObjects() {
    imageAvailableSemaphores.resize(MAX_FRAMES_IN_FLIGHT);
    renderFinishedSemaphores.resize(MAX_FRAMES_IN_FLIGHT);
    inFlightFences.resize(MAX_FRAMES_IN_FLIGHT);

    VkSemaphoreCreateInfo semaphoreInfo{};
    semaphoreInfo.sType = VK_STRUCTURE_TYPE_SEMAPHORE_CREATE_INFO;

    VkFenceCreateInfo fenceInfo{};
    fenceInfo.sType = VK_STRUCTURE_TYPE_FENCE_CREATE_INFO;
    // TRÈS IMPORTANT : On initialise la Fence à l'état SIGNALÉ.
    // Sinon, au tout premier appel de drawFrame, le CPU attendra indéfiniment
    // une Fence qui n'a jamais été soumise au GPU !
    fenceInfo.flags = VK_FENCE_CREATE_SIGNALED_BIT;

    for (size_t i = 0; i < MAX_FRAMES_IN_FLIGHT; i++) {
        if (vkCreateSemaphore(device, &semaphoreInfo, nullptr, &imageAvailableSemaphores[i]) ||
            vkCreateSemaphore(device, &semaphoreInfo, nullptr, &renderFinishedSemaphores[i]) ||
            vkCreateFence(device, &fenceInfo, nullptr, &inFlightFences[i]) != VK_SUCCESS) {
            throw std::runtime_error("Erreur : Impossible de créer les objets de synchronisation");
        }
    }
}
```

Étape 3 : La fonction de dessin principale `drawFrame()`

Voici le cœur de notre boucle de rendu. Lisez attentivement les commentaires pour comprendre le rôle de chaque ligne.

```

void drawFrame() {
    // 1. Attendre que la frame précédente associée à cet index soit terminée par le GPU
    // Le paramètre VK_TRUE indique que l'on attend toutes les fences (ici une seule).
    // UINT64_MAX désactive le timeout.
    vkWaitForFences(device, 1, &inFlightFences[currentFrame], VK_TRUE, UINT64_MAX);

    // 2. Acquérir une image de la Swapchain
    uint32_t imageIndex;
    VkResult result = vkAcquireNextImageKHR(
        device,
        swapChain,
        UINT64_MAX,
        imageAvailableSemaphores[currentFrame], // Sera signalé quand l'image sera disponible
        VK_NULL_HANDLE,
        &imageIndex
    );

    // Vérifier si la Swapchain doit être recrée (ex: redimensionnement de fenêtre)
    if (result == VK_ERROR_OUT_OF_DATE_KHR) {
        recreateSwapChain();
        return;
    } else if (result != VK_SUCCESS && result != VK_SUBOPTIMAL_KHR) {
        throw std::runtime_error("Erreur : Impossible d'acquérir une image de la Swapchain");
    }

    // N'oublions pas de réinitialiser la Fence uniquement si nous sommes sûrs de soumettre
    vkResetFences(device, 1, &inFlightFences[currentFrame]);

    // 3. Configurer la soumission du Command Buffer
    VkSubmitInfo submitInfo{};
    submitInfo.sType = VK_STRUCTURE_TYPE_SUBMIT_INFO;

    // Quels sémaphores le GPU doit attendre avant de commencer l'exécution
    VkSemaphore waitSemaphores[] = { imageAvailableSemaphores[currentFrame] };
    // À quelle étape du pipeline graphique doit-on attendre ?
    // On attend d'avoir l'image pour écrire les couleurs (Color Attachment Output).
    // Les calculs de Vertex Shaders peuvent commencer avant ! C'est une optimisation majeure
    VkPipelineStageFlags waitStages[] = { VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT };

    submitInfo.waitSemaphoreCount = 1;
    submitInfo.pWaitSemaphores = waitSemaphores;
    submitInfo.pWaitDstStageMask = waitStages;

    // Quel Command Buffer soumettre (celui correspondant à l'image acquise)
    submitInfo.commandBufferCount = 1;
    submitInfo.pCommandBuffers = &commandBuffers[imageIndex];

    // Quel sémaphore signaler une fois que le GPU a fini le rendu de ce Command Buffer

```

```

VkSemaphore signalSemaphores[] = { renderFinishedSemaphores[currentFrame] };
submitInfo.signalSemaphoreCount = 1;
submitInfo.pSignalSemaphores = signalSemaphores;

// Soumettre le travail à la Queue Graphique en associant la Fence de la frame courante
if (vkQueueSubmit(graphicsQueue, 1, &submitInfo, inFlightFences[currentFrame]) != VK_SUCCESS)
    throw std::runtime_error("Erreur : Échec de la soumission du Command Buffer à la Queue Graphique");
}

// 4. Configurer la Présentation de l'image à l'écran
VkPresentInfoKHR presentInfo{};
presentInfo.sType = VK_STRUCTURE_TYPE_PRESENT_INFO_KHR;

// Attendre que le rendu soit complètement terminé (renderFinishedSemaphore)
presentInfo.waitSemaphoreCount = 1;
presentInfo.pWaitSemaphores = signalSemaphores;

VkSwapchainKHR swapChains[] = { swapChain };
presentInfo.swapchainCount = 1;
presentInfo.pSwapchains = swapChains;
presentInfo.pImageIndices = &imageIndex;
presentInfo.pResults = nullptr; // Optionnel

// Envoyer la demande de présentation
result = vkQueuePresentKHR(presentQueue, &presentInfo);

// Vérifier si la Swapchain est obsolète après la présentation
if (result == VK_ERROR_OUT_OF_DATE_KHR || result == VK_SUBOPTIMAL_KHR || framebufferResized)
    recreateSwapChain();
} else if (result != VK_SUCCESS) {
    throw std::runtime_error("Erreur : Échec de la présentation de l'image !");
}

// 5. Passer à la frame suivante (index circulaire)
currentFrame = (currentFrame + 1) % MAX_FRAMES_IN_FLIGHT;
}

```

Étape 4 : Gestion du redimensionnement (Recreate Swapchain)

Si l'utilisateur redimensionne la fenêtre, la taille des buffers de la Swapchain ne correspond plus à la surface d'affichage. Vulkan lève alors l'erreur `VK_ERROR_OUT_OF_DATE_KHR`. Nous devons détruire et reconstruire la Swapchain.

Voici l'implémentation de la reconstruction de la Swapchain :


```

void recreateSwapChain() {
    // Gestion de la minimisation de la fenêtre (taille 0)
    int width = 0, height = 0;
    glfwGetFramebufferSize(window, &width, &height);
    while (width == 0 || height == 0) {
        glfwGetFramebufferSize(window, &width, &height);
        glfwWaitEvents(); // Met l'application en pause en attendant un événement
    }

    // Attendre que le GPU ait fini toutes ses tâches avant de détruire quoi que ce soit
    vkDeviceWaitIdle(device);

    // Nettoyage de l'ancienne Swapchain et de ses dépendances
    cleanupSwapChain();

    // Recréation complète (voir Modules 1.3, 2.3 et 3.1)
    createSwapChain();
    createImageViews();
    createRenderPass();
    createGraphicsPipeline();
    createFramebuffers();
    createCommandBuffers(); // Ré-enregistrement des commandes de dessin
}

```

Note : La fonction `cleanupSwapChain()` doit détruire les Framebuffers, les Image Views, la Swapchain elle-même, et libérer les Command Buffers associés.

Étape 5 : Intégration dans la boucle principale (Main Loop)

Votre fonction principale de boucle d'événements ressemble désormais à ceci :

```

void mainLoop() {
    while (!glfwWindowShouldClose(window)) {
        glfwPollEvents(); // Gère les entrées utilisateur
        drawFrame();      // Dessine la frame suivante
    }

    // Une fois la boucle quittée, on attend que le GPU soit inactif avant de nettoyer la m
    vkDeviceWaitIdle(device);
}

```

Étape 6 : Nettoyage des ressources (Cleanup)

Il est impératif de détruire les objets de synchronisation lors de la fermeture de l'application pour éviter les fuites de mémoire.

```
void cleanup() {
    // 1. Nettoyage de la Swapchain
    cleanupSwapChain();

    // 2. Destruction des objets de synchronisation
    for (size_t i = 0; i < MAX_FRAMES_IN_FLIGHT; i++) {
        vkDestroySemaphore(device, renderFinishedSemaphores[i], nullptr);
        vkDestroySemaphore(device, imageAvailableSemaphores[i], nullptr);
        vkDestroyFence(device, inFlightFences[i], nullptr);
    }

    // 3. Destruction du périphérique logique et de l'instance (Modules précédents)
    vkDestroyDevice(device, nullptr);
    // ...
}
```

4. Pièges Fréquents et Bonnes Pratiques

1. Le blocage infini au démarrage (Deadlock)

Symptôme : L'application se lance mais reste figée sur un écran noir sans aucun message d'erreur. **Cause** : Vous avez oublié de configurer le flag `VK_FENCE_CREATE_SIGNALED_BIT` lors de la création de vos Fences. Le CPU exécute `vkWaitForFences` sur la première frame, mais comme aucune tâche GPU n'a encore été soumise, la Fence n'est pas signalée. Le CPU attend indéfiniment. **Solution** : Assurez-vous d'avoir `fenceInfo.flags = VK_FENCE_CREATE_SIGNALED_BIT;`.

2. Oubli de réinitialiser la Fence

Symptôme : Les Validation Layers de Vulkan crient au scandale ou l'application plante après quelques frames. **Cause** : Avant de soumettre du travail au GPU avec `vkQueueSubmit`, vous devez réinitialiser la Fence associée via `vkResetFences`. Sinon, Vulkan considère que la

Fence est déjà signalée et ne bloquera pas le CPU à la frame suivante, brisant toute la synchronisation.

3. Crash lors de la minimisation de la fenêtre

Symptôme : L'application crash instantanément lorsque l'on réduit la fenêtre dans la barre des tâches. **Cause :** Lors de la minimisation, la largeur et la hauteur de la fenêtre deviennent `0`. Créer une Swapchain de taille `0 \times 0` est interdit par Vulkan et provoque un crash. **Solution :** Ajouter la boucle d'attente `while (width == 0 || height == 0)` dans `recreateSwapChain()` pour mettre l'application en pause tant que la fenêtre est minimisée.

5. Synthèse Pédagogique

Voici un tableau récapitulatif du rôle de chaque objet de synchronisation dans notre boucle de rendu :

Objet	Type	Qui l'attend ?	Qui le signale ?	Rôle exact
<code>inFlightFences</code>	Fence (CPU-GPU)	CPU (au début de <code>drawFrame</code>)	GPU (à la fin de l'exécution du Command Buffer)	Empêche le CPU de préparer une nouvelle frame si le GPU n'a pas fini de traiter cette même frame du cycle précédent.
<code>imageAvailableSemaphores</code>	Semaphore (GPU-GPU)	GPU (étape d'écriture des couleurs dans le pipeline)	GPU (moteur de présentation, quand l'image est libérée)	Garantit que le GPU ne commence pas à dessiner sur une image qui est encore affichée à l'écran.
<code>renderFinishedSemaphores</code>	Semaphore (GPU-GPU)	GPU (moteur de présentation, avant d'afficher)	GPU (pipeline graphique, quand le dessin est fini)	Garantit que l'écran n'affiche pas une image à moitié dessinée (évite le

Objet	Type	Qui l'attend ?	Qui le signale ?	Rôle exact
				screen tearing).

6. Exercice Pratique d'Application

Objectif : Créer un effet de pulsation colorée en arrière-plan

Pour valider votre compréhension de la boucle de rendu, vous allez modifier la couleur de fond de l'écran (le "Clear Color") de manière dynamique à chaque frame, afin de créer un effet de pulsation lumineuse (par exemple, une transition sinusoïdale de la couleur rouge).

Indices

1. Où est définie la couleur de fond ? Dans le Module 3.1, lors de l'enregistrement du Command Buffer, vous avez défini une structure `VkClearValue` : `cpp VkClearValue clearColor = {{{0.0f, 0.0f, 0.0f, 1.0f}}}; // Noir par défaut`
2. Le problème de l'enregistrement statique : Si vous avez enregistré vos Command Buffers une fois pour toutes lors de l'initialisation, la couleur de fond restera statique. Pour la rendre dynamique, vous devez ré-enregistrer le Command Buffer à chaque frame à l'intérieur de la fonction `drawFrame()`, juste avant de le soumettre.
3. Calcul du temps : Utilisez `<chrono>` ou `glfwGetTime()` pour obtenir le temps écoulé en secondes, puis appliquez la fonction sinus `std::sin()` pour obtenir une valeur oscillant entre `0.0f` et `1.0f`.

Correction Complète de l'Exercice

Voici comment modifier votre code pour réaliser cet exercice.

1. Modification de la fonction d'enregistrement du Command Buffer

Nous allons modifier la fonction `recordCommandBuffer` pour qu'elle accepte l'index de l'image et la valeur de couleur dynamique en paramètre.

```
void recordCommandBuffer(VkCommandBuffer commandBuffer, uint32_t imageIndex, float redIntensity) {
    VkCommandBufferBeginInfo beginInfo{};
    beginInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO;

    if (vkBeginCommandBuffer(commandBuffer, &beginInfo) != VK_SUCCESS) {
        throw std::runtime_error("Erreur : Impossible de commencer l'enregistrement du Command Buffer");
    }

    VkRenderPassBeginInfo renderPassInfo{};
    renderPassInfo.sType = VK_STRUCTURE_TYPE_RENDER_PASS_BEGIN_INFO;
    renderPassInfo.renderPass = renderPass;
    renderPassInfo.framebuffer = swapChainFramebuffers[imageIndex];
    renderPassInfo.renderArea.offset = {0, 0};
    renderPassInfo.renderArea.extent = swapChainExtent;

    // Application de la couleur dynamique (pulsation sur le canal Rouge)
    VkClearColorValue clearColor = {{{redIntensity, 0.0f, 0.0f, 1.0f}}};
    renderPassInfo.clearValueCount = 1;
    renderPassInfo.pClearValues = &clearColor;

    vkCmdBeginRenderPass(commandBuffer, &renderPassInfo, VK_SUBPASS_CONTENTS_INLINE);

    // Liaison de la pipeline graphique
    vkCmdBindPipeline(commandBuffer, VK_PIPELINE_BIND_POINT_GRAPHICS, graphicsPipeline);

    // Commande de dessin du triangle (3 sommets, 1 instance)
    vkCmdDraw(commandBuffer, 3, 1, 0, 0);

    vkCmdEndRenderPass(commandBuffer);

    if (vkEndCommandBuffer(commandBuffer) != VK_SUCCESS) {
        throw std::runtime_error("Erreur : Impossible de finaliser l'enregistrement du Command Buffer");
    }
}
```

2. Mise à jour de la boucle de rendu `drawFrame()`

Dans `drawFrame()`, nous calculons l'intensité du rouge en fonction du temps, nous ré-enregistrons le Command Buffer de la frame courante, puis nous le soumettons.

```

void drawFrame() {
    // 1. Attente de la Fence
    vkWaitForFences(device, 1, &inFlightFences[currentFrame], VK_TRUE, UINT64_MAX);

    // 2. Acquisition de l'image
    uint32_t imageIndex;
    VkResult result = vkAcquireNextImageKHR(device, swapChain, UINT64_MAX, imageAvailableSemaphore, &imageIndex);

    if (result == VK_ERROR_OUT_OF_DATE_KHR) {
        recreateSwapChain();
        return;
    } else if (result != VK_SUCCESS && result != VK_SUBOPTIMAL_KHR) {
        throw std::runtime_error("Erreur : Échec de l'acquisition de l'image !");
    }

    // Réinitialisation de la Fence
    vkResetFences(device, 1, &inFlightFences[currentFrame]);

    // --- DEBUT DE LA MODIFICATION POUR L'EXERCICE ---
    // Calcul du temps écoulé pour créer une pulsation sinusoïdale
    float time = static_cast<float>(glfwGetTime());
    // std::sin renvoie une valeur entre -1 et 1.
    // On la transforme pour obtenir une valeur entre 0.0 et 1.0.
    float redIntensity = (std::sin(time * 2.0f) + 1.0f) / 2.0f;

    // On ré-enregistre le Command Buffer avec la nouvelle couleur de fond
    vkResetCommandBuffer(commandBuffers[imageIndex], 0);
    recordCommandBuffer(commandBuffers[imageIndex], imageIndex, redIntensity);
    // --- FIN DE LA MODIFICATION ---

    // 3. Soumission du Command Buffer
    VkSubmitInfo submitInfo{};
    submitInfo.sType = VK_STRUCTURE_TYPE_SUBMIT_INFO;

    VkSemaphore waitSemaphores[] = { imageAvailableSemaphores[currentFrame] };
    VkPipelineStageFlags waitStages[] = { VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT };
    submitInfo.waitSemaphoreCount = 1;
    submitInfo.pWaitSemaphores = waitSemaphores;
    submitInfo.pWaitDstStageMask = waitStages;

    submitInfo.commandBufferCount = 1;
    submitInfo.pCommandBuffers = &commandBuffers[imageIndex];

    VkSemaphore signalSemaphores[] = { renderFinishedSemaphores[currentFrame] };
    submitInfo.signalSemaphoreCount = 1;
    submitInfo.pSignalSemaphores = signalSemaphores;

    if (vkQueueSubmit(graphicsQueue, 1, &submitInfo, inFlightFences[currentFrame]) != VK_SUCCESS) {
        throw std::runtime_error("Erreur : Échec de la soumission du Command Buffer !");
    }
}

```

```

        throw std::runtime_error("Erreur : Échec de la soumission du Command Buffer !");
    }

    // 4. Présentation
    VkPresentInfoKHR presentInfo{};
    presentInfo.sType = VK_STRUCTURE_TYPE_PRESENT_INFO_KHR;
    presentInfo.waitSemaphoreCount = 1;
    presentInfo.pWaitSemaphores = signalSemaphores;

    VkSwapchainKHR swapChains[] = { swapChain };
    presentInfo.swapchainCount = 1;
    presentInfo.pSwapchains = swapChains;
    presentInfo.pImageIndices = &imageIndex;

    result = vkQueuePresentKHR(presentQueue, &presentInfo);

    if (result == VK_ERROR_OUT_OF_DATE_KHR || result == VK_SUBOPTIMAL_KHR || framebufferResized) {
        framebufferResized = false;
        recreateSwapChain();
    } else if (result != VK_SUCCESS) {
        throw std::runtime_error("Erreur : Échec de la présentation !");
    }

    // 5. Incrémentation de l'index de frame
    currentFrame = (currentFrame + 1) % MAX_FRAMES_IN_FLIGHT;
}

```

Félicitations ! En exécutant ce code, vous devriez voir un magnifique triangle blanc se dessiner sur un fond rouge qui pulse doucement au cours du temps. Vous venez de franchir l'étape la plus difficile de l'apprentissage de Vulkan : votre premier pipeline graphique complet et fonctionnel est opérationnel !

